



CAPACITAÇÃO BACK-END

Back-end, do zero ao ar

Quatro encontros construindo uma API de autenticação em Ruby on Rails: do primeiro `curl` ao deploy com Docker, Kamal e HTTPS.

Fabício Siqueira

AUTOMIC JR. · 2026

Sumário

Aula 0: Preparação

1. Se você usa Windows: instale o WSL2 primeiro
 2. Git
 3. Ruby 3.4.9 via mise
 - Instalar o mise
 - Instalar as dependências de compilação do Ruby
 - Instalar o Ruby
 - Instalar o Rails
 4. Docker
 5. VS Code e extensões
 6. Insomnia (ou Postman)
 7. Contas
 - GitHub
 - O servidor de SSH: teste isso com antecedência
 - Clone o repositório da capacitação
- Checagem final
- Como a capacitação funciona
- Se algo deu errado

Aula 1: O que é back-end, Ruby e o primeiro Rails

- Antes de tudo: como usar esta apostila
1. Os dois lados
 2. A rede, o mínimo necessário
 - O que é um servidor, afinal
 - IP e porta
 - DNS
 - Anatomia de uma URL
 3. HTTP
 - Como ela é, por dentro
 - Cabeçalhos
 - Content-Type importa
 - Os métodos
 - Os status

HTTP não tem memória

Prática 1: HTTP com as próprias mãos

4. API REST

5. Ruby, partindo do Python

5.1 O primeiro parágrafo

5.2 Variáveis e o que é "falso"

5.3 Tudo é objeto

5.4 Strings e símbolos

5.5 Coleções

5.6 Condicionais

5.7 Blocos

5.8 Métodos e argumentos nomeados

5.9 ? e !

5.10 Lidando com nil

5.11 Classes

5.12 Struct

5.13 Módulos e mixins

5.14 Exceções

5.15 Dependências

5.16 O mapa: onde cada coisa aparece

Prática 2: Ruby no irb

6. Rails

As duas leis

Rails × Django

MVC

Zeitwerk: o nome diz o caminho

Os três ambientes

7. Mão na massa

Criar o projeto

Subir o banco

Preparar e subir a aplicação

O tour das pastas

Os comandos do dia a dia

8. A primeira rota

Teste

As práticas da aula

Prática 3: Crie o seu projeto

Prática 4: Suba o banco e a aplicação

Prática 5: A sua primeira rota

Prática 6: O teste

Prática 7: Commit e GitHub

Bônus, se sobrou tempo

Recapitulando

Aula 2: Banco de dados, ActiveRecord e o model User

De onde viemos, e para onde vamos hoje

1. Por que Postgres, e não SQLite

Transação

2. ActiveRecord

ActiveSupport: os métodos que o Rails inventou

3. Migrations

schema.rb

Migrations são incrementais

4. Senha: o que nunca fazer

Hash não é criptografia

Por que bcrypt e não SHA-256

has_secure_password

5. Validações

Regra de ouro: normalize antes de validar

6. Associações

Quem guarda a chave

A migration

Os dois lados

Usando

A armadilha N+1

7. Concerns: o mixin da Aula 1, na prática

Três decisões dentro do concern, e o porquê de cada uma

8. O console

9. Seeds e fixtures

10. Testando o model

11. Uma sutileza de segurança: authenticate_by_email

As práticas da aula

Prática 1: A gem do bcrypt, e o banco de pé

Prática 2: As três migrations da tabela users

Prática 3: senha no console, vendo o bcrypt trabalhar

Prática 4: O validador de senha

Prática 5: O model User

Prática 6: A segunda tabela e a associação

Prática 7: Os dois concerns

Prática 8: Fixtures, testes e commit

Bônus, se sobrou tempo

Recapitulando

Aula 3: As rotas de autenticação

De onde viemos, e o combinado de hoje

1. O caminho de uma requisição

2. A arquitetura de pastas

Por que service object

Por que serializer

Um formato de erro só

before_action: o filtro

A pilha de middleware

Strong parameters

3. Rota 1: Cadastro

A decisão que não é óbvia

4. E-mail: o mailer

deliver_now e não deliver_later

5. Ativação da conta

6. O problema: HTTP não tem memória

JWT

O detalhe que quebra tudo

7. Rota 2: Login

O que é um cookie

XSS

CSRF

401 × 403

A mesma mensagem para os dois erros

8. Rota 3: Logout, o problema difícil

- a) Apagar no cliente
- b) Denylist por jti: revoga aquele token
- c) token_version: derruba tudo de uma vez

A verificação completa

9. Rota 4: Recuperação de senha

request sempre responde 200
confirm e a transação

10. Testando

As ferramentas de qualidade

11. CORS

12. O fluxo inteiro, no terminal

As práticas da aula

Prática 1: Traga o código e leia a arquitetura

Prática 2: Cadastro, no terminal e no Insomnia

Prática 3: O e-mail e a confirmação da conta

Prática 4: Login, e o token na mão

Prática 5: Logout que revoga de verdade

Prática 6: Recuperação de senha, e as sessões que caem

Prática 7: Testes e qualidade

Prática 8: quebre a assinatura do token (avançado)

Bônus

Recapitulando

Aula 4: Servidor, Docker, Kamal e deploy

O último dia

1. O que é colocar no ar

O problema

As quatro opções de hospedagem

Por que uma VM com containers, e não Kubernetes

Hoje: a sua máquina é o servidor

2. A sua máquina vira o servidor

Antes: duas chaves, um par

Instalar o servidor de SSH

O par de chaves da capacitação

Autorizar a si mesmo

Permissões, já que estamos aqui

3. Docker

Imagem × container

O Dockerfile do Rails

.dockerignore

Registry

O token do registry

Arquitetura

4. Kamal

config/deploy.yml, por partes

As três camadas de segredo

5. Segredos do Rails

credentials e master.key

Antes: o que é uma variável de ambiente

Credentials × ENV

O seu .env

6. HTTPS na sua máquina

HTTPS é HTTP dentro de um túnel

O handshake, em três passos

Certificado e autoridade

Proxy reverso

Gerar o certificado

.test e o /etc/hosts

Autoassinado, na prática

7. GitHub Actions: o portão

8. O primeiro deploy

9. Operar

Backup

Parar e voltar

10. E num servidor de verdade

O deploy automático

As práticas da aula

Prática 1: a sua máquina vira o servidor

Prática 2: os arquivos de deploy e o token do registry

Prática 3: o .env

Prática 4: o certificado e o /etc/hosts

Prática 5: o primeiro deploy

Prática 6: o certificado, com os olhos

Prática 7: o sistema inteiro, e o backup

Bônus, se sobrou tempo

Recapitulando

O que o seem-backend tem a mais

Apêndice: o mesmo servidor, na nuvem

O que muda

1. Criar a VM na Azure

Cotas antes de tudo

O assistente

A chave privada

Abrir o SSH só para você

Primeiro acesso

2. Provisionar o Ubuntu

Atualizar

Firewall

Docker, do repositório oficial

Root e permissões

Swap

Endurecer o SSH

E então, o deploy

3. Cloudflare e TLS

Por que uma CA pública

DNS

Certificado Origin CA

Full (strict)

Por que não Let's Encrypt

4. OIDC: deploy sem senha

Criando

5. O workflow de deploy

Cadastrando no GitHub

O primeiro deploy

6. Antes de ir embora

Recapitulando

Ruby para quem sabe Python

Sintaxe

Só em Ruby

Aspas: a diferença que Python não tem

Números: a pegadinha da divisão

Condicionais e case

Ternário e loops

puts, print e p

require × import

Splat

Estruturas de dados

Símbolos

Blocos, o coração do Ruby

Bloco, proc e lambda

Equivalências

Argumentos nomeados

O atalho do Ruby 3.1

Struct

Exceções

Atalhos de literal

Classes

? e !

Módulos e mixins

Dependências e ferramentas

Rails × Django

Consultas lado a lado

Pegadinhas de quem vem do Python

As versões que usamos

Na sua máquina

As gems do projeto

Nas imagens Docker

Serviços de fora

Conferindo tudo de uma vez

Se você estiver lendo isto no futuro

Glossário

Troubleshooting

Ambiente

mise: command not found

A instalação do Ruby falha compilando

gem install rails reclama de permissão

permission denied ao rodar docker

No Windows, docker não aparece dentro do Ubuntu

Banco

address already in use ao subir o compose

PG::ConnectionBad: could not connect to server

PG::UndefinedTable ou PendingMigrationError

Quero começar do zero

Testes

Um teste passa sozinho e falha junto com os outros

O teste de logout passa mas a revogação não funciona

Bullet::Notification::UnoptimizedQueryError

Autenticação

Sempre 401, mesmo com o token certo

O e-mail não chega em desenvolvimento

ActionController::ParameterMissing (400)

422 no cadastro sem dizer o motivo direito

GitHub

O CI ficou vermelho logo no primeiro push

gh: command not found

gh repo create reclama de autenticação

O SSH da sua máquina

Connection refused na porta 22

Funcionava, reiniciei o Windows, e parou

Permission denied (publickey)

Too many authentication failures

Host key verification failed

docker: command not found só pela SSH

Deploy

kamal config reclama de variável faltando

O push da imagem falha por cota ou armazenamento

denied ou unauthorized ao empurrar a imagem para o ghcr.io

exec format error ao subir o container

Kamal: Docker is not installed

A aplicação sobe mas não acha o banco

O curl recusa o certificado, com o erro 60

curl diz Could not resolve host: seunome.test

curl conecta mas devolve 404 do proxy

curl dá Connection refused na 443

SMTP dá timeout na VM

O deploy passa mas o site responde 500

Preciso voltar atrás agora

Apêndice: nuvem (Azure e Cloudflare)

NotAvailableForSubscription ao criar a VM

SSH dá timeout na VM da Azure

Perdi o acesso depois de mexer no sshd, e não tenho sessão aberta

OIDC: AADSTS700213

az: command not found ou falha transitória do Azure CLI

A regra temporária ficou no NSG

Cloudflare 521 / 522

Cloudflare 525 / 526

Let's Encrypt falha atrás do Cloudflare

Aula 0: Preparação

Faça isso antes do primeiro encontro. Instalar Ruby, Docker e criar contas leva tempo, e é o que mais atrasa capacitação. Se travar em algum passo, manda mensagem no grupo: não espere o dia.

Ao final você deve conseguir rodar os quatro comandos da seção [Checagem final](#).

1. Se você usa Windows: instale o WSL2 primeiro

Ruby e Rails rodam mal no Windows nativo. A forma suportada é o **WSL2** (um Linux de verdade dentro do Windows). Todo o resto deste guia você fará **dentro do WSL2**, não no PowerShell.

No PowerShell **como administrador**:

```
ws1 --install -d Ubuntu-24.04
```

Reinicie o computador. Ao abrir o Ubuntu pela primeira vez, ele pede um usuário e uma senha, anote a senha, ela é o seu `sudo`.

A partir daqui, "terminal" significa **o terminal do Ubuntu (WSL2)**.

Se você usa Linux ou macOS, pule esta seção.

2. Git

```
# Ubuntu / WSL2
sudo apt update && sudo apt install -y git curl build-essential

# macOS (instala as ferramentas de linha de comando da Apple, que incluem o git)
xcode-select --install
```

Configure seu nome e e-mail. Eles vão em todo commit que você fizer:

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
```

3. Ruby 3.4.9 via mise

O `mise` é um gerenciador de versões: ele instala a versão exata de Ruby que o projeto pede, sem mexer no Ruby do sistema. É o equivalente do `pyenv` do mundo Python.

Por que não `apt install ruby`? Porque a versão do sistema é antiga e compartilhada. Se dois projetos pedirem versões diferentes, você fica sem saída. `mise` resolve isso por projeto.

Instalar o mise

```
curl https://mise.run | sh
echo 'eval "$( ~/.local/bin/mise activate bash )"' >> ~/.bashrc
exec bash
```

Usa `zsh` (padrão do macOS)? Troque as duas ocorrências de `bash` por `zsh` e o `~/.bashrc` por `~/.zshrc`.

Instalar as dependências de compilação do Ruby

```
# Ubuntu / WSL2
sudo apt install -y autoconf patch rustc libssl-dev libyaml-dev libreadline-dev \
  zlib1g-dev libgmp-dev libncurses-dev libffi-dev libgdbm-dev libdb-dev uuid-dev \
  libpq-dev libvips

# macOS (precisa do Homebrew: https://brew.sh)
brew install openssl@3 readline libyaml gmp libpq vips
```

Instalar o Ruby

```
mise use --global ruby@3.4.9
```

Isso compila o Ruby do zero: leva de **5 a 15 minutos**. É normal.

```
ruby -v      # ruby 3.4.9 ...
gem -v
```

Instalar o Rails

```
gem install rails -v 8.1.3
rails -v      # Rails 8.1.3
```

4. Docker

O banco de dados (PostgreSQL) vai rodar dentro de um container Docker, em vez de instalado na sua máquina. Assim todo mundo tem exatamente a mesma versão, e desinstalar é apagar um container.

- **Windows/WSL2 e macOS:** instale o [Docker Desktop](#). No Windows, nas configurações, habilite a integração com a distro `Ubuntu-24.04`.
- **Linux:** siga o [guia oficial](#) e depois rode `sudo usermod -aG docker $USER` e **saia e entre de novo na sessão**.

Testando:

```
docker run --rm hello-world
```

Se aparecer "Hello from Docker!", está pronto.

5. VS Code e extensões

Baixe em code.visualstudio.com. No Windows, instale no **Windows** (não dentro do WSL): ele se conecta ao WSL sozinho.

Extensões (Ctrl+Shift+X e busque pelo nome):

Extensão	Para quê
Ruby LSP (Shopify)	Autocomplete, ir para definição, erros em tempo real
Ruby Rails (Aki)	Navegação entre model/controller/view
Docker (Microsoft)	Ver e gerenciar containers pela barra lateral
WSL (Microsoft)	<i>Só Windows:</i> abrir projetos do Ubuntu no VS Code
GitLens	Ver quem escreveu cada linha e quando
vscode-icons	Ícones por tipo de arquivo (a mesma da capacitação de front)

6. Insomnia (ou Postman)

Nossa API não tem tela: ela responde JSON. Para chamar as rotas, usaremos um cliente HTTP. Baixe o [Insomnia](#) (mais simples) ou o [Postman](#), o que preferir.

7. Contas

GitHub

Crie uma conta em github.com se ainda não tem. É só isso: a capacitação não usa nada pago do GitHub.

Instale o **GitHub CLI**, que na Aula 1 publica o seu projeto num comando:

```
# Ubuntu / WSL2
sudo apt install -y gh
# macOS
brew install gh

gh auth login      # escolha GitHub.com → HTTPS → login pelo navegador
gh auth status     # tem que dizer "Logged in to github.com"
```

Configure também o acesso por SSH ao GitHub, se ainda não tem:

```
ssh-keygen -t ed25519 -C "seu@email.com"    # Enter em tudo
gh ssh-key add ~/.ssh/id_ed25519.pub --title "meu-notebook"
ssh -T git@github.com                       # tem que cumprimentar você pelo nome
```

O servidor de SSH: teste isso com antecedência

Na Aula 4 a **sua própria máquina** vira o servidor onde a API vai rodar. Para isso ela precisa aceitar conexão SSH, e o programa que faz isso é o `openssh-server`. Você já usa o *cliente* de SSH (é o comando `ssh`); o que falta é o *servidor*.

Se você usa Windows, tudo abaixo é **dentro do WSL2**, que é o seu Linux.

```
# Ubuntu, Debian e WSL2
sudo apt update && sudo apt install -y openssh-server
sudo service ssh start

# Fedora
sudo dnf install -y openssh-server && sudo systemctl enable --now sshd
```

No **macOS** não se instala nada: ligue em **Ajustes do Sistema** → **Geral** → **Compartilhamento** → **Sessão remota**.

Confira, conectando na sua própria máquina:

```
ssh "$USER"@127.0.0.1 'echo FUNCIONOU'
```

Ele vai pedir a sua senha (na Aula 4 isso vira uma chave) e tem que responder `FUNCIONOU`. **Mande o resultado no grupo**. Dez minutos hoje valem a aula inteira.

WSL2: o `sshd` não sobe sozinho a cada boot do Windows, a não ser que você habilite o systemd. Se um dia o `ssh` parar de conectar do nada, é isso, e a saída é `sudo service ssh start`.

Clone o repositório da capacitação

Você vai usá-lo o curso inteiro, para consultar o código pronto:

```
git clone <url-do-repositório> ~/capacitacao-gabarito
```

Checagem final

Cole os comandos no terminal. Se todos responderem, você está pronto:

```
ruby -v           # ruby 3.4.9
rails -v          # Rails 8.1.3
docker -v         # Docker version 2x.x.x
git --version
gh auth status    # Logged in to github.com
ssh "$USER"@127.0.0.1 'echo ok' # ok (é o servidor da Aula 4)
```

Como a capacitação funciona

Você vai trabalhar em **dois diretórios**:

```
~/capacitacao-gabarito/  ← o repositório da capacitação. Só leitura.
~/automic_auth_api/      ← o SEU projeto. Criado na Aula 1, é onde você escreve.
```

O gabarito você já clonou, alguns passos acima:

```
ls ~/capacitacao-gabarito # se não existir:
# git clone <url-do-repositório> ~/capacitacao-gabarito
```

O seu projeto você cria no primeiro encontro, com o passo a passo de `01-fundamentos.md`. Ele precisa ficar no **seu** GitHub: na Aula 4 é de lá que o CI roda e é para a sua conta que a imagem Docker vai.

Se algo deu errado

Sintoma	Provável causa e saída
<code>mise: command not found</code>	A linha do <code>activate</code> não foi para o arquivo certo. Confira <code>~/.bashrc</code> (ou <code>~/.zshrc</code>) e rode <code>exec bash</code> .
A instalação do Ruby falha com erro de compilação	Faltou alguma dependência do passo 3. Rode o <code>apt install / brew install</code> de novo e tente <code>mise install ruby@3.4.9</code> outra vez.
<code>permission denied</code> no <code>docker</code>	Você não está no grupo <code>docker</code> . Rode <code>sudo usermod -aG docker \$USER</code> e saia e entre de novo (só reabrir o terminal não basta).
No Windows, o <code>docker</code> não aparece dentro do Ubuntu	Docker Desktop → Settings → Resources → WSL Integration → habilite a distro <code>Ubuntu-24.04</code> .
<code>ssh \$USER@127.0.0.1</code> dá <code>Connection refused</code>	O <code>sshd</code> não está rodando: <code>sudo service ssh start</code> . No macOS, ligue a Sessão remota.
<code>gem install rails</code> reclama de permissão	Você está usando o Ruby do sistema, não o do <code>mise</code> . Confira com <code>which ruby</code> : deve apontar para dentro de <code>~/.local/share/mise</code> .

Mais casos em `troubleshooting.md`.

Aula 1: O que é back-end, Ruby e o primeiro Rails

Você sai daqui com: uma API respondendo `GET /api/v1/status`. **Gabarito:** `cd ~/capacitacao-gabarito && git checkout aula-01`

Pré-requisitos instalados em `00-preparacao.md`.

Antes de tudo: como usar esta apostila

Você não precisa entender tudo hoje. **Precisa fazer funcionar hoje:** o entendimento vem completando os buracos nas próximas semanas, e é assim mesmo que se aprende back-end.

Três coisas que vale saber antes de começar:

Você vai travar. Não é sinal de que você não serve para isso; é o trabalho. A diferença entre quem começou ontem e quem faz isso há dez anos não é travar menos: é travar melhor, e ler a mensagem de erro em vez de entrar em pânico. Toda prática desta apostila termina com uma tabela **Se der errado**, com os erros que acontecem de verdade. Ela existe exatamente para isso.

As práticas são a aula. Os textos entre elas explicam o porquê; as práticas é que fixam. Se o tempo apertar e você tiver que escolher, escolha fazer.

Perguntar é rápido. Se você travou por mais de dez minutos no mesmo ponto, pergunte. Ninguém vai achar a pergunta boba, e quem está do seu lado provavelmente está no mesmo lugar, calado.

Uma nota sobre o tom: esta apostila afirma as coisas com bastante convicção, porque explicar com mil ressalvas fica ilegível. Mas quase toda decisão aqui tem alternativa razoável. Quando você discordar de alguma, **pergunta:** essa conversa costuma valer mais que o parágrafo.

1. Os dois lados

Na capacitação de front-end você aprendeu a fazer o que o usuário vê. Back-end é o outro lado.

Front-end	Back-end
A parte visível, que o usuário toca	A parte lógica, nos bastidores
HTML, CSS, JavaScript	Ruby, Python, Java, Go, Node
Roda no navegador ou no celular	Roda num servidor, longe do usuário
Se cair, a tela quebra	Se cair, nada funciona

O back-end guarda os dados, decide quem pode ver o quê, aplica as regras de negócio, autentica usuários e conversa com outros sistemas (e-mail, pagamento, mapas).

Por que isso não pode viver no front? Porque tudo que roda no navegador o usuário consegue ler e alterar. A validação de senha no front é conveniência; a que vale é a do back.

A analogia: o salão do restaurante é o front-end. Mesa, cardápio, garçom. A cozinha é o back-end: ninguém entra, mas é lá que a comida sai. O pedido é a requisição, o prato é a resposta. Você não pede pra ver a receita; você pede o prato.

2. A rede, o mínimo necessário

O que é um servidor, afinal

Não é uma máquina especial. É um **programa esperando**: ele fica ligado, escutando numa porta, e responde ao que chegar ali. `bin/rails server` sobe esse programa na porta 3000: na sua máquina ou numa VM na nuvem, é o mesmo programa.

IP e porta

IP é o endereço da máquina (`57.156.65.151`). **Porta** é a sala dentro dela: um número de 1 a 65535. Uma máquina tem milhares de portas, e um programa diferente pode escutar em cada uma.

Porta	Quem mora ali
22	SSH
80	HTTP
443	HTTPS
5432	Postgres
3000	Rails em desenvolvimento

`127.0.0.1` (o `localhost`) significa sempre "esta máquina aqui". Na Aula 4 ele vira o endereço do seu servidor, e a sua API passa a atender na 443.

DNS

Ninguém decora `57.156.65.151`. O **DNS** é a agenda telefônica da internet: você digita `api.seemxxiii.tech` e alguém pergunta ao DNS qual é o IP.

A resposta fica em cache por um tempo: o **TTL**. É por isso que apontar um domínio para outro servidor não vale na hora. Na Aula 4 você faz isso na mão, no `/etc/hosts`, que é o atalho local para a mesma ideia.

Anatomia de uma URL

```
https :// api.seemxxiii.tech : 443 /api/v1/lectures ?page=2 #topo
|         |           |         |         |         |
esquema  host      porta  caminho  query  fragmento
```

- **Esquema:** o protocolo. `http`, `https`, `ssh`, `postgres`.
- **Porta:** opcional. `http` assume 80, `https` assume 443.
- **Fragmento:** nunca vai para o servidor; é só para o navegador.

É por isso que `localhost:3000` tem os dois pontos: a porta não é a padrão.

3. HTTP

Cliente é quem pede (navegador, app, outro servidor). Servidor é quem responde. **O cliente sempre começa a conversa:** o servidor nunca liga primeiro.

Uma **requisição** tem método, caminho, cabeçalhos e corpo. Uma **resposta** tem status, cabeçalhos e corpo.

Como ela é, por dentro

```
POST /api/v1/sessions HTTP/1.1      ← método, caminho, versão
Host: api.seemxxiii.tech             ↴
Content-Type: application/json        | cabeçalhos
Accept: application/json              ↴
                                     ← linha em branco separa
{"email":"ana@ufop.br","password":"..."} ← corpo
```

É **texto puro**. HTTP é literalmente isso trafegando num cano.

Cabeçalhos

São **metadados**: informação *sobre* a mensagem, não a mensagem. Um par `chave: valor` por linha.

Cabeçalho	Para quê
Content-Type	Em que formato vai o corpo
Accept	Em que formato eu quero a resposta
Authorization	Quem sou eu (vai ser o nosso token, na Aula 3)
Set-Cookie / Cookie	Como o navegador guarda estado
User-Agent	Que programa está chamando

Content-Type importa

O mesmo dado, dois formatos:

```
application/json      → {"email":"ana@ufop.br"}
application/x-www-form-urlencoded → email=ana%40ufop.br
```

Sem o cabeçalho, o servidor não sabe como ler o corpo. Esquecer isso no `curl` é o erro nº 1 de quem começa: vem `400` e a mensagem não ajuda em nada. Por isso todo `curl` deste material tem `-H 'Content-Type: application/json'`.

Os métodos

Método	Significa
GET	Me dá isso. Não muda nada.
POST	Cria isso.
PUT / PATCH	Altera isso.
DELETE	Apaga isso.

O método é uma promessa. Um `GET` que apaga registro é bug, não criatividade.

Os status

Faixa	Significa	Exemplos
2xx	Deu certo	<code>200</code> ok, <code>201</code> criado
3xx	Foi pra outro lugar	<code>301</code> , <code>302</code>
4xx	Você errou	<code>400</code> pedido mal feito, <code>401</code> não autenticado, <code>403</code> sem permissão, <code>404</code> não existe, <code>422</code> dado inválido
5xx	Nós erramos	<code>500</code> bug nosso

`401` é "quem é você?". `403` é "sei quem você é, e você não pode". Vamos usar os dois na Aula 3.

HTTP não tem memória

Cada requisição começa do zero. O servidor esquece tudo entre uma e outra.

Guarde essa frase. Ela é o problema inteiro da Aula 3: se o servidor esquece tudo, como ele sabe que você já fez login?

Prática 1: HTTP com as próprias mãos

É a primeira vez que você fala com um servidor sem navegador no meio. Faça comando por comando, e **leia a saída** de cada um antes de passar para o próximo.

a) A requisição mais simples que existe

```
curl -i https://api.seemxxiii.tech/api/v1/status
```

```
HTTP/2 200
content-type: application/json; charset=utf-8

{"status":"ok","service":"seem-backend"}
```

O `-i` manda o `curl` mostrar os **cabeçalhos** junto com o corpo. Isso é uma API real, no ar: é exatamente o que você vai construir.

b) Veja a requisição inteira, dos dois lados

```
curl -v https://api.seemxxiii.tech/api/v1/status
```

Linhas com `>` são o que **você mandou**; com `<`, o que o **servidor respondeu**. Ache no meio:

- a linha `> GET /api/v1/status HTTP/2`: o método e o caminho
- o cabeçalho `> user-agent: curl/...`: quem você disse que era
- a linha `< HTTP/2 200`: o status

c) Provoque um erro de propósito

Antes de rodar, responda para você: o servidor vai responder alguma coisa, ou a conexão vai falhar? Responda de verdade, mentalmente, e só depois rode. Prever antes de observar é o que transforma "vi acontecer" em "entendi".

```
curl -i https://api.seemxxiii.tech/api/v1/rota-que-nao-existe
```

Veio `404`. Repare que o servidor **respondeu**: 404 não é "deu erro de conexão", é uma resposta perfeitamente bem-sucedida dizendo "esse recurso não existe".

d) Só os cabeçalhos

```
curl -I https://api.seemxxiii.tech/api/v1/status
```

`-I` faz um `HEAD`: pede só os cabeçalhos, sem o corpo. É como um monitoramento checa se o site está de pé sem baixar a página inteira.

e) Avançado: mande um POST e veja o `Content-Type` importar

```
curl -i -X POST https://api.seemxxiii.tech/api/v1/status \
-H "Content-Type: application/json" \
-d '{"oi": true}'
```

A rota de status só aceita `GET`, então você vai levar um `404` ou `405`. **O ponto não é o sucesso** é você ter montado uma requisição com método, cabeçalho e corpo na mão, que é o que o Insomnia faz por baixo.

Confere: você consegue apontar, na saída do `-v`, onde termina a sua requisição e onde começa a resposta. E consegue dizer o que significa cada um dos três números que viu: 200, 404 e o do POST.

Se der errado

Erro	Causa	Saída
<code>curl: command not found</code>	o <code>curl</code> não está instalado	<code>sudo apt install curl</code> (ou <code>brew install curl</code>)
<code>Could not resolve host</code>	sem internet, ou o nome está errado	teste <code>curl -i https://example.com</code>
<code>Connection refused</code>	o servidor não está no ar	avise no grupo: pode ser a API que caiu, e não você
<code>curl: (60) SSL certificate problem</code>	relógio da máquina errado, ou proxy da rede	confira a data do sistema; na Aula 4 você vai entender esse erro por dentro
a saída sai toda numa linha só, ilegível	faltou o <code>-i</code> , ou o JSON não tem quebra	acrescente <code>\ python3 -m json.tool</code> no fim

4. API REST

API é a porta de entrada do sistema para outros programas. **REST** é um estilo de organizar essa porta: cada coisa do sistema é um **recurso**, com um endereço.

```
/api/v1/lectures    → as palestras
/api/v1/lectures/7  → a palestra 7
```

O que você faz com o recurso é o **verbo**, não o endereço:

GET	/api/v1/lectures	lista
POST	/api/v1/lectures	cria
GET	/api/v1/lectures/7	mostra uma
DELETE	/api/v1/lectures/7	apaga

`/criarPalestra` · `POST /lectures`

O `v1` no caminho é a versão. Quando a API mudar de forma incompatível, nasce uma `/v2` e a `/v1` continua funcionando, porque o app na loja do celular demora dias para atualizar.

5. Ruby, partindo do Python

Ruby foi criado por **Yukihiro Matsumoto (Matz)** em 1995, no Japão, com um objetivo declarado: *fazer o programador feliz*. É interpretada, dinâmica e orientada a objetos, como Python.

Esta seção é a **referência de sintaxe da capacitação**. Não é a linguagem inteira: é exatamente o que aparece nas quatro aulas, na ordem em que aparece, e cada bloco diz **onde no projeto você vai encontrar aquilo**. A tabela lado a lado mais completa está em `ruby-para-pythonistas.md`.

Abra um `irb` numa aba do terminal e vá colando. Ler sintaxe não fixa; digitar fixa.

5.1 O primeiro parágrafo

<pre># Python def saudacao(nome, formal=False): if formal: return f"Prezado {nome}" return f"Oi, {nome}"</pre>	<pre># Ruby def saudacao(nome, formal: false) return "Prezado #{nome}" if formal "Oi, #{nome}" end</pre>
--	--

Quatro coisas para reparar, e são elas que fazem Ruby parecer estranho no primeiro dia:

1. `end` fecha o bloco, em vez da indentação.
2. `if` **no fim da linha**: é o *modificador*, e é muito usado.
3. **A última linha avaliada é o retorno.** `return` só se você quiser sair antes.
4. `#{}` interpola, como o `f"..."` do Python.

Não há ponto e vírgula, e parênteses na chamada são opcionais: `puts "oi"` e `puts("oi")` são a mesma coisa. O projeto usa parênteses quando há argumento e omite quando não há.

5.2 Variáveis e o que é "falso"

<code>nome = "Ana"</code>	<code># variável local</code>
<code>@nome = "Ana"</code>	<code># variável de instância (do objeto)</code>
<code>NOME = "Ana"</code>	<code># constante (maiúscula)</code>

O `@` é o `self` do Python, só que sem precisar declarar no `__init__`. Dentro de uma classe, `@email` é o atributo daquele objeto.

A pegadinha número um de quem vem do Python:

```
if 0      then puts "entrou" end  # ENTRA. 0 é verdadeiro em Ruby.
if ""     then puts "entrou" end  # ENTRA. string vazia é verdadeira.
if []     then puts "entrou" end  # ENTRA. array vazio é verdadeiro.
if nil    then puts "entrou" end  # não entra
if false  then puts "entrou" end  # não entra
```

Só `nil` e `false` são falsos. Todo o resto é verdadeiro. Em Python, `0`, `""`, `[]` e `{}` são todos falsos: aqui, não.

É por isso que no projeto você vai ver `if params[:email].present?` e não `if params[:email]`: a string vazia passaria pela segunda.

	Ruby	Python
ausência de valor	<code>nil</code>	<code>None</code>
vazio conta como falso?	não	sim
testar "tem conteúdo?"	<code>.present?</code> (Rails)	<code>if x</code>
testar "está vazio?"	<code>.blank?</code> (Rails)	<code>if not x</code>

5.3 Tudo é objeto

Em Python você chama `len(x)`. Em Ruby, `x.length`. **Não existe função solta**: é sempre método de alguém. Até número é objeto.

```
3.times { puts "oi" }
-5.abs      # 5
"ana".capitalize # "Ana"
nil.to_a    # []
1.class     # Integer
```

Isso muda como você lê o código: quando vir `user.email.strip.downcase`, leia da esquerda para a direita, cada método operando no resultado do anterior.

5.4 Strings e símbolos

```
'texto simples'      # aspas simples: não interpola
"olá, #{nome}"       # aspas duplas: interpola
"linha\n"            # aspas duplas: entende \n
'linha\n'             # aspas simples: dois caracteres, \ e n
```

Use aspas simples quando não há nada para interpolar: é a convenção, e o RuboCop cobra.

Símbolo é um texto que serve de **etiqueta**, não de conteúdo:

```
:aluno :email :admin
```

O mesmo símbolo é sempre o mesmo objeto na memória; duas strings iguais são dois objetos. Não existe em Python. Lá você usaria uma string.

Regra prática: conteúdo que o usuário lê → string. Nome de coisa no código → símbolo.

No projeto, símbolo aparece em chave de hash (`params[:email]`), em nome de validação (`validates :email, presence: true`) e em nome de callback (`before_action :autenticar!`).

5.5 Coleções

```
# Array – a lista do Python
nomes = ["ana", "bia", "caio"]
nomes[0]      # "ana"
nomes[-1]     # "caio"
nomes << "davi" # append
nomes.first   # "ana"
nomes.size    # 4

# Hash – o dict do Python
user = { nome: "Ana", role: :admin }
user[:nome]   # "Ana"
user[:idade]  # nil (NÃO estoura, diferente do dict do Python)
user.fetch(:idade) # aí sim: KeyError
user.fetch(:idade, 0) # 0
```

Repare em `{ nome: "Ana" }`: é o atalho para `{ :nome => "Ana" }`. **A chave é um símbolo**, não uma string, e `user["nome"]` devolveria `nil`. Esse é o erro mais comum da primeira semana.

`fetch` é importante: ele **falha alto** quando a chave não existe. É por isso que o `config/deploy.yml` da Aula 4 usa `ENV.fetch("SERVER_IP")`: se a variável não estiver definida, você descobre na hora, e não três passos depois.

5.6 Condicionais

```
if idade >= 18
  puts "maior"
elsif idade >= 16
  puts "vota"
else
  puts "menor"
end

unless ativo?      # o "if not" do Ruby
  puts "inativo"
end

puts "maior" if idade >= 18      # modificador: cabe numa linha
puts "inativo" unless ativo?

status = idade >= 18 ? "maior" : "menor"    # ternário, igual ao C
```

E o `case`, que aceita mais coisa que o do Python:

```
case status
when "ok"          then 200
when "not_found"   then 404
when 400..499      then "erro do cliente"    # aceita intervalo
else               500
end
```

5.7 Blocos

Um bloco é um pedaço de código que você entrega para um método executar. É o `lambda` do Python, mas usado o tempo todo, e é a construção mais característica da linguagem.

```
usuarios.each do |u|          # for u in usuarios
  puts u.nome
end

usuarios.map { |u| u.nome }    # [u.nome for u in usuarios]
usuarios.select { |u| u.ativo? } # [u for u in usuarios if u.ativo]
usuarios.reject { |u| u.ativo? } # o inverso
usuarios.find { |u| u.admin? } # o primeiro que casar, ou nil
usuarios.any? { |u| u.admin? } # true/false
usuarios.count { |u| u.admin? } # quantos
```

Chaves quando cabe numa linha, `do ... end` quando não cabe. É convenção, e o RuboCop cobra.

O que está entre `{ | | }` são os parâmetros do bloco. Com dois:

```
{ a: 1, b: 2 }.each { |chave, valor| puts "#{chave}=# {valor}" }
```

Blocos também servem para **delimitar um trecho**, e é assim que o projeto usa transação:

```
User.transaction do
  user.save!
  LoginEvent.create!(user: user)
end
```

Tudo dentro do bloco acontece, ou nada acontece.

5.8 Métodos e argumentos nomeados

```
def somar(a, b)          # posicionais
  a + b
end

def login(email:, password:, client: "mobile") # nomeados
  # ...
end

login(email: "a@b.c", password: "x")
```

Os dois-pontos **depois** do nome (`email:`) tornam obrigatório nomear na chamada. Sem valor padrão, o argumento é obrigatório; com valor padrão (`client: "mobile"`), é opcional.

O projeto usa nomeados em todo service, de propósito: `Users::Register.call(email:, password:)` se lê sozinho, e `Users::Register.call("a@b.c", "x")` não.

O atalho do Ruby 3.1: quando a variável tem o mesmo nome da chave, você omite o valor.

```
user = User.first
Result.new(success?: true, user:) # é o mesmo que user: user
```

Isso aparece em todo service do projeto. **Não é erro de digitação.**

5.9 ? e !

Sufixo	Significa	Exemplo
?	Devolve verdadeiro ou falso	<code>user.admin?</code> , <code>email.blank?</code>
!	A versão perigosa: estoura erro em vez de devolver <code>false</code>	<code>user.save!</code>

É convenção, não regra da linguagem. Mas todo mundo segue, e o Rails também:

```
user.save # false se as validações falharem – você tem que checar
user.save! # levanta ActiveRecord::RecordInvalid
```

No projeto, dentro de uma transação usamos sempre a versão com `!`: se falhar no meio, tem que estourar para a transação desfazer tudo. Um `save` silencioso ali deixaria o banco pela metade.

5.10 Lidando com `nil`

Três atalhos que aparecem o tempo todo, e que economizam muito `if`:

```
user&.email      # se user for nil, devolve nil em vez de estourar
                  # (é o "safe navigation")

@lista ||= []    # atribui SÓ se estiver nil ou false
                  # ("ou-igual": o padrão de memoização do Ruby)

nome = params[:nome] || "anônimo"  # valor padrão
```

Sem o `&.`, `user.email` com `user` valendo `nil` levanta `NoMethodError: undefined method 'email' for nil`. **Esse é o erro que você mais vai ver**, e quando vir, a pergunta certa é "quem aqui virou `nil`?".

E dois métodos que o Rails acrescenta e o Ruby puro não tem:

```
".blank?      # true    - nil, "", " ", [], e {} são todos "blank"
".present?    # false
"ana".present? # true
```

5.11 Classes

```
class Usuario
  attr_reader :email      # cria o método usuario.email
  attr_accessor :nome     # cria o .nome e o .nome=

  def initialize(email:, nome: nil)
    @email = email
    @nome = nome
  end

  def admin?
    @email.end_with?("@automic.com")
  end

  private                 # daqui para baixo, só de dentro do objeto

  def normalizar
    @email = @email.strip.downcase
  end
end

u = Usuario.new(email: "ANA@x.com ")
u.email      # "ANA@x.com "
u.admin?     # false
```

`initialize` é o `__init__`. `attr_reader` gera o getter, e evita escrever um método de uma linha para cada atributo. E `private` vale **da linha em diante**, não por método.

Método de classe (o `@staticmethod` do Python) leva `self.`:

```
class Users::Register
  def self.call(email:, password:)
    new(email:, password:).call
  end
end
```

Todo service do projeto tem exatamente esse formato: um `self.call` que instancia e chama.

5.12 Struct

Quando você só quer agrupar valores, sem comportamento:

```
Result = Struct.new(:success?, :user, :errors, keyword_init: true)

r = Result.new(success?: true, user: ana, errors: [])
r.success?    # true
r.user        # ana
```

É o `namedtuple` / `dataclass` do Python. **Todo service deste projeto devolve um desses:** é o que permite ao controller escrever `if resultado.success?` sem saber nada de dentro do service.

5.13 Módulos e mixins

Ruby não tem herança múltipla. Tem **módulo**: um saco de métodos que você inclui numa classe.

```
module EmailVerifiable
  def gerar_codigo_de_verificacao
    # ...
  end
end

class User < ApplicationRecord
  include EmailVerifiable
  include PasswordResettable
end
```

Vamos escrever esses dois na Aula 2. O nome deles no mundo Rails é **concern**, e o motivo de existirem é manter o `User` legível: sem eles, o model teria 200 linhas misturando três assuntos.

Módulo também serve de **namespace**, e é assim que o projeto organiza as pastas:

```
module Users
  class Register          # a classe é Users::Register
  end                    # e o arquivo é app/services/users/register.rb
end
```

5.14 Exceções

Python	Ruby
<code>try</code>	<code>begin</code>
<code>except</code>	<code>rescue</code>
<code>finally</code>	<code>ensure</code>
<code>raise</code>	<code>raise</code>
base para capturar	<code>StandardError</code> (não <code>Exception</code>)

```
def call
  arriscado
rescue StandardError => e
  Rails.error.report(e)
  nil
end
```

Dentro de um método o `begin` é implícito: dá para escrever só o `rescue` no fim. Criando o seu tipo de erro:

```
class InvalidToken < StandardError; end

raise InvalidToken if token_ruim?
```

Herde sempre de `StandardError`, nunca de `Exception`: capturar `Exception` pega até `Ctrl+C`.

5.15 Dependências

Python	Ruby
<code>pip install</code>	<code>gem install</code>
<code>requirements.txt</code>	<code>Gemfile</code>
<code>venv</code> por projeto	<code>bundler</code> resolve tudo
<code>pip freeze</code> para travar	<code>Gemfile.lock</code> , gerado sozinho
<code>python script.py</code>	<code>ruby script.rb</code>
<code>python -m pytest</code>	<code>bundle exec rspec</code> / <code>bin/rails test</code>

O `Gemfile.lock` é o `requirements.txt` travado: só que automático e **sempre versionado**. Ele é a garantia de que a sua máquina e o servidor rodam exatamente as mesmas versões.

E o `bundle exec` na frente do comando significa "rode usando as versões do `Gemfile.lock`", não as que estiverem soltas na máquina".

5.16 O mapa: onde cada coisa aparece

Sintaxe	Onde você vai encontrar
símbolo, hash	<code>params[:email]</code> , validações (Aula 2 e 3)
bloco	<code>each</code> , <code>map</code> , <code>User.transaction do</code> (Aula 2)
<code>?</code> e <code>!</code>	<code>user.save!</code> , <code>valid?</code> (Aula 2)
<code>&.</code> e <code>\ \ =</code>	tratamento de <code>nil</code> nos controllers (Aula 3)
argumento nomeado + atalho 3.1	todo service (Aula 3)
<code>Struct</code>	o retorno de todo service (Aula 3)
módulo / <code>include</code>	os concerns do <code>User</code> (Aula 2)
namespace com módulo	<code>Api::V1::SessionsController</code> (Aula 3)
<code>rescue</code>	tratamento de erro da API (Aula 3)
<code>ENV.fetch</code>	<code>config/deploy.yml</code> (Aula 4)

Prática 2: Ruby no `irb`

Não pule esta: é a única vez na capacitação em que você mexe em Ruby sem Rails no caminho, e é aqui que a sintaxe entra.

Abra o console interativo:

```
irb
```

a) Tudo é objeto

```
3.class      # Integer
"oi".class   # String
:oi.class    # Symbol
nil.class    # NilClass
3.times { |i| puts i }
```

b) A pegadinha do "falso"

```
puts "entrou" if 0      # entra! 0 é verdadeiro
puts "entrou" if ""     # entra!
puts "entrou" if nil    # não entra
```

Se você veio do Python, pare cinco segundos aqui.

c) Hash e símbolo

```

usuario = { nome: "Ana", role: :admin }
usuario[:nome]      # "Ana"
usuario["nome"]     # nil  <- por quê?
usuario[:idade]     # nil
usuario.fetch(:idade) # KeyError

```

Responda para você mesmo por que `usuario["nome"]` deu `nil`.

d) Blocos

```

[1, 2, 3].map { |n| n * n }
[1, 2, 3].select { |n| n.odd? }
%w[ana bia caio].each { |n| puts n.capitalize }

```

`%w[...]` é o atalho para array de strings: aparece bastante em código Rails.

e) Nil, e como não apanhar dele

```

user = nil
user.email      # NoMethodError – leia a mensagem inteira
user&.email     # nil, sem estourar
nome = user&.email || "anônimo"

```

f) Um método, com nomeados e retorno implícito

```

def saudacao(nome:, formal: false)
  return "Prezado #{nome}" if formal
  "Oi, #{nome}"
end

saudacao(nome: "Ana")
saudacao(nome: "Ana", formal: true)
saudacao("Ana")      # ArgumentError – leia a mensagem

```

g) Um `Struct`, que é o que todo service vai devolver

```

Result = Struct.new(:success?, :user, :errors, keyword_init: true)
r = Result.new(success?: true, user: "ana", errors: [])
r.success?
r.errors

```

h) Avançado: escreva um módulo e inclua numa classe

Faça isto sem olhar a seção 5.13, e depois confira.

```

module Saudavel
  def cumprimentar = "Oi, #{nome}"
end

class Pessoa
  include Saudavel
  attr_reader :nome
  def initialize(nome) = @nome = nome
end

Pessoa.new("Ana").cumprimentar

```

O `def metodo = expressão` é o *endless method*, do Ruby 3. Serve para método de uma linha só.

Confere: você conseguiu explicar, em voz alta, por que `usuario["nome"]` deu `nil` e por que `saudacao("Ana")` deu `ArgumentError`. Se conseguiu, a sintaxe entrou.

Se der errado

Erro	Causa	Saída
<code>irb: command not found</code>	o <code>mise</code> não está ativo no shell	<code>exec bash</code> e confira com <code>which ruby</code>
<code>NameError: undefined local variable</code>	você digitou o nome errado, ou perdeu o <code>@</code>	releia a linha; Ruby não avisa antes de rodar
<code>NoMethodError: undefined method 'x' for nil</code>	alguma coisa virou <code>nil</code> antes	a pergunta certa é "quem virou <code>nil</code> ?", não "o que é esse método"
<code>ArgumentError: missing keyword: :nome</code>	o método pede argumento nomeado e você passou posicional	<code>saudacao(nome: "Ana")</code>
<code>syntax error, unexpected end-of-input</code>	faltou um <code>end</code>	conte os <code>def</code> / <code>do</code> / <code>if</code> abertos

6. Rails

Framework web em Ruby, criado por **David Heinemeier Hansson** em 2004: extraído do Basecamp, um produto real. Traz tudo junto: rotas, banco, e-mail, filas, testes, deploy. Estamos na versão 8.

As duas leis

Convenção sobre configuração. Você não configura o óbvio, você segue o combinado. Classe `User`? Então a tabela é `users`, o arquivo é `user.rb`, a chave primária é `id`. Ninguém precisa dizer. Seguindo a convenção você escreve quase nada; brigando com ela, o dobro.

DRY: *Don't Repeat Yourself*. Cada regra existe num lugar só.

E uma atitude: **omakase**, "deixa comigo, chef". O Rails já escolheu as ferramentas por você. Você pode trocar, mas só troque quando tiver um motivo real. Isso é liberdade: sobra tempo pro problema de verdade.

Rails × Django

Django	Rails
<code>models.py</code>	<code>app/models/</code> , um arquivo por classe
<code>makemigrations</code> / <code>migrate</code>	<code>rails g migration</code> / <code>db:migrate</code>
<code>urls.py</code>	<code>config/routes.rb</code>
<code>views.py</code>	<code>app/controllers/</code>
Django ORM	ActiveRecord
<code>manage.py</code>	<code>bin/rails</code>

Quem vem de Flask ou FastAPI vai sentir mais diferença: lá você monta cada peça, aqui elas já vêm encaixadas.

MVC

- **Model:** os dados e as regras. Conversa com o banco.
- **View:** a tela. Na nossa API, é o JSON.
- **Controller:** recebe a requisição e decide o que fazer.
- **Rota:** o mapa. Este endereço vai para aquele controller.

O caminho é sempre: **rota** → **controller** → **model** → **resposta**.

Zeitwerk: o nome diz o caminho

Você nunca escreve `require` no Rails. Por quê?

O **Zeitwerk** carrega a classe no instante em que você a menciona, e descobre o arquivo pelo **nome da classe**:

```
Api::V1::StatusController → app/controllers/api/v1/status_controller.rb
```

Errou o caminho, a classe simplesmente não existe. Não é questão de estilo: é o mecanismo que faz "convenção sobre configuração" funcionar de verdade.

Os três ambientes

Ambiente	Onde	Como se comporta
<code>development</code>	sua máquina	recarrega o código a cada requisição, log verboso, erro na tela
<code>test</code>	os testes	banco separado, limpo a cada teste
<code>production</code>	o servidor	código congelado, log enxuto, erro genérico

Cada um tem um arquivo em `config/environments/`. `Rails.env` diz onde você está.

É assim que o e-mail abre no navegador em desenvolvimento e sai por SMTP em produção: mesmo código, ambientes diferentes.

7. Mão na massa

Criar o projeto

```
rails new automic_auth_api --api -d postgresql \
  --skip-action-mailbox --skip-action-text --skip-active-storage \
  --skip-jbuilder --skip-action-cable
cd automic_auth_api
```

- `--api`: sem HTML, sem CSS, sem sessão de navegador. Só JSON.
- `-d postgresql`: o mesmo banco que usamos em produção.
- Os `--skip` tiram peças que não vamos usar. Cada peça a menos é uma peça a menos para dar problema.

Subir o banco

Crie o `compose.yaml` (ou copie o deste repositório) e rode:

```
docker compose up -d
docker compose ps          # tem que aparecer "healthy"
```

Se der `address already in use`, você já tem um Postgres na máquina ocupando a 5432. Rode `DB_PORT=5433 docker compose up -d` e exporte `DB_PORT=5433` no shell.

Preparar e subir a aplicação

```
bin/rails db:prepare
bin/rails server
```

Abra `http://localhost:3000/up`. Verde é a aplicação de pé.

O tour das pastas

Pasta	O que tem
<code>app/</code>	O seu código. É onde você passa 90% do tempo.
<code>config/</code>	Rotas, banco, ambientes, credenciais
<code>db/</code>	Migrations e o retrato atual do banco (<code>schema.rb</code>)
<code>test/</code>	Os testes
<code>bin/</code>	Os comandos: <code>rails</code> , <code>setup</code> , <code>dev</code>
<code>Gemfile</code>	As dependências

Os comandos do dia a dia

```
bin/rails server      # sobe a aplicação
bin/rails console     # um irb com o seu projeto carregado dentro
bin/rails routes      # todas as rotas que existem
bin/rails generate    # cria arquivos seguindo a convenção
bin/rails db:migrate  # aplica as mudanças de banco
bin/rails test        # roda os testes
```

O **console** é a ferramenta mais subestimada do Rails: dá para criar usuário, rodar query e testar método sem escrever um arquivo.

8. A primeira rota

`config/routes.rb`:

```
Rails.application.routes.draw do
  get "up" => "rails/health#show", as: :rails_health_check

  namespace :api do
    namespace :v1 do
      get "status", to: "status#show"
    end
  end
end
```

`app/controllers/api/v1/status_controller.rb`:

```

module Api
  module V1
    class StatusController < ApplicationController
      def show
        render json: {
          status: "ok",
          service: "atomic-auth-api",
          environment: Rails.env
        }
      end
    end
  end
end

```

Repare na convenção: `namespace :api` + `namespace :v1` + `status#show` implica exatamente o arquivo `app/controllers/api/v1/status_controller.rb`, classe `Api::V1::StatusController`, método `show`. Ninguém configurou isso.

Teste

`test/controllers/api/v1/status_controller_test.rb`:

```

require "test_helper"

class Api::V1::StatusControllerTest < ActionDispatch::IntegrationTest
  test "responde ok" do
    get "/api/v1/status"

    assert_response :ok
    assert_equal "ok", response.parsed_body["status"]
  end
end

```

```
bin/rails test
```

As práticas da aula

Sete práticas, em ordem crescente. As duas primeiras você já fez no meio da aula:

#	O que	Onde
1	HTTP com as próprias mãos	seção 3
2	Ruby no <code>irb</code>	seção 5
3	Crie o seu projeto	aqui
4	Suba o banco e a aplicação	aqui
5	A sua primeira rota	aqui
6	O teste	aqui
7	Commit e GitHub	aqui

Onde você trabalha: no **seu** projeto, que você cria na Prática 3. O repositório da capacitação é o **gabarito**: abra para consultar, não para escrever.

Prática 3: Crie o seu projeto

```
cd ~
rails new automic_auth_api --api -d postgresql \
  --skip-action-mailbox --skip-action-text --skip-active-storage \
  --skip-jbuilder --skip-action-cable
cd automic_auth_api
```

Cada flag tira uma parte do Rails que este projeto não usa. `--api` é a mais importante: gera um Rails sem views e sem os middlewares de navegador.

Confere:

```
ls # tem app/, config/, db/, test/
cat Gemfile | head -5 # a linha do rails, com a versão
bin/rails -v # Rails 8.1.x
```

Se der errado

Erro	Causa	Saída
<code>rails: command not found</code>	o <code>mise</code> não está ativo neste shell	<code>exec bash</code> , depois <code>which ruby</code> : tem que apontar para dentro de <code>~/.local/share/mise</code>
<code>Could not find gem 'rails'</code>	Ruby do sistema, não o do <code>mise</code>	<code>mise use -g ruby@3.4.9</code> e <code>gem install rails</code>
<code>You don't have write permissions</code>	you está usando o Ruby do sistema com <code>sudo</code>	nunca use <code>sudo gem install</code> ; conserte o <code>mise</code>
a pasta <code>automic_auth_api</code> já existe	you rodou duas vezes	<code>rm -rf ~/automic_auth_api</code> e refaça, ou escolha outro nome
trava em <code>Bundle complete</code> por minutos	está compilando gem nativa	é normal na primeira vez; espere

Prática 4: Suba o banco e a aplicação

É a prática em que mais gente trava, e quase sempre é porta ocupada.

Crie um arquivo `compose.yaml` na raiz do projeto:

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: automic
      POSTGRES_PASSWORD: automic
      POSTGRES_DB: automic_auth_api_development
    ports:
      - "${DB_PORT:-5432}:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U automic"]
      interval: 5s
      retries: 10

volumes:
  pgdata:
```

E em `config/database.yml`, dentro do bloco `default: &default`, acrescente:

```
host: <%= ENV.fetch("DB_HOST", "localhost") %>
port: <%= ENV.fetch("DB_PORT", 5432) %>
username: <%= ENV.fetch("DB_USER", "automic") %>
password: <%= ENV.fetch("DB_PASSWORD", "automic") %>
```

```
docker compose up -d
docker compose ps          # tem que aparecer "healthy"

bin/rails db:prepare
bin/rails server
```

Confere: abra <http://localhost:3000/up>. Verde é a aplicação de pé, falando com o banco.

Se der errado

Esta é, de longe, a prática em que mais gente trava: em toda turma, em qualquer material. Os dois erros de sempre são porta ocupada e permissão do Docker, e **nenhum dos dois é sobre programar**. São detalhes de ambiente que todo mundo enfrenta uma vez na vida e depois nunca mais. Achou o seu na tabela? Resolve em dois minutos.

Erro	Causa	Saída
<code>address already in use</code> na 5432	you already have a PostgreSQL on the machine	<code>DB_PORT=5433 docker compose up -d</code> e <code>export DB_PORT=5433</code> no mesmo shell do <code>rails</code>
<code>permission denied</code> no <code>docker</code>	you are not in the <code>docker</code> group	<code>sudo usermod -aG docker \$USER</code> , e saia e entre de novo (reabrir o terminal não basta)
<code>docker: command not found</code> no WSL2	lack of Docker Desktop integration	Docker Desktop → Settings → Resources → WSL Integration → enable the distro
<code>PG::ConnectionBad: could not connect</code>	the container has not started yet, or the port is not open	<code>docker compose ps</code> : the status must be <code>healthy</code> , not <code>starting</code>
<code>PG::ConnectionBad: password authentication failed</code>	the <code>database.yml</code> does not match the <code>compose.yml</code>	user and password must be <code>automatic</code> in both
<code>ActiveRecord::NoDatabaseError</code>	the database was not created	<code>bin/rails db:prepare</code> (it creates and migrates)
the page <code>/up</code> is red	the Rails app started but did not reach the database	it's the same problem as above; read the log of <code>rails server</code>
<code>Address already in use - bind(2) for 127.0.0.1:3000</code>	there is already a Rails running	kill with <code>lsof -i :3000</code> and mate, or <code>bin/rails server -p 3001</code>

Prática 5: A sua primeira rota

O coração da aula.

Em `config/routes.rb`, dentro do `Rails.application.routes.draw do`:

```
namespace :api do
  namespace :v1 do
    get "status", to: "status#show"
  end
end
```

Crie o arquivo `app/controllers/api/v1/status_controller.rb`: o caminho tem que ser exatamente esse, é o Zeitwerk que exige:

```
module Api
  module V1
    class StatusController < ApplicationController
      def show
        render json: {
          status: "ok",
          service: "atomic-auth-api",
          environment: Rails.env
        }
      end
    end
  end
end
```

Confere:

```
bin/rails routes -g api          # a sua rota tem que aparecer na lista
curl -i localhost:3000/api/v1/status
```

Tem que vir `200` e o JSON. Depois monte a mesma requisição no Insomnia e confira o status ali também: é a ferramenta que você vai usar nas Aulas 3 e 4.

Se der errado

O erro campeão aqui é `uninitialized constant`. Ele assusta porque parece dizer que a sua classe não existe, e o que ele está dizendo, na verdade, é *"procurei no caminho que o nome promete e não achei"*. É o Zeitwerk da seção 6, e a correção é sempre no nome do arquivo ou da pasta.

Erro	Causa	Saída
<code>uninitialized constant Api::V1::StatusController</code>	o arquivo está no caminho errado	o caminho tem que ser <code>app/controllers/api/v1/status_controller.rb</code> tudo minúsculo, com <code>_controller</code>
<code>uninitialized constant Api::V1</code>	you criou a classe como <code>class Api::V1::StatusController</code> sem os módulos abertos, ou faltou uma pasta	use os três <code>module</code> / <code>class</code> exemplo
<code>No route matches [GET] "/api/v1/status"</code>	a rota não entrou	<code>bin/rails routes -g api;</code> <code>namespace</code> está fora do <code>draw</code>
a rota lista como <code>/api/v1/status</code> mas dá 404	you está batendo em outra porta	confira em que porta o <code>rails</code>
<code>ActionController::RoutingError ... status#show</code>	o método <code>show</code> não existe no controller	o nome do método tem que ser <code>show</code> <code>to: "status#show"</code>
<code>NameError: undefined local variable 'Rails'</code>	erro de digitação (<code>rails.env</code>)	Ruby é sensível a maiúsculas
mudou o código e nada mudou na resposta	you está em <code>production</code> , que faz cache	em desenvolvimento o Rails não faz cache confira <code>Rails.env</code>

Prática 6: O teste

Um teste que você escreve hoje é o que vai te avisar, na Aula 4, que o deploy quebrou alguma coisa.

Crie `test/controllers/api/v1/status_controller_test.rb`:

```
require "test_helper"

class Api::V1::StatusControllerTest < ActionDispatch::IntegrationTest
  test "responde ok" do
    get "/api/v1/status"

    assert_response :ok
    assert_equal "ok", response.parsed_body["status"]
  end
end
```

```
bin/rails test
```

Confere: sai `1 runs, 2 assertions, 0 failures`. Agora **quebre de propósito:** troque `"ok"` por `"OK"` no teste, rode de novo, e leia a mensagem de falha inteira. Depois desfaça.

Ver o teste falhar é o que prova que ele está realmente testando alguma coisa.

Se der errado

Erro	Causa	Saída
<code>ActiveRecord::PendingMigrationError</code>	o banco de teste está atrasado	<code>bin/rails db:test:prepare</code>
<code>PG::ConnectionBad</code> no teste	o container do banco não está de pé	<code>docker compose up -d</code>
<code>NoMethodError: parsed_body</code>	versão antiga do Rails	confira <code>bin/rails -v</code> ; este material é para o Rails 8
<code>0 runs, 0 assertions</code>	o arquivo não foi encontrado	o nome tem que terminar em <code>_test.rb</code> e estar dentro de <code>test/</code>
<code>NameError: uninitialized constant ...ControllerTest</code>	o nome da classe não bate com o do arquivo	<code>status_controller_test.rb</code> → <code>StatusControllerTest</code>

Prática 7: Commit e GitHub

Não é opcional: na Aula 4 o CI roda no **seu** repositório e a imagem Docker vai para a **sua** conta.

```
git add -A
git commit -m "Rota de status"
gh repo create automic_auth_api --private --source=. --push
```

Sem o `gh` instalado, crie o repositório pelo site e depois:

```
git remote add origin git@github.com:SEU-USUARIO/automic_auth_api.git
git push -u origin main
```

Confere:

```
gh repo view --web      # abre o seu repositório no navegador
git log --oneline       # os seus commits estão lá
```

Se der errado

Erro	Causa	Saída
<code>gh: command not found</code>	o <code>gh</code> não está instalado	<code>sudo apt install gh</code> / <code>brew install gh</code>
<code>gh auth status</code> diz que não está logado	falta autenticar	<code>gh auth login</code> → GitHub.com → HTTPS → pelo navegador
<code>Permission denied (publickey)</code> no push por SSH	a sua chave não está no GitHub	<code>gh ssh-key add ~/.ssh/id_ed25519.pub</code>
<code>remote origin already exists</code>	you rodou o <code>remote add</code> duas vezes	<code>git remote set-url origin <url></code>
<code>src refspec main does not match any</code>	you ainda não commitou nada	<code>git add -A && git commit -m "..."</code> primeiro
<code>Updates were rejected</code>	o repositório remoto tem commits que você não tem	<code>git pull --rebase origin main</code> e empurre de novo

Bônus, se sobrou tempo

- Faça a rota devolver também `RUBY_VERSION` e `Rails.version`.
- Acrescente uma asserção no teste para os dois campos novos.
- Faça `curl -i localhost:3000/api/v1/status` e confira que o `content-type` é `application/json`. Por que ele é isso e não `text/html`?

Travou em qualquer prática? Abra o mesmo arquivo no gabarito, entenda o que está diferente, e conserte o seu:

```
cd ~/capacitacao-gabarito && git checkout aula-01
cat app/controllers/api/v1/status_controller.rb
```

Recapitulando

- Back-end é a cozinha: regra, dado e permissão.
- HTTP é o idioma; verbo, status e JSON são o vocabulário.
- HTTP não tem memória: segure essa ponta até a Aula 3.
- Ruby é Python com outra sintaxe e mais açúcar.
- Rails decide o óbvio por você. Siga a convenção.

Na próxima: o banco de dados entra em cena, e a API ganha um `User` com senha de verdade.

Aula 2: Banco de dados, ActiveRecord e o model `User`

Você sai daqui com: um `User` com validações e senha hasheada, testado. **Gabarito:** `cd ~/capacitacao-gabarito && git checkout aula-02`

De onde viemos, e para onde vamos hoje

Na Aula 1 você subiu uma API que responde `GET /api/v1/status`. Ela funciona, e não guarda nada: desligou o servidor, acabou.

Hoje ela ganha memória. Você vai criar as tabelas, escrever o model `User`, guardar senha do jeito certo (que não é guardar senha) e ligar duas tabelas uma na outra. No fim, um usuário criado no console continua lá depois de você reiniciar tudo.

É a aula em que você mais escreve código nas quatro, e é a base da Aula 3, em que esse `User` vira cadastro, login e recuperação de senha de verdade.

Travou? A tabela **Se der errado** no fim de cada prática cobre os erros que realmente acontecem. Consulte antes de chamar, e chame se não resolver em dez minutos.

1. Por que Postgres, e não SQLite

SQLite é um arquivo. Funciona muito bem para um app de celular ou um script. Para um servidor com várias requisições ao mesmo tempo, ele trava: só uma escrita por vez no banco inteiro.

Postgres é um servidor: aguenta concorrência, tem tipos ricos (`jsonb`, arrays, intervalos de tempo), índices parciais e transações de verdade. É o que o `seem-backend` usa em produção, então é o que usamos aqui: **desenvolver no mesmo banco da produção evita a categoria inteira de bug que só aparece no deploy.**

Transação

Um bloco de operações que acontece **inteiro ou não acontece**. Deu erro no meio, o banco desfaz tudo: isso se chama *rollback*.

O exemplo clássico é transferir dinheiro: debitar de um e creditar no outro têm que ser a mesma operação. Debitar sozinho é dinheiro que sumiu.

```
User.transaction do
  user.save!
  user.invalidate_sessions!
  user.clear_password_reset_code!
end
```

Repare no `!`: dentro de uma transação você **quer** que estoure. `save` devolve `false` em silêncio e a transação seguiria feliz, gravando metade.

Na Aula 3, trocar a senha vai precisar exatamente disso.

2. ActiveRecord

É o ORM do Rails: cada classe é uma tabela, cada objeto é uma linha.

Django ORM	ActiveRecord
<code>User.objects.filter(role="admin")</code>	<code>User.where(role: "admin")</code>
<code>User.objects.get(id=1)</code>	<code>User.find(1)</code>
<code>User.objects.all().order_by("-created_at")</code>	<code>User.order(created_at: :desc)</code>
<code>User.objects.count()</code>	<code>User.count</code>
<code>u.save()</code>	<code>u.save</code>

A diferença de filosofia: no Django você declara os campos na classe. **No Rails a classe não declara nada**: ela lê as colunas do banco em tempo de execução. A fonte da verdade é a migration.

```
class User < ApplicationRecord
end
```

Isso já tem `name`, `email`, `id`, `created_at`: tudo que existir na tabela `users`.

ActiveSupport: os métodos que o Rails inventou

O Rails adiciona métodos às classes do próprio Ruby. Isso é a gem `activesupport`, não Ruby puro, fora do Rails esses métodos somem.

```
15.minutes
1.day.ago
2.weeks.from_now
Time.current      # respeita o fuso configurado na app; Time.now não
```

E o par que você vai usar mais que qualquer outro:

```
nil.blank?      # true
"".blank?       # true
"  ".blank?     # true  ← só espaço também conta como vazio
[].blank?       # true
0.blank?        # false  ← zero NÃO é vazio

"oi".present?   # true  ← present? é o contrário de blank?
```

Pegadinha para quem vem de Python: em Ruby puro, só `nil` e `false` são falsos. `if 0` executa. `if ""` executa. É justamente por isso que `blank?` existe.

3. Migrations

Uma migration é uma **mudança no banco, versionada em código**. Ela roda uma vez, em ordem, em todas as máquinas: a sua, a do colega e o servidor.

```
bin/rails generate migration CreateUsers
```

Isso cria `db/migrate/20260803142949_create_users.rb`. O número é a data e hora: é ele que define a ordem.

```
class CreateUsers < ActiveRecord::Migration[8.1]
  def change
    create_table :users do |t|
      t.string :name, null: false
      t.string :email, null: false
      t.string :password_digest, null: false
      t.string :course, null: false
      t.string :matricula, null: false
      t.datetime :terms_accepted_at, null: false

      t.timestamps
    end

    add_index :users, :email, unique: true
    add_index :users, :matricula, unique: true
  end
end
```

```
bin/rails db:migrate
```

Repare em duas coisas:

- `t.timestamps` cria `created_at` e `updated_at`, e o Rails mantém as duas sozinho.
- `add_index ... unique: true` é uma restrição no banco, não só validação no model. Isso importa: duas requisições simultâneas passam pela validação juntas (as duas consultam e as duas não acham ninguém), mas só uma vence o índice único. **Validação de model é para dar mensagem bonita; índice é para garantir.**

`schema.rb`

Depois de migrar, o Rails reescreve `db/schema.rb`: o retrato atual do banco. É ele que `db:prepare` usa para criar o banco de teste, e por isso **vai versionado**. Você nunca edita esse arquivo à mão.

Migrations são incrementais

Neste projeto a tabela `users` nasceu em três migrations, porque as funcionalidades chegaram em momentos diferentes:

```
20260803142949_create_users.rb          # o básico
20260803142950_add_email_verification_to_users.rb # confirmação de e-mail
20260803142951_add_password_reset_to_users.rb   # recuperação de senha
```

É assim que acontece na vida real. Nunca edite uma migration que já rodou em produção: crie outra.

4. Senha: o que nunca fazer

Nunca guarde senha em texto. Se o banco vazar, e bancos vazam, você entregou a senha de todos. E como as pessoas repetem senha, você entregou o e-mail e o banco delas junto.

Hash não é criptografia

Criptografia tem volta: basta ter a chave. **Hash não tem volta**, é um caminho só. Você não guarda a senha; guarda o resultado de passar a senha pela função. Na hora do login, passa de novo e compara os resultados.

Por que bcrypt e não SHA-256

SHA-256 é rápido: **e isso é o problema**. Uma GPU testa bilhões de SHA-256 por segundo. bcrypt foi feito para ser **lento de propósito** e tem um fator de custo ajustável: quando o hardware melhora, você aumenta o custo.


```
u = User.new(name: "Teste")
u.valid?      # false
u.errors.full_messages
```

Regra de ouro: normalize antes de validar

`Ana@UFOP.br` e `ana@ufop.br` são o mesmo e-mail. `20.112-34` e `2011234` são a mesma matrícula.

```
def self.normalize_email(email)
  email.to_s.strip.downcase
end

def self.normalize_matricula(matricula)
  matricula.to_s.gsub(/\D/, "")
end
```

Sem isso o índice único não serve para nada: o banco acha que são valores diferentes.

6. Associações

Uma tabela nunca basta. Todo sistema real tem tabelas que se referem umas às outras: usuário tem muitos pedidos; palestra acontece numa sala; sala tem muitas palestras.

Vamos criar a segunda tabela do projeto: `login_events`, o histórico de acessos da conta. É o que o seu banco usa para mandar "novo acesso detectado" por e-mail.

Quem guarda a chave

A relação um-para-muitos mora numa coluna só: a **chave estrangeira**. `login_events.user_id` aponta para `users.id`.

- quem **carrega** a coluna usa `belongs_to`;
- quem é **apontado** usa `has_many`.

Regra prática: a chave fica sempre no lado "muitos".

A migration

```
create_table :login_events do |t|
  t.references :user, null: false, foreign_key: true
  t.string :client, null: false
  t.string :ip_address
  t.datetime :occurred_at, null: false
  t.timestamps
end

add_index :login_events, [ :user_id, :occurred_at ]
```

- `t.references :user` cria a coluna `user_id`, o índice `e` a chave estrangeira.
- `foreign_key: true` faz o **banco** recusar uma linha órfã: não é só validação de model.
- O índice composto tem `user_id` primeiro porque a consulta é sempre "os últimos logins **deste** usuário". Num índice composto, **a ordem das colunas importa**: primeiro o que filtra.

Os dois lados

```
class User < ApplicationRecord
  has_many :login_events, dependent: :delete_all
end

class LoginEvent < ApplicationRecord
  belongs_to :user

  scope :recentes, -> { order(occurred_at: :desc) }
end
```

- `dependent: :delete_all`: apagar o usuário apaga o histórico junto. Sem isso sobram linhas apontando para um `id` que não existe mais.
- `belongs_to` já exige presença desde o Rails 5: `LoginEvent` sem `user` é inválido.
- `scope` é uma consulta com nome, e ela encadeia.

Usando

```
# Criar PELA associação já preenche o user_id – não precisa passar:
user.login_events.create!(client: "mobile", occurred_at: Time.current)

user.login_events.count
user.login_events.recentes.limit(5)
evento.user.name # navega para o outro lado
```

A armadilha N+1

```
users.each { |u| puts u.login_events.count }
```

Com 100 usuários isso são **101 consultas**: uma para buscar os usuários e cem para buscar o histórico de cada um. É o problema de performance número 1 de aplicação Rails.

A correção é uma palavra:

```
User.includes(:login_events).each { |u| puts u.login_events.count }
```

Duas consultas, sempre. O `seem-backend` usa a gem **Bullet**, que estoura um erro em desenvolvimento quando você escreve um N+1 sem perceber.

7. Concerns: o mixin da Aula 1, na prática

O `User` vai ganhar confirmação de e-mail e recuperação de senha. São dois assuntos que não conversam entre si. Jogar os dois no `user.rb` produz um arquivo de 800 linhas que ninguém abre com vontade.

Cada assunto vira um módulo em `app/models/concerns/`:

```
# app/models/concerns/email_verifiable.rb
module EmailVerifiable
  extend ActiveSupport::Concern

  CODE_TTL = 15.minutes
  RESEND_COOLDOWN = 1.minute

  included do
    scope :email_verified, -> { where.not(email_verified_at: nil) }
  end

  def email_verified?
    email_verified_at.present?
  end

  def issue_email_verification_code!
    code = format("%06d", SecureRandom.random_number(1_000_000))

    update!(
      email_verification_code_digest: BCrypt::Password.create(code),
      email_verification_expires_at: CODE_TTL.from_now,
      email_verification_sent_at: Time.current
    )

    code
  end
end
```



```
class User < ApplicationRecord
  include EmailVerifiable
  include PasswordResetable
end
```

- `extend ActiveSupport::Concern` habilita o bloco `included do ... end`, onde vão coisas que só fazem sentido na classe (scopes, validações, associações).
- O código completo dos dois concerns está em `app/models/concerns/`.

Três decisões dentro do concern, e o porquê de cada uma

- 1. O código de 6 dígitos também vira digest bcrypt.** Ele é uma senha temporária. Quem lê o banco não pode confirmar a conta de ninguém.
- 2. O código em texto só existe no retorno do método.** `issue_email_verification_code!` devolve o código para o service mandar por e-mail, e depois ele some. Não fica em coluna, nem em log.
- 3. `email_verified_at` é data, não booleano.** Um booleano responde "sim". Uma data responde "sim, em 12 de agosto às 14h", e isso você vai querer saber quando alguém abrir um chamado.

8. O console

```
bin/rails console
```

```
u = User.new(
  name: "Ana Souza",
  email: "ana@aluno.ufop.edu.br",
  password: "Automic@2026",
  course: "Engenharia de Controle e Automação",
  matricula: "2011234",
  terms_accepted_at: Time.current
)

u.valid?
u.save
u.password_digest      # o hash, não a senha
u.authenticate("Automic@2026") # devolve o user
u.authenticate("errada")    # false

User.count
User.where("email LIKE ?", "%ufop%")
User.find_by(email: "ana@aluno.ufop.edu.br")

codigo = u.issue_email_verification_code!
u.verify_email_code!(codigo)
u.email_verified?
```

```
bin/rails console --sandbox
```

 desfaz tudo ao sair. Bom para experimentar sem sujar o banco.

9. Seeds e fixtures

Seeds povoam o banco de desenvolvimento:

```
bin/rails db:seed
```

Fixtures são os dados dos testes, em `test/fixtures/users.yml`. O Rails carrega antes de cada teste e limpa depois: todo teste começa do mesmo estado.

```
ana:
  name: Ana Souza
  email: ana@aluno.ufop.edu.br
  password_digest: <%= BCrypt::Password.create("Automic@2026") %>
  email_verified_at: <%= 1.day.ago.to_fs(:db) %>
```

No teste, `users(:ana)` devolve esse registro.

10. Testando o model

```
test "guarda o digest e nunca a senha em texto" do
  user = build_user
  user.save!

  assert_not_equal "Automic@2026", user.password_digest
  assert user.authenticate("Automic@2026")
  assert_not user.authenticate("outra-senha")
end
```

```
bin/rails test
bin/rails test test/models/user_test.rb      # só um arquivo
bin/rails test test/models/user_test.rb:42   # só um teste
```

O Rails vem com **Minitest**. Se você conhece `pytest`, a ideia é a mesma; a sintaxe é `assert_*`.

11. Uma sutileza de segurança: `authenticate_by_email`

```
DUMMY_PASSWORD_DIGEST = BCrypt::Password.create("automic-timing-safe-dummy").freeze

def self.authenticate_by_email(email, password)
  user = find_by(email: normalize_email(email))
  digest = user&.password_digest || DUMMY_PASSWORD_DIGEST
  autenticado = BCrypt::Password.new(digest).is_password?(password.to_s)

  user if autenticado
end
```

Por que esse `DUMMY_PASSWORD_DIGEST`?

O jeito ingênuo seria `return nil unless user`. Só que bcrypt é lento **de propósito**. Se o e-mail não existe, a resposta volta em 1ms; se existe e a senha está errada, volta em 100ms. Cronometrando as respostas, dá para descobrir quem tem conta no sistema: é um **ataque de temporização**.

Rodando o bcrypt sempre, com um digest descartável quando não há usuário, as duas respostas custam o mesmo. Vamos usar esse método na Aula 3.

As práticas da aula

Oito práticas, cada uma logo depois do bloco que a explica. Todas terminam com uma conferência: não siga em frente com ela vermelha.

#	O que	Depois de
1	A gem do bcrypt, e o banco de pé	seção 1
2	As três migrations da tabela <code>users</code>	seção 3
3	Senha no console: veja o bcrypt trabalhar	seção 4
4	O validador de senha	seção 5
5	O model <code>User</code>	seção 5
6	A segunda tabela e a associação	seção 6
7	Os dois concerns	seção 7
8	Fixtures, testes e commit	seção 10

No **seu** projeto (`~/automic_auth_api`), continuando de onde a Aula 1 parou.

Prática 1: A gem do bcrypt, e o banco de pé

Aquecimento, e garanta que o ambiente da Aula 1 continua funcionando.

```
cd ~/automic_auth_api
docker compose up -d
docker compose ps          # healthy
```

No `Gemfile`, descomente (ou acrescente) a linha:

```
gem "bcrypt", "~> 3.1.7"
```

```
bundle install
```

Confere:

```
bin/rails runner 'puts BCrypt::Password.create("teste")[0, 7]'
```

Sai algo como `$2a$12$`. Esse prefixo é o algoritmo e o custo: você vai entender os dois na seção 4.

Se der errado

Erro	Causa	Saída
<code>Could not find gem 'bcrypt'</code>	esqueceu o <code>bundle install</code>	rode-o
<code>An error occurred while installing bcrypt</code>	falta compilador ou headers	Linux: <code>sudo apt install build-essential</code> ; macOS: <code>xcode-select --install</code>
<code>uninitialized constant BCrypt</code>	a linha ficou comentada no <code>Gemfile</code>	tire o <code>#</code> do começo
<code>PG::ConnectionBad</code>	o container não subiu	<code>docker compose up -d</code> e confira o <code>ps</code>

Prática 2: As três migrations da tabela `users`

A prática mais longa do dia. Faça uma migration por vez, e migre entre elas.

```
bin/rails generate migration CreateUsers
bin/rails generate migration AddEmailVerificationToUsers
bin/rails generate migration AddPasswordResetToUsers
```

O conteúdo da primeira está na seção **3. Migrations**. As outras duas:

```
# add_email_verification_to_users.rb
add_column :users, :email_verification_code_digest, :string
add_column :users, :email_verification_expires_at, :datetime
add_column :users, :email_verification_sent_at, :datetime
add_column :users, :email_verified_at, :datetime

# add_password_reset_to_users.rb
add_column :users, :password_reset_code_digest, :string
add_column :users, :password_reset_expires_at, :datetime
add_column :users, :password_reset_sent_at, :datetime
```

```
bin/rails db:migrate
```

Confere:

```
bin/rails db:migrate:status # três linhas "up"
sed -n '/create_table "users"/,/^ end/p' db/schema.rb | grep '^ t\.' | grep -vc 't\.index'
```

O segundo comando conta as colunas da tabela, e tem que sair **16**.

Repare em duas coisas que ele mostra: o `id` não está na lista, porque o `create_table` cria essa coluna sozinho e nem a menciona; e as duas linhas `t.index` que o comando descarta são os índices únicos de `email` e `matricula`, que são restrição do banco e não coluna.

Abra o `db/schema.rb` e leia: ele é o retrato do banco **agora**, montado sozinho pelas migrations.

Se der errado

Migration que falha no meio dá uma mensagem enorme e assustadora. **Ignore a primeira linha**, ela só diz que a migration parou. A resposta está na **linha seguinte**, e costuma ser bem específica. Esse hábito sozinho economiza horas ao longo de uma carreira.

E nesta fase do projeto, `db:drop db:create db:migrate` é sempre uma saída legítima: não há dado nenhum para perder. Não tenha dó.

Erro	Causa	Saída
<code>PG::DuplicateTable: relation "users" already exists</code>	you rodou a migration duas vezes, ou criou a tabela na mão	<code>bin/rails db:drop db:create db:migrate</code> e tudo bem agora)
<code>PG::DuplicateColumn</code>	mesma coisa, com coluna	idem
<code>An error has occurred, this and all later migrations canceled</code>	uma migration falhou no meio	leia a linha seguinte: é ela que diz o erro arquivo e rode de novo
<code>ActiveRecord::IrreversibleMigration</code> ao fazer <code>rollback</code>	a migration usou <code>change</code> com algo que não sabe desfazer	troque por <code>up</code> / <code>down</code> , ou refaça com <code>db</code>
o <code>schema.rb</code> não mudou	a migration não rodou	<code>bin/rails db:migrate:status</code> : a sua es
saiu menos que 16	falta alguma coluna	compare com o gabarito: <code>git -C ~/capacitacao-gabarito show a</code>
<code>Multiple migrations have the name ...</code>	you gerou duas com o mesmo nome	apague a duplicada em <code>db/migrate/</code>

Prática 3: senha no console, vendo o bcrypt trabalhar

Sem escrever arquivo nenhum. É a prática que faz a seção 4 parar de ser teoria.

```
bin/rails console
```

Antes de rodar, aposte: a mesma senha, hashada duas vezes, dá o mesmo resultado ou resultados diferentes? Decida agora: a graça do exercício está em você errar essa aposta.

```
# a) o mesmo texto, dois digests diferentes
BCrypt::Password.create("Automic@2026")
BCrypt::Password.create("Automic@2026")
```

Rodou duas vezes e deu **diferente**? Esse é o *salt*, e é ele que impede alguém de descobrir que dois usuários têm a mesma senha.

```
# b) mas os dois conferem
digest = BCrypt::Password.create("Automic@2026")
BCrypt::Password.new(digest) == "Automic@2026"    # true
BCrypt::Password.new(digest) == "errada"          # false
```

```
# c) por que é lento de propósito
require "benchmark"
```

```
Benchmark.realtime { BCrypt::Password.create("Automic@2026") } # UM bcrypt
Benchmark.realtime { 1000.times { Digest::SHA256.hexdigest("Automic@2026") } } # MIL SHA-256
```

Confere: o segundo comando fez **mil** hashes de SHA-256 e mesmo assim terminou muito antes do primeiro, que fez **um** bcrypt.

É essa diferença que protege o seu banco. Quem rouba a tabela de usuários testa senha em lote: com SHA-256 ele testa bilhões por segundo, e com bcrypt, algumas dezenas. Diga em voz alta por que a lentidão aqui é uma *característica* e não um defeito.

Se der errado

Erro	Causa	Saída
<code>uninitialized constant BCrypt</code>	a gem não entrou	volte para a Prática 1
<code>uninitialized constant Digest</code>	não carregou	<code>require "digest"</code> antes
o console não abre, erro de banco	container caiu	<code>docker compose up -d</code>
você fechou o console sem querer	<code>exit</code> ou <code>Ctrl+D</code>	é só reabrir; nada foi perdido

Prática 4: O validador de senha

O primeiro arquivo Ruby seu, do zero, nesta capacitação.

Crie `app/validators/password_policy_validator.rb`:

```

class PasswordPolicyValidator
  SPECIAL_CHARACTERS = %r{[!@#$%^&*(),.?":{}|<>]}
  MIN_LENGTH = 8
  MAX_LENGTH = 16

  def self.validate(password)
    new(password).validate
  end

  def initialize(password)
    @password = password.to_s
  end

  # Devolve uma lista de mensagens. Vazia significa senha aceita.
  def validate
    errors = []
    errors << "deve ter entre #{MIN_LENGTH} e #{MAX_LENGTH} caracteres" unless length_valid?
    errors << "deve incluir pelo menos uma letra maiúscula" unless @password.match?(/[A-Z]/)
    errors << "deve incluir pelo menos uma letra minúscula" unless @password.match?(/[a-z]/)
    errors << "deve incluir pelo menos um dígito" unless @password.match?(/\d/)
    errors << "deve incluir pelo menos um caractere especial" unless @password.match?
(SPECIAL_CHARACTERS)
    errors
  end

  private

  def length_valid?
    @password.length.between?(MIN_LENGTH, MAX_LENGTH)
  end
end

```

Repare em três coisas da Aula 1: o `self.validate` que instancia e chama, o `@password` de instância, e o `private` valendo da linha em diante.

Ele fica fora do model porque o cadastro precisa validar a senha **antes** de existir um `User` (Aula 3), e a recuperação de senha valida de novo na hora de trocar.

Confere:

```

bin/rails runner 'p PasswordPolicyValidator.validate("abc")'
# a lista de erros
bin/rails runner 'p PasswordPolicyValidator.validate("Automic@2026")'
# []

```

Avançado: `PasswordPolicyValidator.validate(nil)` também tem que devolver a lista de erros, sem estourar. Descubra qual linha do código garante isso.

Se der errado

Erro	Causa	Saída
<code>uninitialized constant PasswordPolicyValidator</code>	caminho ou nome do arquivo errado	tem que ser <code>app/validators/password_policy_validator.rb</code> e criar o <code>Zeitwerk</code> de novo
<code>NoMethodError: undefined method 'length' for nil</code>	você tirou o <code>.to_s</code> do <code>initialize</code>	é ele que transforma <code>nil</code> em <code>""</code>
<code>undefined method 'validate' for an instance of Class</code>	escreveu <code>def validate</code> onde queria <code>def self.validate</code>	o <code>self.</code> faz o método ser da classe
a senha boa devolve erro de caractere especial	a regex saiu com escape errado	copie a linha do <code>SPECIAL_CHARACTERS</code> e substitua
<code>syntax error, unexpected end</code>	faltou ou sobrou um <code>end</code>	conte: <code>class</code> , <code>def</code> ×4, <code>if</code> ...

Prática 5: O model `User`

Crie `app/models/user.rb` com `has_secure_password validations: false`, as seis validações, os três normalizadores e o `authenticate_by_email` (seções 4, 5 e 11).

Confere:

```
bin/rails runner '
u = User.new(name: "Ana", email: "ana@ufop.br", password: "Automic@2026",
              course: "Automação", matricula: "2011234", terms_accepted_at: Time.current)
puts u.valid?
puts u.password_digest[0, 20]
'
```

Tem que sair `true` e um digest começando em `$2a$12$`.

Agora **quebre de propósito** e leia a mensagem:

```
bin/rails runner 'u = User.new(email: "NAO-E-EMAIL"); u.valid?; p u.errors.full_messages'
```

Se der errado

Erro	Causa	Saída
<code>NoMethodError: undefined method 'password='</code>	falta o <code>has_secure_password</code>	acrescente-o no topo da classe
<code>PG::UndefinedColumn: column "password_digest" does not exist</code>	a migration não rodou	<code>bin/rails db:migrate</code>
<code>valid?</code> devolve <code>false</code> e você não sabe por quê	as mensagens estão no objeto	<code>p u.errors.full_messages</code>
<code>valid?</code> devolve <code>true</code> com e-mail inválido	falta a validação de formato	releia a seção 5
o e-mail salvou com maiúscula	o normalizador não rodou	<code>normalizes :email, with:</code>
<code>ArgumentError: wrong number of arguments</code>	passou posicional onde é nomeado	<code>User.new(name: ..., email</code>

Prática 6: A segunda tabela e a associação

É aqui que o banco deixa de ser uma tabela e vira um *modelo de dados*.

```
bin/rails generate migration CreateLoginEvents
```

O conteúdo está na seção **6. Associações**. Depois crie `app/models/login_event.rb` com o `belongs_to :user` e o `scope :recentes`, e acrescente no `User`:

```
has_many :login_events, dependent: :delete_all
```

```
bin/rails db:migrate
```

Confere, no console:

```
u = User.first || User.create!(name: "Ana", email: "ana@ufop.br", password: "Automic@2026",
  course: "Automação", matricula: "2011234", terms_accepted_at: Time.current)
u.login_events.create!(client: "mobile", occurred_at: Time.current)
u.login_events.count # 1
u.login_events.recentes.first.user.name # navega para o outro lado
```

Avançado: apague o usuário e confirme que o histórico foi junto.

```
LoginEvent.count # 1
u.destroy
LoginEvent.count # 0 <- é o dependent: :delete_all
```

Se der errado

Erro	Causa	Saída
<code>PG::ForeignKeyViolation</code>	você criou um <code>login_event</code> com <code>user_id</code> que não existe	crie sempre a partir do usuário: <code>u.login_events.create!</code>
<code>NameError: uninitialized constant User::LoginEvent</code>	o model não existe ou está no caminho errado	<code>app/models/login_event.rb</code>
<code>ActiveRecord::RecordInvalid: Validation failed: User must exist</code>	o <code>belongs_to</code> é obrigatório por padrão no Rails 5+	é isso mesmo; passe o usuário
<code>undefined method 'recentes'</code>	o scope não foi declarado	<code>scope :recentes, -> { order(oc</code>
apagou o usuário e sobraram os eventos	falta o <code>dependent:</code>	acrescente no <code>has_many</code>
<code>unknown attribute 'occurred_at'</code>	a migration não tem a coluna	confira o <code>schema.rb</code>

Prática 7: Os dois concerns

O momento em que os módulos da Aula 1 deixam de ser teoria.

Crie `app/models/concerns/email_verifiable.rb` e `app/models/concerns/password_resetable.rb`. O primeiro está inteiro na seção **7. Concerns**; o segundo é o mesmo desenho, trocando `email_verification_*` por `password_reset_*` e sem o `email_verified_at`.

Métodos que cada um precisa ter:

EmailVerifiable	PasswordResettable
email_verified?	—
issue_email_verification_code!	issue_password_reset_code!
verify_email_code!	verify_password_reset_code!
verification_expired?	password_reset_expired?
resend_verification_allowed?	password_reset_request_allowed?
—	clear_password_reset_code!

Confere, no console:

```
u = User.first
codigo = u.issue_email_verification_code!
codigo # 6 dígitos
u.email_verification_code_digest # o hash, NUNCA o código
u.verify_email_code!(codigo) # true
u.verify_email_code!(codigo) # false – só vale uma vez
u.email_verified? # true
```

Repare: o banco guarda o **digest** do código, não o código. Mesma lição da senha.

Se der errado

Erro	Causa	Saída
undefined method 'issue_email_verification_code!'	faltou o <code>include EmailVerifiable</code> no <code>User</code>	acrescente
NameError: uninitialized constant EmailVerifiable	o arquivo está fora de <code>app/models/concerns/</code>	mová
undefined method 'extend' / 'included'	faltou <code>extend ActiveSupport::Concern</code> no topo do módulo	acrescente
<code>verify_email_code!</code> devolve <code>true</code> duas vezes	o método não limpa o digest depois de usar	é a segunda linha do <code>verify_</code> código usado
<code>verify_email_code!</code> devolve <code>false</code> com o código certo	you comparou o código com o digest	compare com <code>BCrypt::Password.new(digest)</code>
o código expira na hora	<code>expires_at</code> está no passado	<code>15.minutes.from_now</code> , não

Prática 8: Fixtures, testes e commit

Fecha o dia, e é o que a Aula 3 vai usar de base.

Crie `test/fixtures/users.yml` com dois usuários: um verificado, outro não:

```
ana:
  name: Ana Souza
  email: ana@aluno.ufop.edu.br
  password_digest: <%= BCrypt::Password.create("Automic@2026") %>
  course: Engenharia de Controle e Automação
  matricula: "2011234"
  terms_accepted_at: <%= 1.day.ago.to_fs(:db) %>
  email_verified_at: <%= 1.day.ago.to_fs(:db) %>

bruno:
  name: Bruno Lima
  email: bruno@aluno.ufop.edu.br
  password_digest: <%= BCrypt::Password.create("Automic@2026") %>
  course: Engenharia de Minas
  matricula: "2019876"
  terms_accepted_at: <%= 1.day.ago.to_fs(:db) %>
  email_verified_at:
```

Depois escreva os testes de `user_test.rb`, dos dois concerns, do validador e do `login_event_test.rb`. Comece pelo que está na seção **10. Testando o model**.

```
bin/rails test
bin/rubocop
git add -A && git commit -m "Model User, concerns e associações" && git push
```

Confere: 30 testes verdes. O gabarito tem exatamente esse número.

Se der errado

Erro	Causa	Saída
<code>ActiveRecord::Fixture::FixtureError: table "users" has no column named X</code>	a fixture tem um campo que a tabela não tem	compare com o <code>schema.r</code>
<code>ActiveRecord::RecordNotUnique</code> nas fixtures	dois usuários com o mesmo e-mail ou matrícula	troque um deles
<code>NoMethodError: undefined method 'to_fs'</code>	Rails antigo	este material é Rails 8; confira <code>bin/rails -v</code>
um teste passa sozinho e falha junto com os outros	vazamento de estado entre testes	não use <code>User.first</code> no teste: use <code>users(:ana)</code>
<code>PendingMigrationError</code> só no teste	banco de teste atrasado	<code>bin/rails db:test:prepa</code>
<code>bin/rubocop</code> reclama de dezenas de coisas	estilo	<code>bin/rubocop -a</code> conserta maioria sozinho; leia o que sobrar
menos de 30 testes	falta cobrir algum caso	veja quais arquivos o gabarito tem em <code>test/</code>

Bônus, se sobrou tempo

1. Veja o N+1 acontecer. Com o log do Rails aberto:

```
User.all.each { |u| puts u.login_events.count } # uma consulta por usuário
User.includes(:login_events).each { |u| puts u.login_events.count } # duas, no total
```

2. Tente furar o índice único. Crie dois usuários com o mesmo e-mail direto no banco, sem passar pelas validações, e veja o Postgres recusar:

```
User.new(email: User.first.email, ...).save!(validate: false)
```

O erro que vem é `PG::UniqueViolation`, e é por isso que a restrição vive no banco, e não só no model.

Travou em qualquer prática? Compare arquivo por arquivo com o gabarito:

```
cd ~/capacitacao-gabarito && git checkout aula-02
cat app/models/user.rb
```

Recapitulando

- Migration é a fonte da verdade do banco. O model não declara colunas.
- Índice único é garantia; validação é mensagem bonita. Você quer os dois.
- Senha vira hash bcrypt, com salt, e nunca volta.
- Concern é o mixin do Rails: um assunto por arquivo.
- Normalize antes de validar, ou o índice único não serve para nada.
- A chave estrangeira fica no lado "muitos": `belongs_to` carrega, `has_many` é apontado.
- Transação é tudo ou nada. Dentro dela, use os métodos com `!`.
- `includes` resolve N+1, e N+1 é o problema de performance mais comum em Rails.

Na próxima: as rotas. O usuário vai conseguir se cadastrar, entrar, sair e recuperar a senha.

Aula 3: As rotas de autenticação

Você sai daqui com: cadastro, ativação, login, logout e recuperação de senha. **Gabarito:** `cd ~/capacitacao-gabarito && git checkout aula-03`

De onde viemos, e o combinado de hoje

Você já tem uma API (Aula 1) e um `User` que sabe guardar senha (Aula 2). **Hoje as duas coisas viram um sistema de autenticação que funciona de verdade:** o mesmo que roda no `seem-backend`.

O combinado desta aula é diferente das outras. São 1400 linhas em 37 arquivos, e ninguém digita isso em três horas. Você traz o código pronto do gabarito e passa a aula operando, quebrando de propósito e entendendo por quê.

É o que um desenvolvedor faz na maior parte do tempo real: ler código que já existe, descobrir por que foi feito assim, e mexer com segurança. Digitar 1400 linhas copiando ensinaria menos que as oito práticas de hoje.

O que você tem que sair sabendo é **por que cada peça existe**, e é isso que as práticas cobram.

A prática 8 pede que você quebre a verificação de assinatura do token e entre como outra pessoa. É proposital, é o exercício que mais marca, e tem um passo explícito para desfazer. **Não pule o desfazer.**

1. O caminho de uma requisição

```
requisição → rota → controller → service → model → banco
                        ↓
                    serializer → JSON
```

Nesta aula todas essas peças aparecem. A regra que organiza tudo:

Controller recebe requisição e devolve resposta. Regra de negócio não mora nele.

2. A arquitetura de pastas

```
app/controllers/
├─ application_controller.rb
├─ concerns/api/v1/
│  ├─ authentication.rb      # quem está chamando?
│  └─ error_rendering.rb    # um formato de erro só
└─ api/v1/
   ├─ base_controller.rb    # pai de todos: erros, auth, strong params
   ├─ registrations_controller.rb
   ├─ email_verifications_controller.rb
   ├─ sessions_controller.rb
   ├─ password_resets_controller.rb
   └─ me_controller.rb

app/services/                # os casos de uso
├─ auth/
└─ users/

app/serializers/            # o que vai para o JSON
```

Por que service object

O `Users::Register` faz seis coisas: valida campos obrigatórios, aplica a política de senha, confere a confirmação, normaliza os dados, cria o usuário e dispara o e-mail. Dentro do controller isso vira um método de 60 linhas que só dá para testar subindo uma requisição HTTP inteira.

Fora dele:

```
result = Users::Register.call(name: "...", email: "...", ...)
result.success?  # true/false
result.user
result.errors    # [{ field: "password", message: "..."}]
```

Testável direto, reaproveitável (uma task de importação chama o mesmo service) e o controller volta a ter cinco linhas. O padrão sempre igual: **um `call` de classe, um `Result`**.

Por que serializer

```
render json: user
```

Isso manda o objeto inteiro: `password_digest`, `email_verification_code_digest`, `password_reset_code_digest`. Vazamento em uma linha.

O serializer é a lista de convidados: só entra quem está escrito lá.

```
class UserSerializer
  def self.as_json(user)
    { id: user.id, name: user.name, email: user.email, ... }
  end
end
```

Um formato de erro só

```
{
  "error": {
    "code": "validation_failed",
    "message": "Falha na validação",
    "details": [{ "field": "password", "message": "deve incluir pelo menos um dígito" }]
  }
}
```

- **code** é estável: o cliente decide o que fazer com base nele.
- **message** é para o usuário ler. Pode mudar, pode ser traduzida.
- **details** diz qual campo falhou, para o app pintar o input de vermelho.

Uma API que devolve erro de três jeitos diferentes força o cliente a tratar os três.

before_action : o filtro

```
class MeController < BaseController
  before_action :authenticate_user!

  def show
    render_user(current_user) # só roda se passou pelo filtro
  end
end
```

O filtro roda **antes** da action. Se ele renderizar alguma coisa, o **401** por exemplo, a action nem chega a ser chamada. **only:** e **except:** limitam a quais actions ele se aplica:

```
before_action :authenticate_user!, only: :destroy
```

A pilha de middleware

A requisição não cai direto no controller. Ela atravessa uma pilha de camadas: o **middleware**. Cada camada pode ler, alterar, ou responder e cortar o caminho ali mesmo.

```
requisição
↓
[ Rack::Cors ]           ← libera (ou não) a origem
[ ActionController::Cookies ]
[ ... ]
↓
rota → before_action → controller → service → model
↓
serializer → JSON → resposta (voltando pela mesma pilha)
```

```
bin/rails middleware      # lista a pilha inteira do seu app
```

O modo `--api` remove várias camadas. Trouxemos os cookies de volta na mão, em `config/application.rb`:

```
config.middleware.use ActionController::Cookies
```

Strong parameters

```
def registration_params
  expect_root_params(:name, :email, :password, :confirm_password,
                    :course, :matricula, :check_terms_use)
end
```

`params.expect` (Rails 8) **exige** essas chaves e **recusa o resto**. Sem isso alguém manda `"role": "admin"` no cadastro e vira admin. O nome disso é **mass assignment**, e já derrubou sistema grande.

3. Rota 1: Cadastro

```
POST /api/v1/registrations
```

```
{
  "name": "Diego Reis",
  "email": "diego@aluno.ufop.edu.br",
  "password": "Automic@2026",
  "confirm_password": "Automic@2026",
  "course": "Engenharia de Minas",
  "matricula": "2013333",
  "check_terms_use": true
}
```

→ `201 Created` com o usuário e um e-mail de ativação a caminho.

A decisão que não é óbvia

```
REGISTRATION_FAILED_MESSAGE =  
  "Não foi possível concluir o cadastro. Verifique os dados e tente novamente."
```

Quando o e-mail já existe, a resposta **não** diz "este e-mail já está cadastrado". Se dissesse, o cadastro viraria um verificador: eu testo mil e-mails e descubro quem tem conta no sistema. Chama-se **enumeração de usuários**, e vai voltar duas vezes nesta aula.

O custo é real: o usuário legítimo que esqueceu que já tem conta fica sem uma mensagem clara. A saída é o texto convidar a usar "esqueci minha senha".

4. E-mail: o mailer

Um mailer é um controller cujas views são e-mails.

```
class UserMailer < ApplicationMailer  
  def email_verification(user, code)  
    @user = user  
    @code = code  
    mail(to: @user.email, subject: "Ative sua conta")  
  end  
end
```

O template fica em `app/views/user_mailer/email_verification.text.erb`.

Em desenvolvimento não existe SMTP. O `letter_opener` abre o e-mail no navegador:

```
config.action_mailer.delivery_method = :letter_opener
```

`deliver_now` e não `deliver_later`

```
UserMailer.email_verification(user, code).deliver_now
```

O jeito "certo" de livro seria `deliver_later`, jogando numa fila. Aqui não, e o motivo é o código: numa fila, o código de 6 dígitos ficaria **gravado em texto** na tabela de jobs.

O preço é a requisição esperar o SMTP. Por isso o service tem `rescue`: falha de envio não pode virar 500.

```
rescue StandardError => e  
  Rails.error.report(e, handled: true, source: "SendEmailVerification")  
  Result.new(success?: false, error_code: "delivery_failed", ...)  
end
```

5. Ativação da conta

```
POST /api/v1/email_verifications/confirm { email, code }
POST /api/v1/email_verifications/resend  { email }
```

O `resend` **sempre** responde 200 com a mesma mensagem: conta inexistente, já verificada ou em cooldown são indistinguíveis. É a mesma regra do cadastro.

6. O problema: HTTP não tem memória

Lembra da Aula 1? Cada requisição começa do zero. Então como o servidor sabe que você já entrou?

Duas famílias de resposta:

Sessão no servidor	Token
O servidor guarda a sessão e manda um id no cookie	O servidor manda um crachá assinado e não guarda nada
Toda requisição consulta o armazenamento de sessão	A assinatura basta: só validar
Logout é apagar a sessão: imediato	Logout é o problema difícil (já chegamos lá)
Escalar exige sessão compartilhada entre servidores	Qualquer servidor valida sozinho

Escolhemos **token**, porque o cliente principal é um app nativo: que não tem cookie e roda em milhares de celulares.

JWT

```
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiIsInZlciI6MH0 . 4pQ8L_5f...
cabeçalho          payload          assinatura
```

Três partes em **Base64**, separadas por ponto.

Base64 não é segredo. É uma forma de escrever bytes usando só letras e números, para caber num cabeçalho HTTP, que é texto. Não tem chave, não tem criptografia, e qualquer um desfaz:

```
bash echo eyJzdWIiOiJ9 | base64 -d # {"sub":2}
```

Aquele monte de caractere embaralhado do JWT é isso: texto disfarçado.

O payload é público. Cole um token em jwt.io e você lê tudo. A assinatura garante que ninguém **alterou** o conteúdo, não que ninguém **leu**. Nunca ponha no payload nada que não possa ser lido, nem CPF, nem e-mail, nem permissão sensível.

O nosso payload:

```
{
  sub: user.id,           # subject: de quem é o token
  ver: user.token_version, # versão das sessões
  jti: SecureRandom.uuid, # id único deste token
  exp: 24.hours.from_now.to_i, # expiração
  iat: Time.current.to_i    # emitido em
}
```

O detalhe que quebra tudo

```
JWT.decode(token, secret, true, { algorithm: "HS256" })
#                               ^^^^^
```

Esse `true` é a validação da assinatura. Com `false`, o `JWT.decode` aceita **qualquer** token: e qualquer pessoa forja o próprio acesso trocando o `sub`. Já foi CVE em várias bibliotecas.

O teste que prova isso está em `me_controller_test.rb`:

```
test "recusa token assinado com outro segredo" do
  forjado = JWT.encode({ sub: users(:ana).id, ... }, "segredo-do-atacante", "HS256")
  get "/api/v1/me", headers: auth_headers(forjado)
  assert_response :unauthorized
end
```

7. Rota 2: Login

```
POST /api/v1/sessions { email, password, client }
```

`client` é `"mobile"` ou `"web"`, e decide **como** o token é entregue:

Cliente	Transporte	Por quê
<code>mobile</code>	cabeçalho <code>Authorization: Bearer <token></code>	App nativo não tem cookie
<code>web</code>	cookie assinado e <code>httpOnly</code>	JavaScript não lê o cookie, então um XSS não rouba o token

O que é um cookie

Um par `nome=valor` que o servidor manda no cabeçalho `Set-Cookie`. O navegador guarda e **reenvia sozinho**, em toda requisição àquele site. É a memória que o HTTP não tem, colada por fora.

Quem faz esse trabalho é o navegador: o app nativo não participa disso. Por isso o `client`.

```
cookies.signed[AUTH_COOKIE] = {
  value: token,
  httponly: true,           # JavaScript não enxerga
  secure: Rails.env.production?, # só trafega em HTTPS
  same_site: :lax          # mitiga CSRF
}
```

`signed` significa que o Rails carimba o valor: alterou na mão, o Rails recusa.

XSS

Cross-Site Scripting: o atacante consegue rodar **JavaScript dentro da sua página**. Como? Você exibiu texto de usuário sem escapar: alguém salvou `<script>...</script>` como nome e a sua página imprimiu cru.

Com JavaScript rodando ali, ele lê tudo que o JavaScript lê. É exatamente por isso que `httponly` existe: o token no cookie fica **fora do alcance** do JavaScript, e um XSS não o rouba.

CSRF

Cross-Site Request Forgery. Lembra que o navegador reenvia o cookie sozinho?

Um site malicioso monta um formulário apontando para a sua API. A vítima clica; o navegador anexa o cookie dela; a sua API obedece, achando que foi ela quem pediu.

`same_site: :lax` manda o navegador **não enviar o cookie** quando a requisição parte de outro site.

API com token no cabeçalho não sofre de CSRF: ninguém anexa o `Authorization` por você. O problema é exclusivo de quem autentica por cookie, ou seja, do nosso `client: "web"`.

401 × 403

```
when "email_unverified"
  status: :forbidden      # 403: sabemos quem é você, mas ainda não pode
else
  status: :unauthorized   # 401: não sabemos quem é você
end
```

A mesma mensagem para os dois erros

E-mail inexistente e senha errada devolvem `invalid_credentials`, o mesmo código e a mesma mensagem. Terceira vez que a enumeração aparece.

E lembra do `DUMMY_PASSWORD_DIGEST` da Aula 2? Ele fecha o buraco pelo lado do **tempo**: não adianta a mensagem ser igual se a resposta volta em 1ms quando o e-mail não existe e em 100ms quando existe.

8. Rota 3: Logout, o problema difícil

Um JWT é válido até expirar. O servidor não guarda sessão. Então **"sair" não existe**, o token continua funcionando por até 24h.

Três respostas possíveis, e nós usamos duas:

a) Apagar no cliente

O app joga o token fora. Resolve o caso normal e **não resolve nada** se alguém copiou o token.

b) Denylist por `jti`: revoga aquele token

```
module Auth::TokenDenylist
  def revoke(payload)
    jti = payload[:jti]
    ttl = payload[:exp].to_i - Time.current.to_i
    return false if jti.blank? || ttl <= 0

    Rails.cache.write(key(jti), true, expires_in: ttl.seconds)
  end

  def revoked?(jti)
    Rails.cache.exist?(key(jti))
  end
end
```

A sacada está no **TTL**: cada entrada vive exatamente o tempo que faltava para o token expirar. Depois disso o token morre sozinho e a entrada não serve mais. **A lista se limpa sem varredura e sem lixo acumulado.**

Isso revoga **um** token. Logout no celular não derruba o notebook: que é o comportamento certo.

c) `token_version`: derruba tudo de uma vez

```
def token_version_matches?(submitted_version)
  token_version == submitted_version.to_i
end

def invalidate_sessions!
  update!(token_version: token_version + 1)
end
```

O token carrega o `ver` de quando foi emitido. Incrementar a coluna no banco invalida **todos** os tokens do usuário, em todos os dispositivos. Usamos isso na troca de senha.

A verificação completa

```
def user_from_token(token)
  payload = Auth::DecodeToken.call(token)      # 1. a assinatura confere?
  return if Auth::TokenDenylist.revoked?(payload[:jti]) # 2. foi revogado?

  user = User.find_by(id: payload[:sub])
  user if user&.token_version_matches?(payload[:ver]) # 3. a versão bate?
rescue Auth::DecodeToken::InvalidToken
  nil
end
```

9. Rota 4: Recuperação de senha

Dois passos, e o mais interessante do dia.

```
POST /api/v1/password_resets/request { email }
POST /api/v1/password_resets/confirm { email, code, password, confirm_password }
```

`request` sempre responde 200

```
GENERIC_MESSAGE = "Se o e-mail estiver cadastrado, enviamos um código para redefinir sua senha."
```

Sempre. E-mail inexistente, não verificado, em cooldown, SMTP fora do ar: mesma resposta, mesmo status. Se respondesse 404 para e-mail inexistente, esta rota pública viraria um verificador de cadastro. É a quarta vez que o assunto aparece: **em fluxo de autenticação, respostas diferentes vazam informação.**

Duas guardas dentro:

- **Só conta verificada** recebe código. Senão dá para sequestrar um cadastro feito com o e-mail de outra pessoa antes de ela confirmar.
- **Cooldown de 1 minuto**, senão qualquer um usa o seu servidor de e-mail para incomodar terceiros.

`confirm` e a transação

```
User.transaction do
  user.save! # troca a senha
  user.invalidate_sessions! # derruba todas as sessões ativas
  user.clear_password_reset_code! # consome o código
end
```

As três juntas ou nenhuma. Sem a transação, uma falha no meio deixa a senha trocada com o código ainda valendo.

E a ordem das validações importa: a política de senha é conferida **antes** de consumir o código. Senão o usuário que digita a senha nova errada perde o código e precisa pedir outro.

`invalidate_sessions!` **é o ponto do fluxo inteiro**. Se alguém invadiu a conta, trocar a senha tem que expulsar o invasor. Sem essa linha, ele continua logado com o token antigo.

10. Testando

```
bin/rails test
```

O teste que resume a aula:

```
test "logout revoga o token apresentado" do
  token = login_as(users(:ana))

  get "/api/v1/me", headers: auth_headers(token)
  assert_response :ok

  delete "/api/v1/sessions", headers: auth_headers(token)

  get "/api/v1/me", headers: auth_headers(token)
  assert_response :unauthorized # sem a denylist, isto daria 200
end
```

Pegadinha: em teste o `Rails.cache` padrão é o `null_store`. Não guarda nada. O teste acima passaria "de graça", provando nada. Por isso o `test_helper.rb` troca por um `MemoryStore`.

As ferramentas de qualidade

```
bin/rubocop      # estilo – o black/ruff do Ruby
bin/brakeman     # análise estática de segurança
bundle exec bundler-audit check --update # gems com CVE conhecido
```

As três rodam no CI (Aula 4). O `brakeman` procura SQL injection, mass assignment, redirect aberto e mais uma dúzia de padrões.

11. CORS

```
origins(ENV.fetch("CORS_ORIGINS", "http://localhost:5173").split(","))
```

O navegador bloqueia, por padrão, uma página em `painel.exemplo.com` chamar `api.exemplo.com`. Esta config é a API dizendo quais origens aceita.

O app nativo não passa por CORS: isso é regra de navegador. Na prática, essa configuração existe por causa do painel web.

12. O fluxo inteiro, no terminal

```
API=localhost:3000/api/v1

# cadastro
curl -X POST $API/registrations -H 'Content-Type: application/json' -d '{
  "name": "Diego Reis", "email": "diego@aluno.ufop.edu.br",
  "password": "Automic@2026", "confirm_password": "Automic@2026",
  "course": "Engenharia de Minas", "matricula": "2013333", "check_terms_use": true}'

# login antes de confirmar → 403 email_unverified
curl -X POST $API/sessions -H 'Content-Type: application/json' \
  -d '{"email": "diego@aluno.ufop.edu.br", "password": "Automic@2026", "client": "mobile"}'

# pegue o código no e-mail que abriu no navegador, e confirme
curl -X POST $API/email_verifications/confirm -H 'Content-Type: application/json' \
  -d '{"email": "diego@aluno.ufop.edu.br", "code": "123456"}'

# login (o token vem no cabeçalho Authorization da resposta)
curl -i -X POST $API/sessions -H 'Content-Type: application/json' \
  -d '{"email": "diego@aluno.ufop.edu.br", "password": "Automic@2026", "client": "mobile"}'

TOKEN=cole-o-token-aqui
curl $API/me -H "Authorization: Bearer $TOKEN" # 200
curl -X DELETE $API/sessions -H "Authorization: Bearer $TOKEN"
curl $API/me -H "Authorization: Bearer $TOKEN" # 401
```

As práticas da aula

São 1400 linhas em 37 arquivos. **Ninguém digita isso em três horas**, e não é esse o objetivo. Aqui você **traz o código pronto** e passa a aula fazendo o sistema funcionar, quebrando de propósito e entendendo *por que* cada peça existe.

Oito práticas. As sete primeiras são o fluxo real, na ordem em que um usuário o vive.

#	O que	Depois de
1	Traga o código e leia a arquitetura	seção 2
2	Cadastro, no terminal e no Insomnia	seção 3
3	O e-mail e a confirmação da conta	seção 5
4	Login, e o token na mão	seção 7
5	Logout que revoga de verdade	seção 8
6	Recuperação de senha, e as sessões que caem	seção 9
7	Testes e qualidade	seção 10
8	Avançado: quebre a assinatura do token	fim

Prática 1: Traga o código e leia a arquitetura

Pouco comando e muita leitura. A leitura é a parte que conta.

```
cd ~/capacitacao-gabarito && git checkout aula-03
rsync -a app config db test Gemfile Gemfile.lock ~/automic_auth_api/

cd ~/automic_auth_api
bundle install
bin/rails db:migrate
bin/rails test
```

Confere: 71 testes verdes. Se não deram, pare aqui e resolva antes de seguir. Todas as práticas seguintes dependem disso.

O `Gemfile` vai junto porque esta aula acrescenta `jwt`, `rack-cors` e `letter_opener`. Sem ele a aplicação nem sobe.

Agora leia cinco arquivos, **nesta ordem, abrindo cada um no editor**:

Arquivo	A pergunta que ele responde
<code>app/services/users/register.rb</code>	Por que a regra não fica no controller?
<code>app/services/auth/issue_token.rb</code>	O que exatamente vai dentro do token?
<code>app/controllers/concerns/api/v1/authentication.rb</code>	Como o servidor sabe quem está chamando?
<code>app/services/auth/token_denylist.rb</code>	Como um JWT deixa de valer antes de expirar?
<code>app/services/users/complete_password_reset.rb</code>	Por que tudo numa transação?

Em cada um, ache o `Struct` de retorno e os argumentos nomeados da Aula 1. **Eles estão em todos.**

Se der errado

Se os 71 testes não passarem de primeira, quase sempre é o `Gemfile` que ficou para trás no `rsync`, e o erro que aparece (`uninitialized constant Rack::Cors`) não diz isso em lugar nenhum. É um caso clássico de mensagem de erro que aponta para o sintoma e não para a causa. Você vai ver muitos assim.

Erro	Causa	Saída
<code>uninitialized constant Rack::Cors</code>	o <code>Gemfile</code> não veio junto no <code>rsync</code>	refaça o <code>rsync</code> incluindo <code>Gemfile Gemfile.lock</code> , e <code>bundle install</code>
<code>Could not find gem 'jwt'</code>	idem	<code>bundle install</code>
<code>PendingMigrationError</code>	as migrations novas não rodaram	<code>bin/rails db:migrate</code>
menos de 71 testes	o <code>rsync</code> não trouxe <code>test/</code>	confira que <code>test/</code> estava na lista
<code>rsync: command not found</code>	não instalado	<code>sudo apt install rsync</code> / <code>brew install rsync</code>
o <code>rsync</code> jogou tudo na raiz do projeto	você usou caminho errado	os caminhos são relativos e a barra final importa; refaça exatamente como está escrito
conflito com arquivos seus da Aula 2	o <code>rsync</code> sobrescreveu	é o esperado: a partir daqui o gabarito é a base

Prática 2: Cadastro, no terminal e no Insomnia

A primeira rota que cria alguma coisa.

Com o servidor rodando (`bin/rails server`), num segundo terminal:

```
API=localhost:3000/api/v1
EMAIL=voce@aluno.ufop.edu.br

curl -i -X POST $API/registrations -H 'Content-Type: application/json' -d "{
  \"name\": \"Seu Nome\", \"email\": \"$EMAIL\",
  \"password\": \"Automic@2026\", \"confirm_password\": \"Automic@2026\",
  \"course\": \"Engenharia\", \"matricula\": \"2013333\", \"check_terms_use\": true}"
```

Confere: `201`, e o e-mail abriu numa aba do navegador (é o `letter_opener`). **Anote o código de 6 dígitos.**

Agora monte a mesma requisição no **Insomnia**, e aproveite para criar a coleção inteira: você vai usá-la o resto da aula. Todas com `Content-Type: application/json`:

```

POST /api/v1/registrations
POST /api/v1/email_verifications/confirm
POST /api/v1/email_verifications/resend
POST /api/v1/sessions
DELETE /api/v1/sessions
POST /api/v1/password_resets/request
POST /api/v1/password_resets/confirm
GET /api/v1/me

```

Avançado: mande o cadastro de novo, com o mesmo e-mail. Que status vem? E que mensagem? Ela entrega ao atacante que aquele e-mail já existe?

Se der errado

Erro	Causa	Saída
<code>400 param is missing</code>	faltou um campo obrigatório no JSON	a mensagem diz qual; o <code>params.expect</code> é rígido de propósito
<code>422</code> com lista de erros	as validações da Aula 2 recusaram	leia a lista: senha fraca, e-mail inválido, termos não aceitos
<code>415 Unsupported Media Type</code>	faltou o <code>-H 'Content-Type: application/json'</code>	acrescente
o e-mail não abriu no navegador	<code>letter_opener</code> só funciona com o servidor rodando no seu desktop	veja em <code>tmp/letter_opener/</code> ou no log do <code>rails server</code>
<code>Connection refused</code>	o <code>rails server</code> não está de pé	suba-o no outro terminal
a resposta veio em HTML de erro	exceção não tratada	leia o log do <code>rails server</code> , não o <code>curl</code>
aspas quebrando no shell	o JSON tem aspas dentro de aspas	use o Insomnia, ou salve o JSON num arquivo e use <code>-d @arquivo.json</code>

Prática 3: O e-mail e a confirmação da conta

É aqui que fica claro por que existe conta "não ativada".

```

# tente entrar ANTES de confirmar
curl -i -X POST $API/sessions -H 'Content-Type: application/json' \
  -d '{"email\\":\\"$EMAIL\\",\\"password\\":\\"Automic@2026\\",\\"client\\":\\"mobile\\"}'

```

Confere: `403 email_unverified`. A senha está certa. O que falta é a conta estar ativa.

```
curl -i -X POST $API/email_verifications/confirm -H 'Content-Type: application/json' \
-d '{"email":"$EMAIL","code":"COLE_O_CODIGO"}'
```

Avançado: mande o **mesmo código** de novo. Ele funciona duas vezes? Por quê? (Volte à Prática 7 da Aula 2 se precisar.)

Se der errado

Erro	Causa	Saída
<code>422 invalid_code</code> com o código certo	você colocou com espaço, ou o código já foi usado	peça outro com <code>/email_verifications/resend</code>
<code>422 code_expired</code>	passaram os 15 minutos	<code>/email_verifications/resend</code>
<code>429</code> ou "aguarde" no resend	há um intervalo mínimo entre reenvios	espere o tempo indicado: é proteção contra abuso
<code>403 email_unverified</code> mesmo depois de confirmar	você confirmou outro e-mail	confira o <code>\$EMAIL</code> do shell
não acho o código	a aba do <code>letter_opener</code> fechou	<code>ls tmp/letter_opener/</code> e abra o mais recente

Prática 4: Login, e o token na mão

O núcleo da aula.

```
TOKEN=$(curl -s -D- -o /dev/null -X POST $API/sessions -H 'Content-Type: application/json' \
-d '{"email":"$EMAIL","password":"Automic@2026","client":"mobile"}' \
| grep -i '^authorization:' | sed 's/.*Bearer //I' | tr -d '\r\n')
echo $TOKEN

curl -i $API/me -H "Authorization: Bearer $TOKEN" # 200
```

Confere: `200`, com os seus dados. Repare que o servidor **não guardou sessão nenhuma**. Ele descobriu quem é você lendo o token.

Agora **leia o seu próprio token**: cole o `$TOKEN` em jwt.io. Ache o `sub`, o `jti`, o `ver` e o `exp`. Converta o `exp` para data e confirme que é daqui a 24 horas.

Avançado: troque um caractere do token e chame `/me` de novo. Que status vem, e por quê?

```
curl -i $API/me -H "Authorization: Bearer ${TOKEN}x"
```

Se der errado

Erro	Causa	Saída
<code>\$TOKEN</code> saiu vazio	o <code>grep</code> não achou o cabeçalho	rode o <code>curl -i</code> sozinho e veja se o <code>Authorization:</code> está na resposta
<code>401</code> logo depois do login	o token não foi copiado inteiro	<code>echo \$TOKEN wc -c</code> : tem que ter centenas de caracteres
<code>401 invalid_token</code>	você colocou com quebra de linha	o <code>tr -d '\r\n'</code> do comando existe para isso
<code>403</code> em vez de <code>200</code>	a conta não está verificada	volte à Prática 3
<code>401 invalid_credentials</code>	senha errada: ou e-mail que não existe	é a mesma mensagem de propósito; veja a seção 7
jwt.io diz "invalid signature"	esperado: ele não tem o seu segredo	você só quer ler o payload, não validar

Prática 5: Logout que revoga de verdade

A prática que mostra a diferença entre "apagar no cliente" e "revogar no servidor".

Antes de rodar, decida: aquele token continua matematicamente válido e ainda não expirou. O segundo comando vai responder `200` ou `401`? A resposta é o assunto inteiro da seção 8.

```
curl -i -X DELETE $API/sessions -H "Authorization: Bearer $TOKEN"
curl -i $API/me -H "Authorization: Bearer $TOKEN"
```

Confere: o segundo comando tem que dar `401`. O token ainda é válido e ainda não expirou, mas está na denylist.

Sem a denylist, esse `curl` responderia `200` até o `exp` chegar. **É este teste que prova que o logout serve para alguma coisa.**

Avançado: veja a denylist por dentro, no console:

```
Rails.cache.read("denylist:#{jti_do_seu_token}")
```

Pegue o `jti` no jwt.io. Por que a entrada tem TTL, e por que o TTL é exatamente o que restava do token?

Se der errado

Erro	Causa	Saída
o segundo <code>curl</code> respondeu <code>200</code>	o <code>DELETE</code> falhou antes	rode-o com <code>-i</code> e leia o status
<code>401</code> já no <code>DELETE</code>	o token expirou ou já foi revogado	faça login de novo e refaça
a denylist não persiste entre reinícios	o cache em dev é em memória	é o esperado em desenvolvimento; em produção é o Solid Cache no Postgres
<code>Rails.cache.read</code> devolve <code>nil</code>	<code>jti</code> errado, ou cache diferente	copie o <code>jti</code> exato do jwt.io

Prática 6: Recuperação de senha, e as sessões que caem

O fluxo mais completo do sistema, e o que tem a decisão de segurança mais sutil.

Faça login de novo para ter um token válido:

```
TOKEN=$(curl -s -D- -o /dev/null -X POST $API/sessions -H 'Content-Type: application/json' \
-d '{"email":"'${EMAIL}'","password":"'${AUTOMIC}@2026'","client":"'mobile'"}' \
| grep -i '^authorization:' | sed 's/.*Bearer //' | tr -d '\r\n')

curl -X POST $API/password_resets/request -H 'Content-Type: application/json' \
-d '{"email":"'${EMAIL}'"}'
# pegue o código no navegador

curl -i -X POST $API/password_resets/confirm -H 'Content-Type: application/json' -d '{
  "email":"'${EMAIL}'","code":"'${CODE}'",
  "password":"'NovaSenha@9'","confirm_password":"'NovaSenha@9'"}'

curl -i $API/me -H "Authorization: Bearer $TOKEN"
```

Confere: o último dá `401`. Trocar a senha **derrubou todas as sessões abertas**. É o `token_version` sendo incrementado. Se alguém tinha roubado o seu token, acabou de perdê-lo.

Confere também, e é o ponto mais importante da prática:

```
curl -i -X POST $API/password_resets/request -H 'Content-Type: application/json' \
-d '{"email":"'ninguem-existe@ufop.br'"}'
```

Responde **exatamente a mesma coisa** do e-mail que existe. É a resistência a enumeração: um atacante não consegue usar esta rota para descobrir quem tem conta.

Se der errado

Erro	Causa	Saída
o <code>/me</code> final respondeu <code>200</code>	o <code>confirm</code> falhou	rode o <code>confirm</code> com <code>-i</code> e leia o status
<code>422</code> no <code>confirm</code>	senha nova não passa na política, ou não bate com a confirmação	leia a lista de erros
<code>422 invalid_code</code>	código errado, expirado, ou já usado	peça outro com <code>/password_resets/request</code>
não veio e-mail para o endereço inexistente	correto!	é justamente o que se espera: resposta igual, e-mail nenhum
esqueceu a senha nova	você trocou para <code>NovaSenha@9</code>	use ela nos próximos logins

Prática 7: Testes e qualidade

```
bin/rails test
bin/rubocop
bin/brakeman --no-pager
```

Confere: 71 testes verdes, RuboCop limpo, Brakeman sem aviso.

Agora **quebre um teste de propósito** para ver a rede de segurança funcionando. Em `app/controllers/concerns/api/v1/authentication.rb`, comente a linha que consulta a denylist. Rode `bin/rails test` e veja **qual** teste fica vermelho. Depois desfaça.

```
git add -A && git commit -m "Rotas de autenticação" && git push
```

Se der errado

Erro	Causa	Saída
testes vermelhos depois de você mexer	é o objetivo do exercício	<code>git checkout .</code> desfaz tudo o que não foi commitado
<code>bin/rubocop</code> com dezenas de ofensas	estilo	<code>bin/rubocop -a</code> conserta a maioria; leia o que sobrar
<code>bin/brakeman</code> acusa	pode ser falso positivo	leia a explicação; ele diz o arquivo e a linha
<code>bin/brakeman: command not found</code>	a gem não veio	<code>bundle install</code>
o push foi recusado	o remoto tem commits novos	<code>git pull --rebase origin main</code>

Prática 8: quebre a assinatura do token (avançado)

Se sobrou tempo. É o exercício que faz entender o que uma assinatura garante.

Em `app/services/auth/decode_token.rb`, troque o `true` por `false`:

```
JWT.decode(@token, TokenSecret.call, false, { algorithm: "HS256" })
```

Esse terceiro parâmetro é "verifique a assinatura?". Agora forje um token assinado com **outro segredo**:

```
bin/rails runner 'puts JWT.encode({sub: User.first.id, ver: 0, jti: "x",  
  exp: 1.hour.from_now.to_i}, "segredo-do-atacante", "HS256")'
```

```
curl -i $API/me -H "Authorization: Bearer <o-token-forjado>"
```

Confere: responde `200`. Você acabou de entrar como outra pessoa, sem saber a senha dela.

Desfaça em seguida (volte o `true`) e rode `bin/rails test`: o teste "recusa token assinado com outro segredo" fica vermelho enquanto o `false` estiver lá. **Era esse teste que estava te protegendo.**

```
git checkout app/services/auth/decode_token.rb  
bin/rails test
```

Se der errado

Erro	Causa	Saída
o token forjado deu <code>401</code> mesmo com <code>false</code>	o <code>ver</code> não bate com o <code>token_version</code> do usuário	use <code>ver: User.first.token_version</code>
<code>JWT::DecodeError</code>	o token foi colado quebrado	copie a linha inteira, sem quebra
esqueceu de desfazer	perigoso: é uma falha de autenticação	<code>git checkout app/services/auth/decode_token.rb</code>

Bônus

`PATCH /api/v1/me` para o usuário editar o próprio `name`, e só o `name`. Tente mandar `"role": "admin"` junto e confirme que o `params.expect` recusa. É *mass assignment*, e é uma das falhas mais comuns em API.

Travou em qualquer prática? O gabarito é o mesmo código: `cd ~/capacitacao-gabarito && git checkout aula-03`.

Recapitulando

- Controller recebe e responde; a regra mora no service.
- Serializer é a lista de convidados do JSON: sem ele o digest vaza.
- O payload do JWT é público. Assinado ≠ criptografado.
- Logout de verdade precisa de denylist por `jti`; o TTL faz a limpeza sozinho.
- `token_version` derruba tudo de uma vez, e é o que a troca de senha usa.
- Em autenticação, respostas diferentes vazam informação. Uma resposta só, sempre.

Na próxima: nada disso serve se está só na sua máquina. Vamos colocar no ar.

Aula 4: Servidor, Docker, Kamal e deploy

Você sai daqui com: a sua API no ar, em `https://seunome.test`, servida por um servidor Linux de verdade que você mesmo subiu, com HTTPS e deploy por um comando. **Gabarito:** `cd ~/capacitacao-gabarito && git checkout aula-04`

Este roteiro é a versão para a turma do guia de infraestrutura do `seem-backend`. As decisões são as mesmas; a máquina é menor, e hoje ela mora no seu notebook.

O último dia

Você tem uma API com cinco fluxos de autenticação, testada, rodando com `bin/rails server`. **Hoje ela deixa de ser "um comando que eu rodo" e vira um sistema publicado:** sobe sozinha, aceita HTTPS, guarda os dados num lugar que sobrevive a reinício, e você consegue atualizar e voltar atrás sem ninguém perceber.

As ferramentas que fazem isso têm nome, e você vai conhecer cada uma no caminho: Docker empacota, um registry distribui, o Kamal publica, e um proxy termina o HTTPS.

E o servidor de hoje é **a sua própria máquina**. Não porque seja um faz de conta: é porque tudo o que você vai fazer aqui é idêntico ao que se faz numa máquina alugada, e num encontro de três horas o tempo é melhor gasto entendendo o deploy do que esperando um provedor liberar cota.

Duas coisas que ajudam:

Um passo mal feito só aparece três passos depois. Por isso toda prática tem uma linha **Confere**. Não siga com ela vermelha, mesmo que pareça que dá.

Hoje você vai encontrar mais erro do que nos outros encontros. É assunto novo e ferramenta nova, e a maioria dos erros não tem nada a ver com programar. É assim para todo mundo, sempre.

E o fecho, para você já saber onde vai chegar: no fim do dia, três comandos `curl` vão te mostrar, na tela, a diferença entre criptografia e confiança. É a Prática 6, e vale a aula.

1. O que é colocar no ar

O problema

`bin/rails server` funciona enquanto o seu terminal está aberto. Não é isso que um sistema em produção faz. Um sistema publicado precisa de:

- **subir sozinho** e continuar de pé sem ninguém olhando;
- **as mesmas versões** de tudo, sempre, em qualquer máquina;
- **atualizar sem sair do ar** quando você faz uma correção;
- **voltar atrás** quando a correção estava errada;
- guardar **segredo** de um jeito que não vaze no histórico do shell.

Tudo isso é o assunto de hoje, e nenhuma dessas cinco coisas depende de onde a máquina está.

As quatro opções de hospedagem

Opção	O que você controla	O que você opera
Hospedagem compartilhada	quase nada	nada
PaaS (Heroku, Render, Fly)	o app	nada, mas paga mais e obedece as regras deles
VPS	tudo: SO, firewall, banco	tudo
Kubernetes	tudo, em escala	muito mais

VPS = *Virtual Private Server*: um computador virtual dentro do servidor físico de um provedor, que é seu. Você tem acesso root, escolhe o sistema e instala o que quiser. É o que o `seem-backend` usa.

Por que uma VM com containers, e não Kubernetes

O `seem-backend` atende ~500 usuários num evento de uma semana. Kubernetes adicionaria custo e operação sem resolver um problema que existe. A escolha foi:

- menos peças para monitorar;
- deploy reproduzível com Docker e Kamal;
- banco e aplicação perto um do outro;
- recuperação simples numa máquina substituta;
- **evoluir só quando uma métrica real pedir.**

Escolher a ferramenta grande antes do problema grande é a forma mais cara de errar.

O que isso custa, e é honesto admitir: uma máquina só é ponto único de falha. Volume Docker não é backup. O Postgres não é gerenciado, e quem cuida dele é você.

Hoje: a sua máquina é o servidor

O Kamal não sabe onde a máquina está. Ele conecta por **SSH**, roda `docker` do outro lado, e pronto. Se o endereço for `127.0.0.1`, ele conecta na sua máquina; se for um IP público, na de lá. **É a mesma conexão e o mesmo arquivo de configuração.**

```
seu terminal —ssh—> a máquina que serve —docker—> os containers
```

Então é isto que muda quando você trocar por um servidor alugado:

	Hoje	Num servidor alugado
<code>SERVER_IP</code> no <code>.env</code>	<code>127.0.0.1</code>	o IP público
<code>KAMAL_SSH_USER</code>	o seu login	o usuário que o provedor criou
Certificado	autoassinado, e você confia nele na mão	emitido por uma CA pública
Antes disso	nada	provisionar a máquina: usuário, firewall, Docker
Quem tenta invadir	ninguém	bots, o dia todo, desde o primeiro minuto

O resto é idêntico: o `Dockerfile`, o `deploy.yml`, o registry, os segredos, o volume do banco, o zero-downtime, o rollback. É por isso que aprender aqui vale.

A seção **10** mostra o percurso numa máquina de verdade, e o apêndice `apendice-azure.md` tem o passo a passo completo para você fazer em casa.

2. A sua máquina vira o servidor

O Kamal precisa de duas coisas do outro lado: **um SSH que aceite a sua chave** e **o Docker rodando**. O Docker você já tem desde a Aula 1. Falta o SSH.

Onde você trabalha hoje: no mesmo lugar de sempre. Se você usa Windows, é dentro do **WSL2**, que é o seu Linux e vai ser o seu servidor. Se usa Linux ou macOS, é o próprio sistema. Você não vai instalar máquina virtual nenhuma.

Antes: duas chaves, um par

Criptografia assimétrica usa duas chaves que se completam. A **pública** você espalha; a **privada** nunca sai da sua máquina. O que uma fecha, só a outra abre.

Daí saem duas coisas diferentes:

- **sigilo:** eu fecho com a *sua* pública, e só você abre;
- **assinatura:** eu fecho com a *minha* privada, e todo mundo confere que fui eu.

É assim que o SSH funciona. Você põe a sua chave pública no servidor (`~/.ssh/authorized_keys`). Ao conectar, o servidor manda um desafio; você responde assinando com a privada; o servidor confere com a pública que já tinha. **A senha nunca trafega, nem existe.**

E é por isso que perder a chave privada é perder o acesso à máquina.

Compare com o JWT da Aula 3: lá a assinatura usa uma chave **simétrica** (HS256), porque quem assina e quem confere são o mesmo servidor.

Instalar o servidor de SSH

Você já usa o **cliente** de SSH (é o comando `ssh`). O que falta é o **servidor**, o programa que fica escutando na porta 22 e atende quem chega.

```
# Ubuntu, Debian e WSL2
sudo apt update && sudo apt install -y openssh-server
sudo service ssh start

# Fedora
sudo dnf install -y openssh-server && sudo systemctl enable --now sshd
```

No **macOS** não se instala nada: o servidor já vem no sistema, e você só liga em **Ajustes do Sistema** → **Geral** → **Compartilhamento** → **Sessão remota**.

Confira que ele está escutando:

```
ss -tlnp | grep :22      # Linux e WSL2
sudo lsof -i :22         # macOS
```

WSL2: o `sshd` não sobe sozinho a cada boot, a não ser que você habilite o systemd. Se depois de reiniciar o Windows o `ssh` parar de conectar, rode `sudo service ssh start` de novo. É o tropeço mais comum do dia, e a saída é uma linha.

O par de chaves da capacitação

Não reaproveite a chave do GitHub. Uma chave por finalidade: dá para revogar uma sem derrubar a outra.

```
ssh-keygen -t ed25519 -f ~/.ssh/capacita -C "capacita-servidor" -N ""
chmod 600 ~/.ssh/capacita
```


`chmod 600` significa "só o dono lê e escreve". O SSH **recusa** usar uma chave com permissão mais frouxa, e essa recusa é a primeira coisa a conferir quando aparecer `Permission denied (publickey)`.

Chave privada não vai por e-mail, não vai por WhatsApp, não vai para o Git. Nunca.

Autorizar a si mesmo

Agora a parte que costuma causar um sorriso: você põe a sua chave pública no `authorized_keys` da sua própria máquina. É exatamente o que você faria num servidor alugado, e é literalmente o mesmo comando.

```
mkdir -p ~/.ssh && chmod 700 ~/.ssh
cat ~/.ssh/capacita.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

E entre:

```
ssh -i ~/.ssh/capacita -o IdentitiesOnly=yes "$USER"@127.0.0.1
```

Você abriu uma sessão SSH da sua máquina para ela mesma. **Não é truque:** é uma conexão de rede de verdade, autenticada por chave, exatamente como a que o Kamal vai abrir daqui a pouco. Saia com `exit`.

O `IdentitiesOnly=yes` importa: sem ele o SSH oferece todas as chaves do seu agente, o servidor recusa depois de algumas, e você leva `Too many authentication failures` com a chave certa na mão.

Permissões, já que estamos aqui

Todo arquivo no Linux tem dono, grupo e três permissões: ler, escrever e executar.

```
chmod 600 arquivo    # o dono lê e escreve; mais ninguém vê nada
chmod 700 pasta      # o dono entra; mais ninguém
```

O SSH é rígido com isso de propósito: uma chave privada que o resto do sistema consegue ler não é uma chave privada. É a mesma lógica do `password_digest` da Aula 2, aplicada a arquivo.

E nunca rode a aplicação como `root`: o Dockerfile que você vai ver na próxima seção cria um usuário `rails` justamente para isso.

3. Docker

Imagem × container

Imagem é a receita: o sistema, o Ruby, as gems, o seu código, congelados. **Container** é o bolo: uma instância rodando. Uma imagem, muitos containers.

O Dockerfile do Rails

O Rails 8 gera um Dockerfile de produção pronto. Vale ler:

```
FROM ruby:3.4.9-slim AS base
# ...

FROM base AS build
RUN apt-get install -y build-essential libpq-dev ...
COPY Gemfile Gemfile.lock ./
RUN bundle install
COPY . .

FROM base
RUN groupadd --system --gid 1000 rails && useradd rails --uid 1000 --gid 1000
USER 1000:1000
COPY --from=build "${BUNDLE_PATH}" "${BUNDLE_PATH}"
COPY --from=build /rails /rails
```

Duas decisões importantes:

Multi-stage. O estágio `build` instala compilador e headers para compilar as gems nativas. A imagem final copia só o resultado. Compilador não vai para produção: menos peso e menos ferramenta à mão de quem invadir.

Usuário não-root. Se alguém escapar da aplicação, cai num usuário sem privilégio. É uma linha que muda o tamanho do estrago.

`.dockerignore`

Diz o que **não** entra na imagem: `.git`, `log/`, `tmp/`, `node_modules`. Sem ele a imagem fica gorda e, pior, o histórico do Git vai junto.

Registry

O registry é o "GitHub das imagens". Usamos o **ghcr.io** (GitHub Container Registry): já vem com a sua conta do GitHub.

O fluxo desta aula:

```
você builda → empurra a imagem para o ghcr.io → o servidor puxa e roda
```

Hoje quem builda e quem roda são a mesma máquina, e mesmo assim a imagem sobe para o GitHub e desce de lá. Parece rodeio, e é de propósito: é exatamente o que aconteceria com um servidor do outro lado do mundo, e é o que faz o mesmo `deploy.yml` servir nos dois casos sem mudar uma linha.

O token do registry

Fora do GitHub Actions não existe `GITHUB_TOKEN`. Crie um **Personal Access Token (classic)**:

GitHub → **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token**, com os escopos `write:packages` e `read:packages`.

Guarde: é o seu `KAMAL_REGISTRY_PASSWORD`. Ele é mostrado **uma vez**.

Deixe o pacote público. Na conta grátis do GitHub, pacote **privado** tem uma cota de armazenamento pequena, e uma imagem Rails come quase toda ela. Pacote **público** é ilimitado e grátis, e não expõe o seu código: o repositório continua privado, e o que fica público é só a imagem já compilada.

Depois do primeiro deploy, em **github.com/SEU-USUARIO?tab=packages** → **automatic-auth-api** → **Package settings** → **Change visibility** → **Public**.

Um token com escopo de pacote e nada mais. É o mesmo princípio da chave SSH separada: quando vazar e um dia vaza, o estrago tem tamanho.

Arquitetura

O Kamal builda a imagem e depois a roda. Hoje as duas coisas acontecem na mesma máquina, então a arquitetura é a sua:

Seu notebook	SERVER_ARCH
Intel ou AMD	amd64
Mac com chip M1/M2/M3/M4	arm64

Descubra com:

```
uname -m          # x86_64 é amd64; aarch64 ou arm64 é arm64
```

4. Kamal

O Kamal (feito pela mesma turma do Rails) faz *zero-downtime deploy* com Docker, via SSH. Sem agente e sem painel: ele conecta na máquina que serve e roda `docker`.

O que ele faz num `kamal deploy`:

1. builda a imagem e empurra para o registry
2. conecta no servidor por SSH
3. puxa a imagem
4. sobe o container novo **ao lado** do antigo
5. espera o **health check** do novo passar, que é uma rota simples (a nossa é `/up`) que responde 200 quando a aplicação está de pé
6. manda o tráfego para o novo e derruba o antigo

Falhou o health check? Ele não troca. O antigo continua servindo.

`config/deploy.yml`, por partes

```
service: automic-auth-api
image: <%= ENV.fetch("GHCR_USER") %>/automic-auth-api
```

O arquivo é ERB: dá para usar `ENV`. É por isso que este `deploy.yml` é **igual para todo mundo da turma**, e é o mesmo que serviria numa VPS. O que muda são as variáveis.

```
ssh:
  user: ubuntu
  run_directory: /home/ubuntu/.kamal
```

Sem login root, então o Kamal precisa de um lugar para gravar lock e auditoria.

```
servers:
  web:
    hosts:
      - <%= ENV.fetch("SERVER_IP") %>
```

`SERVER_IP` é `127.0.0.1` hoje: a sua máquina. Num servidor alugado, é o IP público dele. Uma variável.

O bloco `ssh` também lê `SSH_PORT`, com padrão 22: é o que faz o Kamal funcionar sem mudança nenhuma para quem alcança a VM por encaminhamento de porta.

```
env:
  clear:
    SOLID_QUEUE_IN_PUMA: "true"
    WEB_CONCURRENCY: "1"
    DB_HOST: automic-auth-api-db
  secret:
    - RAILS_MASTER_KEY
    - JWT_SECRET
```

- `clear` vai como texto no comando `docker run`. Configuração, não segredo.
- `secret` vai por um arquivo de ambiente, com permissão restrita no servidor.

Colocar `JWT_SECRET` no `clear` seria o mesmo que publicá-lo: ele apareceria em `docker inspect` e no histórico do shell.

`DB_HOST: automic-auth-api-db` é o **nome do container** do banco. Containers na mesma rede Docker se enxergam por nome: não precisa de IP.

```
accessories:
  db:
    image: postgres:16
    directories:
      - data:/var/lib/postgresql/data
```

Accessory é um container que o Kamal gerencia mas não faz deploy junto com o app. O `directories` cria o volume: é ele que faz os dados sobreviverem a deploy e a rebuild.

`kamal deploy` **não** sobe accessories. Depois de mudar env de accessory: `kamal accessory reboot db`.

As três camadas de segredo

```
o seu shell (source .env)
  ↓ variáveis de ambiente na sua sessão
.kamal/secrets
  ↓ referencia as variáveis – nenhum valor aqui, por isso vai para o Git
config/deploy.yml (env.secret)
  ↓ o Kamal injeta no container
ENV["JWT_SECRET"] no Rails
```

Nenhum valor real toca o repositório em nenhum ponto. Num pipeline de CI a primeira camada vira os Secrets do GitHub Actions; **as outras três não mudam nada**.

5. Segredos do Rails

`credentials` e `master.key`

```
bin/rails credentials:edit
```

O Rails abre um YAML no editor, e ao salvar grava `config/credentials.yml.enc`: criptografado, seguro no Git. A chave que decripta é `config/master.key`, que **nunca** vai para o Git (já está no `.gitignore`).

Em produção, `master.key` vira a variável `RAILS_MASTER_KEY`.

Antes: o que é uma variável de ambiente

Um par `nome=valor` que o sistema operacional entrega ao processo quando ele sobe.

```
export JWT_SECRET=abc123
```

E o programa lê com `ENV["JWT_SECRET"]`.

Por que não deixar o valor no código? Porque o código vai para o Git. A ideia é: mesmo código, ambientes diferentes, valores diferentes. É um dos [12 fatores](#), e é como o Kamal injeta segredo no container.

Credentials × ENV

	Quando
Credentials	segredo estável, que é do app: chave de API de terceiro
ENV	o que muda por ambiente ou por máquina: senha do banco, host de SMTP

Este projeto usa ENV para quase tudo, porque quem provisiona (Kamal) já sabe injetar.

Gere o `JWT_SECRET`:

```
bin/rails secret
```

O seu `.env`

O Kamal lê o `.kamal/secrets`, que só **referencia** variáveis do seu shell. Quem põe valor nelas é você. Copie o exemplo e preencha:

```
cd ~/automic_auth_api
cp .env.example .env
$EDITOR .env
```

E carregue antes de qualquer comando do Kamal:

```
source .env
```

O `.env` está no `.gitignore`: confira antes de commitar. **Este é o arquivo que não pode vaziar.**

6. HTTPS na sua máquina

HTTPS é HTTP dentro de um túnel

O HTTP da Aula 1 é **texto puro**: quem estiver no caminho, o roteador do café ou o provedor, lê tudo, inclusive a senha. O **TLS** embrulha esse texto num túnel criptografado.

Mesmo protocolo, mesmos verbos, mesmos cabeçalhos, só que fechado. O "S" de HTTPS é isso, e nada além disso.

O handshake, em três passos

1. O servidor apresenta o **certificado** dele.
2. O cliente confere se confia em **quem assinou** aquele certificado.
3. Os dois combinam uma chave temporária e conversam com ela.

O par de chaves assimétricas só serve para o passo 3. Depois disso a conversa usa criptografia **simétrica**, que é muito mais rápida.

Certificado e autoridade

Um certificado diz: *"esta chave pública pertence a este domínio"*, e vem **assinado por uma Autoridade Certificadora (CA)**.

O seu navegador já nasce com uma lista de CAs em que confia. Confia na CA → confia em quem ela assinou. É uma cadeia.

- **Autoassinado** é você jurando que é você.
- **Let's Encrypt** é uma CA pública e gratuita, em que os navegadores confiam.

Para conseguir um certificado de CA pública é preciso **provar que o domínio é seu**, e você não tem domínio nenhum apontando para a sua máquina. Então hoje o certificado é autoassinado, e você vai ver, com os próprios olhos, o que isso significa.

Proxy reverso

Um proxy comum fica na frente do **cliente**. Um proxy **reverso** fica na frente do **servidor**: recebe tudo na 443, termina o TLS, e repassa para a aplicação em HTTP interno.

Assim o Rails não precisa saber nada de certificado:

```
seu navegador → kamal-proxy → Rails
```

É por isso que `config.assume_ssl = true`: ele avisa o Rails de que o "http" que chegou já veio de um "https" lá fora, e o Rails para de montar URLs erradas.

Gerar o certificado

Do seu notebook, na raiz do projeto:

```
openssl req -x509 -newkey rsa:2048 -sha256 -days 365 -nodes \
  -keyout tls/capacita-key.pem -out tls/capacita-cert.pem \
  -subj "/CN=seunome.test" \
  -addext "subjectAltName=DNS:seunome.test"
```

O `subjectAltName` não é opcional. Cliente moderno nenhum olha só o `CN`: sem SAN, o certificado é inválido mesmo estando certo.

E ponha o conteúdo nas variáveis, no seu `.env`:

```
KAMAL_PROXY_SSL_CERTIFICATE="$(cat tls/capacita-cert.pem)"
KAMAL_PROXY_SSL_PRIVATE_KEY="$(cat tls/capacita-key.pem)"
export KAMAL_PROXY_SSL_CERTIFICATE KAMAL_PROXY_SSL_PRIVATE_KEY
```

`.test` e o `/etc/hosts`

`seunome.test` não existe em DNS nenhum, e nunca vai existir: `.test` é um domínio **reservado para testes** justamente para isso ([RFC 6761](#)). Ninguém consegue registrar, então ele nunca vai colidir com um site de verdade.

Quem resolve esse nome é o seu próprio `/etc/hosts`, que o sistema consulta **antes** de perguntar ao DNS:

```
echo "SEU_IP seunome.test" | sudo tee -a /etc/hosts
```

```
getent hosts seunome.test # tem que devolver o IP da VM
```

Windows: repita no arquivo `C:\Windows\System32\drivers\etc\hosts`, com o Bloco de Notas aberto como administrador. É esse que o navegador do Windows consulta: o `/etc/hosts` do WSL2 vale só dentro do WSL2.

Autoassinado, na prática

Depois do deploy (seção 8), três comandos que valem a aula inteira.

Antes de rodar o primeiro, decida: o seu servidor está servindo HTTPS de verdade, com um certificado que você mesmo gerou. O `curl` vai funcionar ou vai reclamar? E se reclamar, vai ser por causa da criptografia ou por outra coisa?

```
curl https://seunome.test/api/v1/status
```

```
curl: (60) SSL certificate problem: self-signed certificate
```

A frase exata muda com a versão do `curl`: umas dizem `SSL certificate problem`, outras `SSL certificate OpenSSL verify result`, e as mais antigas escrevem `self signed` sem hífen. O que é sempre igual, e é o que importa, é o número **60** e a expressão *self-signed*.

Isso é o TLS funcionando, não falhando. O túnel subiu; o que falhou foi a *confiança*. O cliente não conhece quem assinou.

```
curl -k https://seunome.test/api/v1/status
```

O `-k` é "criptografa, mas não confere quem é". Funciona, e é exatamente o que você **não** faz em produção: sem verificar o certificado, qualquer um no meio do caminho pode se passar pelo servidor.

```
curl --cacert tls/capacita-cert.pem https://seunome.test/api/v1/status
```

Aqui você não desligou nada: você **disse ao curl em quem confiar**. E é isso que uma CA é. Alguém em quem o seu sistema já decidiu confiar, de fábrica. Let's Encrypt não tem mágica nenhuma que o seu `openssl` não tenha; tem o navegador do mundo inteiro com a chave dela na lista.

Abra `https://seunome.test/api/v1/status` no navegador também. O aviso vermelho é o mesmo fato, dito para um humano.

7. GitHub Actions: o portão

`.github/workflows/ci.yml` roda em todo push e todo pull request:

- `scan_ruby`: Brakeman (segurança estática) e bundler-audit (CVE em gems)
- `lint`: RuboCop
- `test`: `bin/rails test` contra um Postgres descartável

Se qualquer um falhar, o código não entra na `main`.

Por que isso importa aqui, se o deploy é manual? Porque o CI não depende de onde o servidor mora. Ele é o portão: garante que o que você está prestes a colocar no ar compila, passa nos testes e não tem CVE conhecida. Um servidor sem CI publica bug automaticamente; um CI sem servidor ainda te avisa antes.

O deploy automático a cada push é o passo seguinte, e só faz sentido quando o servidor tem IP público: o runner do GitHub não alcança um IP dentro do seu notebook. Ele está pronto em `.github/workflows/deploy.yml`, e a seção **10** explica o que ele faz.

8. O primeiro deploy

Primeiro, valide sem tocar em nada:

```
cd ~/automic_auth_api
source .env
bundle exec kamal config
```

Sai um YAML grande, sem erro. Se reclamar de variável faltando, a mensagem diz exatamente qual.

Agora:

```
bundle exec kamal setup
```

`setup` instala o Docker se faltar, sobe os accessories e faz o primeiro deploy. **Só na primeira vez.** Depois é sempre `kamal deploy`.

Deu certo:

```
curl --cacert tls/capacita-cert.pem https://seunome.test/api/v1/status
```

```
{"status":"ok","service":"automic-auth-api","environment":"production"}
```

O seu código está rodando dentro de um container, num Ubuntu que você provisionou, atrás de um proxy TLS, com um Postgres com volume. Em produção, é isto.

Daqui para frente, todo deploy é:

```
source .env && bundle exec kamal deploy
```

9. Operar

```
source .env

kamal app logs -f           # logs ao vivo
kamal app exec --interactive --reuse "bin/rails console"
kamal app exec "bin/rails db:migrate"
kamal app details           # o que está rodando
kamal accessory reboot db   # depois de mudar env do banco
kamal rollback              # volta para a versão anterior
```

Na máquina que serve, que hoje é a sua:

```
docker ps          # os containers do Kamal, rodando
docker logs -f automic-auth-api-db
free -h
df -h
```

Backup

Volume Docker não é backup. Um `docker volume rm` sem querer, ou a máquina que morre, leva o volume junto. Backup é uma cópia que vive **fora** dali.

```
docker exec automic-auth-api-db \
  pg_dump -U automic_auth_api automic_auth_api_production | gzip > backup-$(date +%F).sql.gz
```

E, o que quase ninguém faz: **restaure num banco de teste**. Backup que nunca foi restaurado não é backup, é esperança.

Parar e voltar

```
kamal app stop          # para a aplicação
kamal accessory stop db  # para o banco
kamal app boot           # traz de volta
```

Os containers ficam parados, não apagados. O volume do banco continua lá.

10. E num servidor de verdade

Tudo o que você fez hoje vale numa máquina alugada sem mudar de forma. O que **entra** são quatro coisas, e todas existem porque agora a máquina está exposta ao mundo:

	Hoje	Num servidor alugado
A máquina	já existe: é a sua	portal do provedor: região, tamanho, imagem, cota
Antes de tudo	nada	provisionar: usuário sem root, <code>apt upgrade</code> , Docker, swap
Firewall	nenhum, a máquina não está exposta	<code>ufw</code> dentro, mais o do provedor fora, com a 22 aberta só para o seu IP
SSH	você autorizou a si mesmo	endurecer o <code>sshd</code> : sem senha, sem root
Nome	<code>/etc/hosts</code>	DNS de verdade, num domínio seu
Certificado	autoassinado	emitido por uma CA, que exige provar que o domínio é seu
Deploy	<code>kamal deploy</code> no seu terminal	GitHub Actions, por OIDC, sem senha guardada

E no `.env`, isso é:

```
export SERVER_IP=57.156.65.151    # em vez de 127.0.0.1
export KAMAL_SSH_USER=ubuntu      # em vez do seu login
```

Duas linhas. O resto do arquivo, o `deploy.yml`, o `Dockerfile` e os comandos são os mesmos.

O deploy automático

O `.github/workflows/deploy.yml` deste repositório é o pipeline pronto. A sequência dele:

1. autentica na nuvem (OIDC)
2. descobre o próprio IP público
3. cria uma regra no firewall liberando a 22 só para esse IP/32
4. faz o deploy
5. APAGA a regra, mesmo se o deploy falhar

O passo 5 tem `if: always()`. **Deixar a 22 aberta é o erro que transforma um deploy ruim num incidente de segurança.**

E o **OIDC** é o motivo de não haver senha nenhuma nisso: em vez de guardar um segredo de longa duração no GitHub, o GitHub emite a cada execução um token curto que diz *"sou o workflow do repositório X, na branch `main`"*, e a nuvem devolve um acesso temporário.

Isto é a demonstração de hoje, não o exercício. O passo a passo completo, com uma VM na Azure, o firewall, o DNS no Cloudflare, o certificado e a credencial federada do OIDC, está em [apendice-azure.md](#), para você fazer em casa com a [Azure for Students](#): US\$100 de crédito, sem cartão.

As práticas da aula

Sete práticas, cada uma logo depois do bloco que a explica. Nesta aula um passo mal feito só aparece três passos depois, então **não siga com uma conferência vermelha**.

#	O que	Depois de
1	A sua máquina vira o servidor	seção 2
2	Os arquivos de deploy e o token do registry	seção 3
3	O <code>.env</code>	seção 5
4	O certificado e o <code>/etc/hosts</code>	seção 6
5	O primeiro deploy	seção 8
6	O certificado, com os olhos	seção 8
7	O sistema inteiro, e o backup	seção 9

Prática 1: a sua máquina vira o servidor

Se você usa Windows, tudo isto é **dentro do WSL2**. É ele o seu Linux.

```
# a) o servidor de SSH
sudo apt update && sudo apt install -y openssh-server      # Ubuntu, Debian, WSL2
sudo service ssh start
# macOS: Ajustes do Sistema → Geral → Compartilhamento → Sessão remota

# b) o par de chaves da capacitação
ssh-keygen -t ed25519 -f ~/.ssh/capacita -C "capacita-servidor" -N ""
chmod 600 ~/.ssh/capacita

# c) autorizar a si mesmo
mkdir -p ~/.ssh && chmod 700 ~/.ssh
cat ~/.ssh/capacita.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

Confere:

```
ssh -i ~/.ssh/capacita -o IdentitiesOnly=yes "$USER"@127.0.0.1 'echo SSH_OK && docker -v'
```

Tem que sair `SSH_OK` e a versão do Docker. Se saiu, o Kamal tem tudo de que precisa.

Se der errado

Erro	Causa	Saída
<code>Connection refused</code> na 22	o <code>sshd</code> não está rodando	<code>sudo service ssh start</code> ; no macOS, ligue a Sessão remota
<code>Connection refused</code> depois de reiniciar o Windows	o WSL2 não sobe o <code>sshd</code> sozinho	<code>sudo service ssh start</code> de novo. É o tropeço mais comum do dia
<code>Permission denied (publickey)</code>	permissão frouxa na chave	<code>chmod 600 ~/.ssh/capacita</code> e <code>chmod 700 ~/.ssh</code>
<code>Permission denied</code> mesmo com <code>chmod</code>	a pública não entrou no <code>authorized_keys</code>	<code>cat ~/.ssh/authorized_keys</code> e confira que a linha está lá
<code>Too many authentication failures</code>	o SSH ofereceu todas as chaves do agente	acrescente <code>-o IdentitiesOnly=yes</code>
<code>Host key verification failed</code>	a chave do host mudou	<code>ssh-keygen -R 127.0.0.1</code> e conecte de novo
<code>docker: command not found</code> pela SSH	o <code>PATH</code> da sessão não interativa é menor	confirme com <code>ssh ... 'which -a docker'</code> ; se não achar, reinstale o Docker pelo pacote do sistema
<code>sudo</code> pede senha e trava o comando	esperado na primeira vez	digite a senha; é só a instalação

Prática 2: os arquivos de deploy e o token do registry

```
cd ~/capacitacao-gabarito && git checkout aula-04
rsync -aR config/deploy.yml config/environments/production.rb \
    .kamal scripts .github .env.example Dockerfile .dockerignore Gemfile Gemfile.lock \
    ~/automic_auth_api/

cd ~/automic_auth_api && bundle install
```

O `-R` não é detalhe: sem ele o `rsync` joga `deploy.yml` e `production.rb` na raiz do projeto, fora de `config/`, e o Kamal não acha nada.

O seu projeto já tinha um `config/deploy.yml`, um `Dockerfile` e um `.kamal/`, gerados pelo `rails new`. Você está **substituindo** o `deploy.yml` genérico pelo nosso.

Crie o **PAT (classic)** no GitHub: **Settings** → **Developer settings** → **Personal access tokens** → **Tokens (classic)** → **Generate new token**, com `write:packages` e `read:packages`. Ele é mostrado **uma vez**.

Confere:

```
ls config/deploy.yml .kamal/secrets .env.example
uname -m           # x86_64 é amd64; aarch64 ou arm64 é arm64. Anote para o .env
```

Se der errado

Erro	Causa	Saída
<code>deploy.yml</code> foi parar na raiz	faltou o <code>-R</code>	apague e refaça o <code>rsync</code> com <code>-R</code>
<code>.kamal/</code> não veio	pasta oculta	o comando lista <code>.kamal</code> explicitamente; copie-o inteiro
<code>Could not find gem 'kamal'</code>	faltou <code>bundle install</code>	rode
<code>git checkout aula-04</code> reclama de alterações locais	you editou o gabarito	<code>git -C ~/capacitacao-gabarito checkout .</code> ; o gabarito é só leitura
o PAT sumiu da tela	é mostrado uma vez só	gere outro; não há como recuperar
criou um token <i>fine-grained</i>	o ghcr.io não aceita	tem que ser classic

Prática 3: o `.env`

Errar aqui é o que causa quase toda falha da Prática 5.

```
cd ~/automic_auth_api
cp .env.example .env
$EDITOR .env
source .env
```

O `.env.example` já vem com `SERVER_IP=127.0.0.1` e `KAMAL_SSH_USER="$USER"` preenchidos. O que falta você preencher: `GHCR_USER`, `SERVER_ARCH`, `APP_HOST`, `KAMAL_REGISTRY_PASSWORD` (o PAT), `RAILS_MASTER_KEY` (o conteúdo de `config/master.key`), `AUTOMIC_AUTH_API_DATABASE_PASSWORD`, `JWT_SECRET` (saída de `bin/rails secret`), `CORS_ORIGINS`, `MAILER_FROM` e os três `SMTP_*`.

Confere:

```
git status # o .env NÃO pode aparecer
bundle exec kamal config > /dev/null && echo "config ok"
```

Se o `.env` aparecer no `git status`, **pare tudo** e conserte o `.gitignore` antes de qualquer commit. Segredo commitado não sai do histórico apagando o arquivo.

Se der errado

Erro	Causa	Saída
<code>key not found: "GHCR_USER"</code>	esqueceu o <code>source .env</code>	<code>source .env</code> , e ele vale só naquele shell
<code>cat: config/master.key: No such file</code>	o Rails ainda não gerou	<code>bin/rails credentials:edit</code> cria; feche o editor para salvar
<code>kamal config</code> reclama de outra variável	ela está vazia no <code>.env</code>	a mensagem diz o nome exato
o <code>.env</code> aparece no <code>git status</code>	falta a linha <code>.env</code> no <code>.gitignore</code>	acrescente antes de commitar
<code>\$EDITOR: command not found</code>	variável não definida	<code>nano .env</code> ou <code>code .env</code>

Prática 4: o certificado e o `/etc/hosts`

```
mkdir -p tls
openssl req -x509 -newkey rsa:2048 -sha256 -days 365 -nodes \
  -keyout tls/capacita-key.pem -out tls/capacita-cert.pem \
  -subj "/CN=seunome.test" -addext "subjectAltName=DNS:seunome.test"

echo "127.0.0.1 seunome.test" | sudo tee -a /etc/hosts
getent hosts seunome.test
```

Confere: o `getent` devolve `127.0.0.1`, e o certificado tem o SAN certo:

```
openssl x509 -in tls/capacita-cert.pem -noout -text | grep -A1 "Subject Alternative Name"
```

Windows: para abrir no navegador do Windows, repita a linha em `C:\Windows\System32\drivers\etc\hosts`, com o Bloco de Notas aberto como administrador. O WSL2 espelha o `localhost` para o Windows, então `127.0.0.1` funciona dos dois lados.

Se der errado

Erro	Causa	Saída
<code>unknown option -addext</code>	OpenSSL 1.0, antigo	atualize, ou use um arquivo de config com <code>[v3_req]</code>
<code>getent</code> não devolve nada	a linha do <code>/etc/hosts</code> saiu torta	tem que ser <code>IP<espaço>nome</code> , sem vírgula
o navegador do Windows não acha o nome	falta a linha no hosts do Windows	veja a nota acima
sem "Subject Alternative Name" na saída	faltou o <code>-addext</code>	gere de novo: sem SAN, cliente moderno nenhum aceita
a chave foi para o Git	<code>.pem</code> de chave é segredo	acrescente <code>tls/*-key.pem</code> ao <code>.gitignore</code>

Prática 5: o primeiro deploy

```
source .env
bundle exec kamal config      # valida sem tocar em nada
bundle exec kamal setup      # só na primeira vez
```

Confere:

```
curl --cacert tls/capacita-cert.pem https://seunome.test/api/v1/status
```

```
{"status":"ok","service":"automic-auth-api","environment":"production"}
```

Pare um segundo aqui. O seu código está rodando dentro de um container, com o Rails em modo produção, atrás de um proxy que termina TLS, falando com um Postgres que tem volume. Você não está mais rodando `bin/rails server`.

Se der errado

Esta é a maior tabela da capacitação, de propósito: o primeiro deploy cobra de uma vez tudo o que você configurou hoje. Se falhar, **quase nunca é o Kamal**. É uma variável do `.env`, o tipo do token, ou a arquitetura. Leia a mensagem, ache a linha aqui, corrija, e rode de novo: `kamal setup` duas vezes não estraga nada.

Erro	Causa	Saída
<code>denied</code> / <code>unauthorized</code> ao empurrar a imagem	PAT sem <code>write:packages</code> , ou <i>fine-grained</i>	gere um classic com os dois escopos
erro de cota ou de armazenamento no <code>push</code>	a conta grátis limita pacote privado	deixe o pacote público (seção 3), e apague versões antigas em Packages
<code>GHCR_USER</code> recusado pelo registry	maiúscula no nome	o ghcr.io só aceita minúsculas
<code>exec format error</code> no container	arquitetura errada	<code>uname -m</code> e ajuste <code>SERVER_ARCH</code>
<code>Docker is not installed</code>	você rodou <code>deploy</code> em vez de <code>setup</code>	o primeiro é sempre <code>setup</code>
<code>Connection refused</code> na 22	o <code>sshd</code> parou	<code>sudo service ssh start</code>
<code>Host key verification failed</code>	primeira conexão do Kamal	<code>ssh -i ~/.ssh/capacita "\$USER"@127.0.0.1</code> uma vez e aceite
<code>address already in use</code> na 80 ou 443	outra coisa ocupa a porta	<code>sudo ss -tlnp grep -E ':(80 443)'</code> e libere
a aplicação sobe mas não acha o banco	<code>kamal deploy</code> não sobe accessories	<code>kamal accessory boot db</code>
<code>404</code> do proxy	<code>APP_HOST</code> diferente do nome que você chamou	os dois têm que bater exatamente
health check falhando em loop	a aplicação estoura ao subir	<code>kamal app logs -f</code> : quase sempre <code>RAILS_MASTER_KEY</code> errada ou migration pendente
o build demora e a máquina engasga	build de imagem consome bastante	feche o resto; a primeira vez é sempre a mais lenta

Prática 6: o certificado, com os olhos

Três comandos, e é a prática que mais ensina do dia.

Antes de rodar o primeiro, decida: o seu servidor está servindo HTTPS de verdade, com um certificado que você mesmo gerou. O `curl` vai funcionar ou vai reclamar? E se reclamar, vai ser por causa da criptografia ou por outra coisa?

```
curl https://seunome.test/api/v1/status
```

Tem que **falhar**, com o erro **60** e a expressão *self-signed*. **Isso é o TLS funcionando, não falhando:** o túnel subiu, e o que faltou foi a confiança.

```
curl -k https://seunome.test/api/v1/status
```

Funciona. O `-k` é "criptografa, mas não confere quem é".

```
curl --cacert tls/capacita-cert.pem https://seunome.test/api/v1/status
```

Funciona **conferindo**. Você não desligou nada: você disse ao `curl` em quem confiar.

Abra também no navegador e leia o aviso.

Confere: escreva numa linha, para você mesmo, a diferença entre o segundo e o terceiro comando. Se conseguir, você entendeu o que uma CA é.

Se der errado

Erro	Causa	Saída
o primeiro comando funcionou	você já tinha confiado nesse certificado na máquina	é raro; confira com outro nome em <code>.test</code>
<code>unable to get local issuer certificate</code>	mensagem diferente, mesmo fato	é a mesma lição: falta confiança
o navegador não deixa passar de jeito nenhum	HSTS de um teste anterior	use aba anônima, ou outro nome em <code>.test</code>

Prática 7: o sistema inteiro, e o backup

- [] Refaça o fluxo da Aula 3 **contra a sua API em produção** (`https://seunome.test`), e desta vez o e-mail chega de verdade na sua caixa de entrada, não no navegador.
- [] Mude a mensagem da rota de status, commite, dê push (o CI roda) e:

```
source .env && bundle exec kamal deploy  
kamal app logs -f
```

- [] Faça um backup, e confira que não está vazio:

```
docker exec automic-auth-api-db \  
  pg_dump -U automic_auth_api automic_auth_api_production | gzip > backup-$(date +%F).sql.gz  
ls -lh backup-*.sql.gz
```

- [] Quando quiser liberar a máquina: `kamal app stop` para o app, e `kamal accessory stop db` para o banco. `kamal app boot` traz de volta.

Confere: o backup tem mais que alguns bytes, e o `kamal app logs` mostrou o container novo subindo ao lado do antigo antes de trocar.

Se der errado

Erro	Causa	Saída
o e-mail não chega	SMTP não configurado, ou porta bloqueada	confira os <code>SMTP_*</code> do <code>.env</code> ; algumas redes bloqueiam a 587
o backup saiu com 20 bytes	o <code>pg_dump</code> falhou e o <code>gzip</code> comprimiu o vazio	tire o <code>\ gzip</code> e leia o erro
<code>docker exec</code> não acha o container	nome diferente	<code>docker ps</code> e use o nome que aparece
o segundo deploy não mudou nada	você não commitou antes	o Kamal usa o commit atual como versão da imagem
<code>kamal deploy</code> diz que há um lock	um deploy anterior morreu no meio	<code>kamal lock release</code>

Bônus, se sobrou tempo

- Derrube o app de propósito (`kamal app stop`) e veja o que o `curl` responde. Depois `kamal app boot` .
- Faça um deploy que **falha** no health check (quebre a rota `/up`) e confirme que o Kamal **não** troca o tráfego: a versão antiga continua servindo.
- Rode `kamal rollback` e veja voltar para a imagem anterior.
- Abra o `apendice-azure.md` e provisione uma máquina de verdade. O `deploy.yml` é o mesmo: muda o `SERVER_IP` e o `KAMAL_SSH_USER` .

Recapitulando

- Publicar não é "rodar num lugar diferente": é empacotar, versionar, entregar e conseguir voltar atrás. Que a máquina seja a sua ou a de um datacenter muda duas linhas do `.env` .
- Escolha a ferramenta do tamanho do problema. Kubernetes não era o tamanho.
- Imagem é receita, container é bolo. Multi-stage e usuário não-root.
- `clear` é configuração, `secret` é segredo, e a diferença aparece no `docker inspect` .
- Volume é o que faz o dado sobreviver ao deploy. E ainda assim não é backup.
- TLS criptografa; **CA é confiança**. São coisas separadas, e o `curl -k` mostra a costura.
- CI é o portão. Ele te avisa antes, com ou sem deploy automático.
- Numa máquina exposta, provisionar direito, o firewall e o segredo de curta duração deixam de ser detalhe. É o assunto do apêndice.

O que o `seem-backend` tem a mais

O que ficou de fora daqui, e por quê:

Recurso	Para quê
Datadog (logs)	logs centralizados e Error Tracking, com o app logando JSON estruturado
rack-attack	bloqueia scanner e rajada de erro por IP: a internet bate na sua porta o dia todo
Active Storage	foto de perfil, em volume Docker persistente
Solid Queue dedicado	processamento em background
Audit log	histórico de toda mutação feita por admin
Export .xlsx	relatório de presença
OpenAPI/Swagger	contrato da API documentado

Nenhum deles é difícil depois do que você viu aqui. Todos estão no `seem-backend`. Agora dá para ler.

Apêndice: o mesmo servidor, na nuvem

Para quem: você terminou a Aula 4, tem a API rodando na VM do seu notebook, e quer colocá-la num IP público, com domínio, certificado de CA pública e deploy automático a cada push.

Quanto tempo: ~2h na primeira vez, quase tudo clicando em portal.

Quanto custa: nada, com a [Azure for Students](#), US\$100 de crédito, sem cartão de crédito, mediante e-mail institucional. A verificação acadêmica **pode demorar dias**; comece por ela.

Você também vai precisar de um domínio, e é a única coisa deste apêndice que custa dinheiro: uns R\$15 por ano. Se você é estudante, o [GitHub Student Pack](#) dá um de graça por um ano, e a aprovação também leva alguns dias. Vale pedir os dois na mesma tarde.

Nada aqui substitui a Aula 4. O Docker, o Kamal e o `deploy.yml` são **os mesmos**. O que este apêndice acrescenta é o que só existe quando a máquina não é a sua: provisionar um Ubuntu do zero, firewall de provedor, DNS, certificado de CA, e um jeito de o GitHub entrar na máquina sem guardar senha.

O que muda

	Na Aula 4	Aqui
A máquina	a sua, já configurada	um Ubuntu cru, no portal da Azure
Provisionar	nada: só instalar o <code>sshd</code>	<code>apt upgrade</code> , <code>ufw</code> , Docker, swap, endurecer o <code>sshd</code>
IP	<code>127.0.0.1</code>	público, e o mundo inteiro alcança
Firewall	nenhum	<code>ufw</code> mais o NSG da Azure
Nome	<code>/etc/hosts</code>	DNS de verdade, no Cloudflare
Certificado	autoassinado	Cloudflare Origin CA, com o público do Cloudflare na frente
Deploy	<code>kamal deploy</code> no seu terminal	GitHub Actions, por OIDC
Usuário do SSH	o seu login	<code>azureuser</code>

Os dois últimos são duas linhas do `.env`:

```
export SERVER_IP=57.156.65.151      # em vez de 127.0.0.1
export KAMAL_SSH_USER=azureuser    # em vez do seu login
```

1. Criar a VM na Azure

Cotas antes de tudo

A assinatura Azure for Students não oferece todos os tamanhos em todas as regiões. O portal responde:

NotAvailableForSubscription

O caminho certo:

1. **Assinaturas** → **Azure for Students** → **Uso + cotas**
2. filtre por `Compute`
3. veja as cotas regionais
4. escolha uma região onde o tamanho aparece

Não insista num tamanho marcado como indisponível. Pedir aumento de cota não resolve quando a oferta não está habilitada para a assinatura.

O assistente

Máquinas virtuais → **Criar** → **Máquina virtual do Azure**

Campo	Valor
Grupo de recursos	Novo: <code>rg-capacita-<seunome></code>
Nome	<code>vm-capacita-<seunome></code>
Região	uma com cota disponível
Imagem	Ubuntu Server 24.04 LTS: x64 Gen2
Tamanho	<code>Standard_B1s</code> (1 vCPU, 1 GiB) ou maior, se houver crédito
Autenticação	Chave pública SSH
Usuário	<code>azureuser</code>
Tipo de chave	Ed25519
Portas públicas	Nenhuma

"Nenhuma" é a escolha mais importante da tela. O assistente, se deixado, cria uma regra liberando SSH para a internet inteira. Bots varrem a porta 22 do IPv4 todo, o dia todo. Vamos abrir a 22 só para o seu IP.

Rede:

- VNet e sub-rede: aceite os padrões

- IP público: Standard
- NSG: avançado (é o firewall. Vamos mexer nele)

Marque tudo com tags (`ambiente=capacitacao`, `dono=<seunome>`): é assim que você acha recurso órfão consumindo crédito depois.

A chave privada

O portal oferece o download **uma vez**.

```
mkdir -p ~/.ssh
mv ~/Downloads/vm-capacita-seunome_key.pem ~/.ssh/azure-capacita
chmod 600 ~/.ssh/azure-capacita
```

`chmod 600` = só você lê. O SSH **recusa** usar uma chave com permissão frouxa.

Abrir o SSH só para você

No NSG da VM, **Regras de segurança de entrada** → **Adicionar**:

Campo	Valor
Origem	Endereços IP
IPs de origem	<code>SEU_IP/32</code>
Portas de destino	<code>22</code>
Protocolo	TCP
Prioridade	<code>100</code>
Nome	<code>allow-ssh-meu-ip</code>

Descubra o seu IP:

```
curl -4 ifconfig.me
```

O `/32` significa "exatamente este endereço". Precisa também de:

Porta	Para quê
<code>80</code>	HTTP (o Cloudflare precisa alcançar)
<code>443</code>	HTTPS

Internet de casa troca de IP. Quando o SSH parar de conectar do nada, é isso: atualize a regra.

Este é o firewall que a Aula 4 não teve. O `ufw` continua valendo dentro da máquina, e é uma segunda camada, mas o NSG é o que faz o pacote nem chegar.

Primeiro acesso

```
ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP_PUBLICO
```

O prompt vira `azureuser@vm-capacita`. **Você está dentro de outro computador**, e agora vem a parte que a Aula 4 não teve: deixar essa máquina pronta para servir.

2. Provisionar o Ubuntu

Na Aula 4 o servidor era a sua máquina, que já estava configurada. Aqui você recebe um Ubuntu cru, e tudo o que ele precisa você instala. **É este bloco que a nuvem acrescenta.**

Atualizar

```
sudo apt update && sudo apt upgrade -y
test -f /var/run/reboot-required && sudo reboot
```

Numa máquina exposta isso não é higiene: é a correção das falhas que já foram descobertas desde a imagem ter sido publicada.

Firewall

Uma máquina só deve aceitar conexão no que ela realmente serve. O Ubuntu traz o **ufw** (*Uncomplicated Firewall*), uma casca amigável sobre as regras do kernel.

```
sudo ufw default deny incoming      # nada entra...
sudo ufw default allow outgoing     # ...mas a máquina pode sair
sudo ufw allow 22/tcp               # SSH
sudo ufw allow 80/tcp               # HTTP
sudo ufw allow 443/tcp              # HTTPS
sudo ufw enable
sudo ufw status verbose
```

Libere a 22 antes do `enable`. Habilitar o firewall com a 22 fechada, numa máquina remota, é o jeito mais rápido de perder o acesso a ela.

Aqui há **dois** firewalls: o `ufw` dentro da máquina e o NSG do provedor, fora dela. O de fora é o que importa mais, porque com ele o pacote nem chega. O `ufw` é a segunda camada, para o caso de a primeira estar mal configurada.

Docker, do repositório oficial

O `apt install docker.io` do Ubuntu instala uma versão velha. Use o repositório da Docker:


```
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

sudo usermod -aG docker azureuser
exit
```

Saia e entre de novo: grupo só vale em sessão nova.

```
ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP 'docker run --rm hello-world'
```

Root e permissões

`root` é o usuário que pode tudo, sem "tem certeza?". Você trabalha como `azureuser` e chama `sudo` quando precisa. Numa máquina exposta essa separação deixa de ser estilo: se alguém escapar da aplicação, cai num usuário sem privilégio.

É a mesma razão de o Dockerfile criar um usuário `rails` e não rodar como root.

Swap

```
ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP 'sudo bash -s' < scripts/server-swap.sh
```

Uma `B1s` tem 1 GiB de RAM. Rails e Postgres juntos estouram isso, e o kernel mata o processo que estiver na frente (o *OOM killer*), com uma mensagem que não explica nada. 2 GiB de swap resolvem.

Na Aula 4 isso não fazia falta, porque o seu notebook tem RAM sobrando. Aqui faz.

Endurecer o SSH

Mantenha a sessão atual aberta enquanto testa em outro terminal.

```
sudo tee /etc/ssh/sshd_config.d/99-hardening.conf >/dev/null <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
EOF

sudo sshd -t          # valida a sintaxe ANTES de recarregar
sudo systemctl reload ssh
```

Em outro terminal:

```
ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP 'echo OK'
```

Só feche a primeira sessão depois que isso responder. Errar a config do SSH com uma sessão só aberta é o jeito clássico de perder acesso à máquina. E aqui, diferente da Aula 4, **você não está sentado na frente dela**: só sobra o Serial Console do portal da Azure, e é bom não depender dele.

E então, o deploy

Daqui em diante é a Aula 4 sem mudar nada. No seu `.env`, duas linhas:

```
export SERVER_IP=57.156.65.151    # o IP público da VM
export KAMAL_SSH_USER=azureuser
```

E `kamal setup`.

3. Cloudflare e TLS

Por que uma CA pública

Na Aula 4 o certificado era autoassinado, e o `curl` reclamava até você passar `--cacert`. Aqui você tem um domínio de verdade: dá para **provar** que ele é seu, e é isso que uma CA pública exige em troca de um certificado em que o mundo já confia.

DNS

No Cloudflare, no seu domínio:

Campo	Valor
Tipo	A
Nome	seunome.capacita
Conteúdo	o IP público da sua VM
Proxy	Proxied (nuvem laranja)

Proxied significa que o tráfego passa pelo Cloudflare antes de chegar na sua VM. Você ganha cache, proteção contra DDoS e, o principal aqui, o IP real da sua máquina não aparece no DNS.

Agora há **dois** proxies reversos no caminho:

```
navegador → Cloudflare → kamal-proxy (na sua VM) → Rails
                (proxy 1)         (proxy 2)
```

Certificado Origin CA

SSL/TLS → **Origin Server** → Create Certificate. O Cloudflare gera o par e mostra **uma vez**.

Ele protege o trecho **Cloudflare** → **sua VM**. O certificado que o *usuário* vê é o público do Cloudflare: o Origin CA nunca aparece para o navegador, e por isso não precisa ser confiável por ele.

É o mesmo lugar do `.env` que o autoassinado ocupava:

```
KAMAL_PROXY_SSL_CERTIFICATE="$(cat origin-ca.pem)"
KAMAL_PROXY_SSL_PRIVATE_KEY="$(cat origin-ca-key.pem)"
```

Full (strict)

SSL/TLS → Overview → **Full (strict)**.

Modo	Cloudflare → sua VM
Flexible	em texto puro: não use
Full	criptografado, mas aceita certificado inválido
Full (strict)	criptografado e o certificado é validado

Com `Flexible`, o cadeado aparece para o usuário e a senha dele trafega em claro no último trecho. É pior que não ter HTTPS, porque mente.

Por que não Let's Encrypt

O Kamal sabe emitir Let's Encrypt sozinho. Atrás do Cloudflare proxied não dá:

- o desafio HTTP-01 não chega ao seu servidor como o Let's Encrypt espera;

- daria para tirar o proxy durante a emissão, mas isso expõe o IP e vira ritual manual a cada renovação.

Sem Cloudflare na frente, com o DNS apontando direto para a VM, o Let's Encrypt é o caminho mais simples e o Kamal cuida sozinho:

```
proxy:
  ssl: true
  host: <%= ENV.fetch("APP_HOST") %>
```

4. OIDC: deploy sem senha

O jeito comum seria criar um *client secret* na Azure e guardar como secret do GitHub. Problemas: ele é longo, vale por meses, e quem tiver acesso ao repositório tem acesso à sua Azure.

OIDC (OpenID Connect) inverte: o GitHub emite, a cada execução, um token de curta duração que diz "sou o workflow do repositório X, na branch Y". A Azure verifica e devolve um acesso temporário.

Não existe segredo de longa duração em lugar nenhum.

Criando

1. Registro de aplicativo. Em Microsoft Entra ID → Registros de aplicativo → Novo registro:

- nome: `github-capacita-<seunome>`
- contas: somente este diretório
- URI de redirecionamento: vazia
- **não crie segredo do cliente**

Anote: Application (client) ID, Directory (tenant) ID, Subscription ID.

2. Credencial federada: no app criado, Certificados e segredos → Credenciais federadas → Adicionar → cenário **GitHub Actions**:

Campo	Valor
Organização	seu usuário do GitHub
Repositório	nome do repositório
Tipo de entidade	Branch
Branch	<code>main</code>

O subject resultante:

```
repo:SEU-USUARIO/SEU-REP0:ref:refs/heads/main
```

Precisa bater **caractere por caractere** com o que o GitHub emite. Erro de maiúscula, de nome ou de branch dá `AADSTS700213`, e a mensagem não ajuda.

3. Permissão mínima: no **NSG** (não na assinatura), IAM → Adicionar atribuição de função → **Colaborador de Rede** → o app criado.

Escopo no NSG e não na assinatura: se a credencial vazar, o estrago se limita a regras de firewall daquela máquina.

4. Chave SSH de deploy. Separada da sua chave pessoal:

```
ssh-keygen -t ed25519 -f ~/.ssh/capacita-github -C "github-actions" -N ""  
  
ssh -i ~/.ssh/azure-capacita azureuser@SEU_IP \  
'umask 077; mkdir -p ~/.ssh; cat >> ~/.ssh/authorized_keys' < ~/.ssh/capacita-github.pub
```

Chave separada dá para revogar o acesso do CI sem trocar o seu.

5. O workflow de deploy

`.github/workflows/deploy.yml` roda a cada push na `main`. O ponto delicado é o SSH: a porta 22 está fechada para a internet, e o runner do GitHub tem IP diferente a cada execução: não dá para liberar antes.

1. autentica na Azure (OIDC)
2. descobre o próprio IP público
3. cria uma regra no NSG liberando a 22 só para esse IP/32
4. faz o deploy
5. APAGA a regra – mesmo se o deploy falhar

O passo 5 tem `if: always()`. **Deixar a 22 aberta é o erro que transforma um deploy ruim num incidente de segurança.**

E aqui o build volta a acontecer no runner, e não no seu notebook. É o que você quer: o deploy deixa de depender da sua máquina estar ligada.

Cadastrando no GitHub

Settings → **Secrets and variables** → **Actions**

Secrets:

Nome	Conteúdo
<code>AZURE_CLIENT_ID</code>	Application (client) ID
<code>AZURE_TENANT_ID</code>	Directory (tenant) ID
<code>AZURE_SUBSCRIPTION_ID</code>	Subscription ID
<code>AZURE_SSH_PRIVATE_KEY</code>	conteúdo de <code>~/.ssh/capacita-github</code> (a privada)
<code>RAILS_MASTER_KEY</code>	conteúdo de <code>config/master.key</code>
<code>AUTOMIC_AUTH_API_DATABASE_PASSWORD</code>	invente uma senha longa
<code>JWT_SECRET</code>	saída de <code>bin/rails secret</code>
<code>SMTP_ADDRESS</code> / <code>SMTP_USERNAME</code> / <code>SMTP_PASSWORD</code>	do provedor de e-mail
<code>KAMAL_PROXY_SSL_CERTIFICATE</code>	certificado Origin CA
<code>KAMAL_PROXY_SSL_PRIVATE_KEY</code>	chave do Origin CA

Repare que **não existe** `KAMAL_REGISTRY_PASSWORD`: dentro do Actions, o `GITHUB_TOKEN` que o próprio GitHub gera já autentica no ghcr.io. O PAT que você criou na Aula 4 só era necessário porque você estava fora dele.

Variables (não são segredo):

Nome	Exemplo
<code>GHCR_USER</code>	seu usuário do GitHub
<code>SERVER_IP</code>	<code>57.156.65.151</code>
<code>KAMAL_SSH_USER</code>	<code>azureuser</code>
<code>SERVER_ARCH</code>	<code>amd64</code>
<code>APP_HOST</code>	<code>seunome.capacita.exemplo.tech</code>
<code>AZURE_RESOURCE_GROUP</code>	<code>rg-capacita-seunome</code>
<code>AZURE_NSG_NAME</code>	nome do NSG
<code>CORS_ORIGINS</code>	<code>http://localhost:5173</code>
<code>MAILER_FROM</code>	Capacitação <noreply@exemplo.tech>

Pelo CLI (não deixa o valor no histórico do shell):

```
gh secret set JWT_SECRET
gh secret set KAMAL_PROXY_SSL_CERTIFICATE < origin-ca.pem
gh variable set APP_HOST
```

O primeiro deploy

Actions → Deploy (Kamal) → Run workflow → command: `setup`

`setup` instala o Docker se faltar, sobe os accessories e faz o primeiro deploy. Só na primeira vez, depois é `deploy`, e ele acontece sozinho a cada push.

```
curl https://seunome.capacita.exemplo.tech/api/v1/status
```

```
{"status":"ok","service":"atomic-auth-api","environment":"production"}
```

E desta vez sem `--cacert`: o certificado que o seu `curl` recebeu foi assinado por uma CA em que ele já confiava.

6. Antes de ir embora

- [] **Desligar a VM** no portal (`Parar`). Parada, ela não consome crédito de computação. Uma `B1s` ligada 24h custa ~US\$8/mês: os US\$100 dão quase um ano, se você desligar.
- [] Conferir que a regra temporária de SSH sumiu do NSG:

```
az network nsg rule list --resource-group SEU_RG --nsg-name SEU_NSG --query "[].name" -o tsv
```

Não pode ter `allow-github-actions-deploy-ssh` na lista.

Recapitulando

- O trabalho é o mesmo. O que a nuvem acrescenta é **exposição**, e cada peça nova aqui existe por causa dela.
- O firewall do provedor vem antes do `ufw`: o pacote nem chega.
- CA pública exige prova de domínio. É a única diferença real entre ela e o seu `openssl`.
- Full (strict), sempre. `Flexible` mente para o usuário.
- OIDC troca segredo de longa duração por token de curta. Prefira sempre.
- A porta 22 abre por um minuto e fecha: inclusive quando dá errado.

Os erros que realmente acontecem neste percurso estão em `troubleshooting.md`.

Ruby para quem sabe Python

Você já sabe programar. Isto aqui é tradução, não curso de lógica.

Sintaxe

Python	Ruby
Indentação delimita bloco	<code>def ... end</code>
<code>None</code> / <code>True</code> / <code>False</code>	<code>nil</code> / <code>true</code> / <code>false</code>
<code>snake_case</code> para funções	<code>snake_case</code> (igual)
<code>PascalCase</code> para classes	<code>PascalCase</code> (igual)
<code>CONSTANTE</code> por convenção	Constante é qualquer nome com inicial maiúscula
<code>f"Olá {nome}"</code>	<code>"Olá #{nome}"</code>
<code># comentário</code>	<code># comentário</code>
<code>return</code> explícito	A última expressão é o retorno
<code>and</code> / <code>or</code> / <code>not</code>	<code>&&</code> / <code>& \ </code> / <code>!</code>
<code>elif</code>	<code>elsif</code>
<code>len(x)</code>	<code>x.length</code>
<code>str(x)</code>	<code>x.to_s</code>
<code>int("3")</code>	<code>"3".to_i</code>
<code>print(x)</code>	<code>puts x</code>

Só em Ruby

```
faca_algo if condicao          # modificador: if no fim da linha
faca_algo unless condicao      # unless = if not
5.times { |i| puts i }        # número é objeto
nome = usuario&.nome          # safe navigation (o ?. de outras linguagens)
valor ||= "padrao"            # atribui só se estiver nil/false
```

Aspas: a diferença que Python não tem

Em Python, `'a'` e `"a"` são a mesma coisa. **Em Ruby, não:**


```
"Olá, #{nome}"      # interpola
'Olá, #{nome}'      # imprime literalmente #{nome}
```

Aspas simples são texto cru. Use aspas duplas por padrão; simples quando o texto tiver `#{` ou muitas barras invertidas.

Números: a pegadinha da divisão

```
7 / 2      # 3.5    em Python 3
7 // 2     # 3
```

```
7 / 2      # 3      - inteiro dividido por inteiro dá inteiro
7.0 / 2    # 3.5
7.fdiv(2)  # 3.5
```

Python 3 mudou esse comportamento; Ruby não. É a origem de um bug silencioso quando se calcula média ou porcentagem.

Condicionais e `case`

```
# Python
if x == 1:
    ...
elif x == 2:
    ...
else:
    ...

match x:                                # Python 3.10+
    case 1: ...
    case 2: ...
```

```
# Ruby
if x == 1
    ...
elsif x == 2
    ...
else
    ...
end

case x
when 1 then ...
when 2 then ...
else ...
end
```

O `then` permite a linha única. O `case` de Ruby também aceita faixa, classe e regex:

```
case idade
when 0..12 then "criança"
when 13..17 then "adolescente"
else
  "adulto"
end
```

E, ao contrário de `switch` em C ou Java, **não existe** `break`: cada `when` já para sozinho.

Isso aparece no `SessionsController`:

```
case session_params[:client]
when "mobile" then response.headers["Authorization"] = "Bearer #{token}"
when "web" then set_auth_cookie(token)
end
```

Ternário e loops

```
mensagem = ativo ? "sim" : "não"      # em Python: "sim" if ativo else "não"

while restam > 0 ... end
until pronto ... end                  # o contrário do while – não existe em Python
loop do ... break if pronto ... end
```

`puts`, `print` e `p`

Ruby	Faz
<code>puts x</code>	imprime com quebra de linha. É o <code>print()</code> do dia a dia.
<code>print x</code>	imprime sem quebra de linha
<code>p x</code>	imprime a representação (com aspas, colchetes) e devolve o objeto

`p` é o que você usa para depurar: `p usuario` mostra o objeto inteiro, e como ele devolve o valor, dá para enfiar no meio de uma expressão sem quebrar nada.

`require` × `import`

```
import json          # Python
from pathlib import Path
```

```
require "json"        # Ruby: biblioteca ou gem
require_relative "helper" # arquivo ao lado, caminho relativo
```

`require` carrega uma vez só e devolve `false` se já estava carregado. **Não** traz nomes para o escopo como o `from x import y`: depois do `require`, você usa `JSON.parse` com o nome completo.

Dentro do Rails você nunca escreve `require`. O Zeitwerk carrega a classe pelo nome do arquivo. Fora do Rails, num script solto, o `require` volta a ser necessário.

Splat

```
def f(*args, **kwargs): ...
```

```
def f(*args, **opts); end
```

```
valores = [1, 2, 3]
```

```
f(*valores) # espalha o array nos argumentos
```

Estruturas de dados

Python	Ruby
<code>[1, 2, 3]</code>	<code>[1, 2, 3]: Array</code>
<code>{"a": 1}</code>	<code>{ a: 1 }: Hash</code> , chave símbolo
<code>{"a": 1}["a"]</code>	<code>{ a: 1 }[:a]</code>
<code>(1, 2)</code> tupla	não existe (use array congelado)
<code>{1, 2}</code> set	<code>Set.new([1, 2])</code>
não tem	<code>:simbolo</code>

```
usuario = { nome: "Ana", role: :admin }
usuario[:nome] # "Ana"
usuario.fetch(:idade, 0) # 0 – como dict.get com padrão
```

Símbolos

Um símbolo é uma **etiqueta**: um texto usado como nome, não como conteúdo. O mesmo símbolo é sempre o mesmo objeto na memória, o que o torna barato de comparar.

```
"admin" == "admin" # true, mas são dois objetos diferentes
:admin == :admin   # true, e é o mesmo objeto
```

Regra prática: conteúdo que o usuário lê → string. Nome de coisa no código → símbolo.

Blocos, o coração do Ruby

Um bloco é código que você entrega para um método executar. Em Python isso é o `lambda`, usado de vez em quando. Em Ruby é o caminho normal.

```
# Python
for u in usuarios:
    print(u.nome)

nomes = [u.nome for u in usuarios]
ativos = [u for u in usuarios if u.ativo]
total = sum(u.pontos for u in usuarios)
```

```
# Ruby
usuarios.each do |u|
  puts u.nome
end

nomes = usuarios.map { |u| u.nome }
ativos = usuarios.select { |u| u.ativo? }
total = usuarios.sum { |u| u.pontos }
```

Chaves quando cabe numa linha, `do ... end` quando não cabe.

Bloco, proc e lambda

Bloco não é objeto: ele existe só na chamada do método. Quando você quer **guardar** o código numa variável ou passar adiante, ele vira `Proc` ou `lambda`.

```
dobro = lambda x: x * 2      # Python
dobro(3)
```

```
dobro = ->(x) { x * 2 }      # Ruby: a "seta" é um lambda
dobro.call(3)
dobro.(3)                    # açúcar
```

A seta `->` é o que você vai ver no código deste projeto:

```
scope :recentes, -> { order(occurred_at: :desc) }
validate :password_meets_policy, if: -> { password.present? }
```

Nos dois casos o Rails guarda aquele pedaço de código para executar **depois**: na hora da consulta ou na hora da validação. Por isso precisa ser um objeto, e não um bloco.

`->` e `Proc.new` são quase a mesma coisa. A diferença que importa: o `lambda` confere o número de argumentos e o `return` dele volta só do `lambda`. O `Proc` é relaxado nos dois pontos. Na dúvida, use `->`.

Equivalências

Python	Ruby
list comprehension	<code>.map</code>
<code>filter</code> / comprehension com <code>if</code>	<code>.select</code> (e <code>.reject</code> para o inverso)
<code>functools.reduce</code>	<code>.reduce</code> / <code>.inject</code>
<code>any()</code> / <code>all()</code>	<code>.any?</code> / <code>.all?</code>
<code>sorted(x, key=...)</code>	<code>x.sort_by { ... }</code>
<code>enumerate</code>	<code>.each_with_index</code>
<code>zip</code>	<code>.zip</code>
<code>next(x for x in l if ...)</code>	<code>.find { ... }</code>

Argumentos nomeados

```
# Python
def login(email, password, client="mobile"): ...
login(email="a@b.c", password="x")
```

```
# Ruby – os dois-pontos DEPOIS do nome tornam obrigatório nomear na chamada
def login(email:, password:, client: "mobile")
  end

login(email: "a@b.c", password: "x")
```

Sem os dois-pontos, é argumento posicional como em Python. Com eles, quem chama **tem** que nomear, e a ordem deixa de importar.

O atalho do Ruby 3.1

Quando a variável tem o mesmo nome da chave, você omite o valor:

```
email = "a@b.c"
password = "x"

login(email:, password:)      # em vez de login(email: email, password: password)
```

Isso aparece em **todo service do projeto**:

```
Result.new(success?: true, user:)    # user: user
```

Não é erro de digitação.

Struct

Quando você só quer agrupar valores, sem comportamento:

```
Result = Struct.new(:success?, :user, :errors, keyword_init: true)

r = Result.new(success?: true, user: ana)
r.success?    # true
r.user        # ana
```

`keyword_init: true` obriga a nomear na criação. É o `namedtuple` / `dataclass` do Python. Todo service deste projeto devolve um `Struct` desses.

Exceções

Python	Ruby
<code>try</code>	<code>begin</code>
<code>except</code>	<code>rescue</code>
<code>finally</code>	<code>ensure</code>
<code>raise</code>	<code>raise</code>
<code>Exception</code> (base para capturar)	<code>StandardError</code>

```
try:
    arriscado()
except ValueError as e:
    print(e)
finally:
    limpar()
```

```
begin
  arriscado
rescue ArgumentError => e
  puts e.message
ensure
  limpar
end
```

Dentro de um método, o `begin` é implícito. Dá para escrever só o `rescue` no fim:

```
def call
  arriscado
rescue StandardError => e
  Rails.error.report(e)
  nil
end
```

Criando o seu tipo de erro:

```
class InvalidToken < StandardError; end

raise InvalidToken if token_ruim?
```

Herde sempre de `StandardError`, nunca de `Exception`. `rescue` sem classe captura `StandardError`; capturar `Exception` pega até `Ctrl+C` e erro de falta de memória.

`raise` sem argumento, dentro de um `rescue`, relança a exceção atual.

Atalhos de literal

```
%w[mobile web]      # => ["mobile", "web"]    – array de strings
%i[name email]       # => [:name, :email]    – array de símbolos
```

Só economizam aspas e vírgulas. Aparecem em constantes por todo o projeto:

```
CLIENTS = %w[mobile web].freeze
REQUIRED_FIELDS = %i[name email password].freeze
```

`.freeze` congela o objeto: tentar alterar depois levanta erro. Em Ruby strings e arrays são mutáveis (diferente de Python), então constante sem `freeze` é constante só no nome.

Classes

```
class Usuario:
    def __init__(self, nome):
        self.nome = nome

    def saudacao(self):
        return f"Oi, {self.nome}"

    @property
    def admin(self):
        return self.role == "admin"
```

```
class Usuario
  attr_accessor :nome          # gera o getter e o setter

  def initialize(nome)
    @nome = nome              # @ marca variável de instância
  end

  def saudacao
    "Oi, #{@nome}"
  end

  def admin?
    role == "admin"
  end
end
```

Python	Ruby
<code>__init__</code>	<code>initialize</code>
<code>self.x</code>	<code>@x</code>
<code>self</code> como primeiro parâmetro	<code>self</code> implícito
<code>@property</code>	método normal (não precisa de parênteses)
<code>@staticmethod</code>	<code>def self.metodo</code>
<code>_privado</code> por convenção	<code>private</code> de verdade
herança múltipla	módulos (<code>include</code>)

? e !

```
user.admin?    # devolve true/false
user.save      # devolve false se falhar
user.save!     # estoura ActiveRecord::RecordInvalid se falhar
```

Convenção, não regra da linguagem. Mas todo mundo segue, e o Rails inteiro depende dela.

Módulos e mixins

Ruby não tem herança múltipla. Tem módulo: um saco de métodos que você inclui.

```
# Python: mixin por herança múltipla
class Usuario(Base, VerificavelPorEmail, RecuperaSenha):
    pass
```

```
# Ruby: include
class Usuario < ApplicationRecord
  include EmailVerifiable
  include PasswordResetable
end
```

```
module EmailVerifiable
  extend ActiveSupport::Concern

  def email_verificado?
    email_verified_at.present?
  end
end
```

No Rails, um módulo desses vive em `app/models/concerns/` e se chama **concern**. É o que impede o `User` de virar um arquivo de 800 linhas.

Dependências e ferramentas

Python	Ruby
<code>pip install x</code>	<code>gem install x</code>
<code>requirements.txt</code>	<code>Gemfile</code>
<code>pip freeze > requirements.txt</code>	<code>Gemfile.lock</code> (gerado sozinho, sempre versionado)
<code>venv</code> / <code>virtualenv</code>	<code>bundler</code> isola por projeto
<code>pyenv</code>	<code>mise</code> / <code>rbenv</code>
<code>python -m x</code>	<code>bundle exec x</code>
<code>pytest</code>	<code>minitest</code> (padrão do Rails) ou <code>rspec</code>
<code>black</code> / <code>ruff</code>	<code>rubocop</code>
<code>mypy</code>	não há equivalente padrão: Ruby é dinâmica de ponta a ponta
<code>python</code> (REPL)	<code>irb</code>

Rails × Django

Django	Rails
<code>models.py</code> com classes <code>Model</code>	<code>app/models/</code> , um arquivo por classe
<code>makemigrations</code> / <code>migrate</code>	<code>rails g migration</code> / <code>rails db:migrate</code>
<code>urls.py</code> , escrito à mão	<code>config/routes.rb</code>
<code>views.py</code>	<code>app/controllers/</code>
<code>serializers.py</code> (DRF)	<code>app/serializers/</code> (feito à mão neste projeto)
Django ORM: <code>User.objects.filter(...)</code>	ActiveRecord: <code>User.where(...)</code>
<code>manage.py</code>	<code>bin/rails</code>
<code>settings.py</code>	<code>config/application.rb</code> + <code>config/environments/</code>
<code>.env</code> + <code>django-environ</code>	<code>Rails.application.credentials</code> + ENV
<code>python manage.py shell</code>	<code>bin/rails console</code>

Consultas lado a lado

```
User.objects.filter(role="admin")
User.objects.get(id=1)
User.objects.filter(email__icontains="ufop").order_by("-created_at")[:10]
User.objects.count()
```

```
User.where(role: "admin")
User.find(1)
User.where("email ILIKE ?", "%ufop%").order(created_at: :desc).limit(10)
User.count
```

Pegadinhas de quem vem do Python

Pegadinha	O que acontece
<code>0</code> e <code>""</code> são verdadeiros em Ruby	Só <code>nil</code> e <code>false</code> são falsos. <code>if 0</code> executa.
<code>puts</code> devolve <code>nil</code>	Não use o retorno de <code>puts</code>
Parênteses são opcionais	<code>user.save</code> e <code>user.save()</code> são a mesma coisa
<code>=</code> no fim do nome define setter	<code>def nome=(valor)</code> habilita <code>obj.nome = "x"</code>
Strings são mutáveis	Diferente de Python. Use <code>.freeze</code> em constantes.
<code>7 / 2</code> dá <code>3</code> , não <code>3.5</code>	Python 3 mudou isso; Ruby não. Use <code>7.0 / 2</code> ou <code>7.fdiv(2)</code> .
<code>'texto #{x}'</code> não interpola	Só aspas duplas interpolam.
<code>case</code> não precisa de <code>break</code>	Cada <code>when</code> já para sozinho.
<code>and</code> / <code>or</code> existem, mas têm precedência diferente de <code>&&</code> / <code>\ \ </code>	Use <code>&&</code> e <code>\ \ </code> sempre
Não existe <code>elif</code>	É <code>elsif</code>
Range com <code>..</code> inclui o fim, com <code>...</code> não	<code>(1..3).to_a</code> → <code>[1,2,3]</code> ; <code>(1...3).to_a</code> → <code>[1,2]</code>

As versões que usamos

Tudo o que aparece na capacitação, com a versão exata. Se algo se comportar diferente do que a apostila descreve, **confira aqui primeiro**: quase sempre é diferença de versão.

Os números foram tirados do próprio projeto (`.ruby-version`, `Gemfile.lock`, `compose.yaml` e `Dockerfile`), então eles são o que roda de verdade, e não o que se pretendia rodar.

Na sua máquina

Programa	Versão	Para quê	Onde instalar
Ruby	<code>3.4.9</code>	a linguagem	<code>00-preparacao.md</code> , via <code>mise</code>
Rails	<code>8.1.3</code>	o framework	<code>gem install rails -v 8.1.3</code>
mise	a mais recente	instala e troca a versão de Ruby por projeto	<code>curl https://mise.run \ sh</code>
Docker	24 ou mais novo	roda o Postgres, e depois a sua API	Docker Desktop, ou <code>docker-ce</code>
Git	2.30 ou mais novo	versionamento	pelo gerenciador do sistema
GitHub CLI (<code>gh</code>)	a mais recente	criar o repositório e cadastrar segredos	<code>apt install gh</code> / <code>brew install gh</code>
OpenSSH (cliente e servidor)	a do sistema	o Kamal chega no servidor por SSH	<code>apt install openssh-server</code>
OpenSSL	3.x	gera o certificado da Aula 4	já vem no sistema
VS Code	a mais recente	editor	code.visualstudio.com
Insomnia	a mais recente	montar requisições sem navegador	insomnia.rest
WSL2	Ubuntu 24.04	só Windows: é o Linux onde você trabalha	<code>wsl --install -d Ubuntu-24.04</code>

A versão do Ruby não é sugestão. O `.ruby-version` do projeto tem `3.4.9`, e o `mise` troca sozinho ao entrar na pasta. Rodar noutra versão funciona quase sempre, e o "quase" costuma aparecer no pior momento.

As gems do projeto

O `Gemfile` declara o que o projeto usa; o `Gemfile.lock` trava a versão exata de tudo, inclusive das dependências das dependências. É por isso que ele vai para o Git: é a garantia de que a sua máquina e o servidor rodam o mesmo código.

Gem	Versão	Para quê	Aula
<code>rails</code>	<code>8.1.3.1</code>	o framework inteiro	1
<code>pg</code>	<code>1.6.3</code>	driver do PostgreSQL	1
<code>puma</code>	<code>8.0.2</code>	o servidor web	1
<code>bcrypt</code>	<code>3.1.22</code>	hash de senha, via <code>has_secure_password</code>	2
<code>jwt</code>	<code>3.2.0</code>	assina e verifica o token de sessão	3
<code>rack-cors</code>	<code>3.0.0</code>	libera o navegador a chamar a API de outra origem	3
<code>letter_opener</code>	<code>1.10.0</code>	abre o e-mail no navegador, em desenvolvimento	3
<code>solid_cache</code>	<code>1.0.10</code>	cache no próprio Postgres; guarda a denylist do logout	3
<code>solid_queue</code>	<code>1.6.0</code>	fila de jobs no próprio Postgres	3
<code>kamal</code>	<code>2.12.0</code>	o deploy	4
<code>thruster</code>	a do lock	proxy HTTP que acompanha o Rails 8	4
<code>bootsnap</code>	a do lock	acelera o boot da aplicação	1
<code>brakeman</code>	<code>8.0.5</code>	análise estática de segurança, no CI	3
<code>bundler-audit</code>	a do lock	procura CVE conhecida nas gems, no CI	3
<code>rubocop-rails-omakase</code>	<code>1.1.0</code>	o estilo padrão do Rails, no CI	3
<code>debug</code>	a do lock	o depurador	1

Para ver a versão exata de qualquer uma, no seu projeto:

```
bundle info bcrypt
bundle list | grep jwt
```

Nas imagens Docker

Imagem	Versão	Onde
<code>postgres</code>	16	<code>compose.yaml</code> em desenvolvimento, e o accessory do Kamal em produção
<code>ruby</code>	<code>3.4.9-slim</code>	base do <code>Dockerfile</code> da aplicação

O Postgres é o mesmo nos dois lugares, de propósito. Desenvolver num banco e publicar noutro é a forma mais barata de descobrir um bug só depois do deploy.

Serviços de fora

Serviço	O que usamos	Custa?
GitHub	repositório, Actions (CI) e o <code>ghcr.io</code> (registry de imagens)	não, em repositório público ou com a conta de estudante
SMTP (Resend ou similar)	entrega os e-mails de verificação e recuperação	plano gratuito serve
Azure	só no apêndice de nuvem	US\$100 de crédito pela Azure for Students, sem cartão
Cloudflare	só no apêndice: DNS e certificado	não

Conferindo tudo de uma vez

Cole no terminal, dentro do seu projeto:

```
ruby -v                # ruby 3.4.9
bin/rails -v           # Rails 8.1.3.1
docker -v
docker compose version
git --version
gh --version
openssl version        # OpenSSL 3.x
ssh -V                 # OpenSSH_9.x
bundle exec kamal version # 2.12.0
ssh "$USER"@127.0.0.1 'echo ok' # ok (o servidor da Aula 4)
```

Se algum deles não responder, a saída está em `00-preparacao.md` ou em `troubleshooting.md`.

Se você estiver lendo isto no futuro

Versões envelhecem, e este material foi escrito com as de cima. Duas coisas envelhecem mais rápido que as outras:

- **O Rails.** A cada versão maior mudam padrões e geradores. `rails new` numa versão diferente pode gerar um projeto com outra cara.

- **O Kamal.** Ele é novo e ainda muda. O `deploy.yml` deste material é da linha **2.x**; a 1.x usava outro formato para proxy e segredos.

O resto (HTTP, SQL, bcrypt, JWT, Docker, SSH, TLS) muda devagar, e é a maior parte do que você está aprendendo aqui.

Glossário

Termos que aparecem nas quatro aulas, em ordem alfabética.

Accessory — Container que o Kamal gerencia mas não faz deploy junto com a aplicação. O Postgres é um. `kamal deploy` não sobe accessory; use `kamal accessory boot|reboot`.

ActiveRecord — O ORM do Rails. Cada classe é uma tabela, cada objeto é uma linha. Equivalente ao Django ORM.

API — *Application Programming Interface*. A porta de entrada do sistema para outros programas. Aqui, uma API REST que responde JSON.

Associação — Ligação entre dois models. `has_many` no lado "um", `belongs_to` no lado "muitos" — que é quem carrega a chave estrangeira.

Base64 — Forma de escrever bytes usando só letras e números, para caber em texto. **Não é criptografia**: não tem chave e qualquer um desfaz. O JWT é Base64.

Bcrypt — Função de hash feita para senha. Lenta de propósito, com fator de custo ajustável e salt aleatório por senha. Ver `02-activerecord.md`.

Bearer — O esquema do cabeçalho `Authorization: Bearer <token>`. "Portador": quem apresenta o token é tratado como o dono dele.

`before_action` — Filtro que roda antes da action do controller. Se ele renderizar algo, a action não é chamada.

Brakeman — Análise estática de segurança para Rails. Procura SQL injection, mass assignment, redirect aberto e afins.

Bundler — O gerenciador de dependências do Ruby. Lê o `Gemfile`, resolve versões e escreve o `Gemfile.lock`.

Autoassinado — Certificado que você mesmo assinou. Criptografa igual a qualquer outro; o que falta é alguém em quem o cliente já confie tendo assinado. É por isso que o navegador reclama.

CA (Autoridade Certificadora) — Quem assina certificados. O navegador nasce com uma lista de CAs em que confia; confiar na CA é confiar em quem ela assinou. Para uma CA pública assinar, você tem que provar que o domínio é seu.

Certificado — Documento que afirma "esta chave pública pertence a este domínio", assinado por uma CA.

Chave estrangeira — Coluna que aponta para o `id` de outra tabela (`login_events.user_id`). Com `foreign_key: true`, o banco recusa linha órfã.

CI/CD — *Continuous Integration / Continuous Deployment*. CI roda testes e verificações a cada push; CD publica automaticamente o que passou.

Concern — Módulo Ruby que uma classe inclui para ganhar um conjunto de métodos. É o mixin do Rails. Vive em `app/models/concerns/` ou `app/controllers/concerns/`.

Container — Uma instância rodando de uma imagem Docker. Descartável.

Cookie — Par nome=valor que o servidor manda no `Set-Cookie` e o navegador reenvia sozinho em toda requisição àquele site. É a memória que o HTTP não tem, colada por fora.

CORS — *Cross-Origin Resource Sharing*. A regra do navegador que impede uma página de um domínio chamar uma API de outro sem permissão explícita. Não se aplica a app nativo.

Credentials — Arquivo YAML criptografado do Rails (`config/credentials.yml.enc`), decriptado pela `master.key`.

Criptografia assimétrica — Duas chaves que se completam: a pública se espalha, a privada nunca sai da máquina. Base do SSH e do TLS.

CSRF — *Cross-Site Request Forgery*. Site malicioso dispara requisição para a sua API, e o navegador anexa o cookie da vítima. Mitigado com `same_site`.

CVE — *Common Vulnerabilities and Exposures*. O identificador público de uma vulnerabilidade conhecida. `bundler-audit` procura CVE nas suas gems.

Denylist — Lista de tokens revogados. No nosso logout, indexada pelo `jti` do JWT.

Digest — O resultado de passar um valor por uma função de hash. `password_digest` guarda o digest, nunca a senha.

DNS — A agenda telefônica da internet: traduz um nome (`api.exemplo.com`) no IP da máquina. A resposta fica em cache pelo TTL.

Docker — Empacota aplicação e dependências numa imagem que roda igual em qualquer lugar.

Dockerfile — A receita da imagem. O nosso é multi-stage: um estágio compila, o final só copia o resultado.

Enumeração de usuários — Ataque em que respostas diferentes (mensagem, status ou tempo) revelam quem tem conta no sistema. Combatido com respostas idênticas e tempo constante.

Fixture — Dados de teste em YAML, carregados antes de cada teste e limpos depois.

Gem — Biblioteca Ruby. O equivalente do pacote do pip.

ghcr.io — GitHub Container Registry. Onde a imagem Docker fica hospedada.

`/etc/hosts` — Arquivo que mapeia nome para IP na sua máquina. O sistema consulta ele **antes** do DNS. É como `seunome.test` acha a VM sem existir em DNS nenhum.

Handshake (TLS) — A negociação inicial: o servidor apresenta o certificado, o cliente valida a cadeia, e os dois combinam uma chave temporária.

Health check — Rota que responde 200 se a aplicação está de pé. A nossa é `/up`. O Kamal só troca o tráfego para o container novo depois que ela passa.

Imagem — A receita congelada: sistema, runtime, dependências e código. Imutável.

IP — O endereço da máquina na rede (`57.156.65.151`). `127.0.0.1` é sempre "esta máquina aqui".

jti — *JWT ID*. Identificador único de um token, usado para revogá-lo individualmente.

JWT — *JSON Web Token*. Cabeçalho, payload e assinatura, em Base64, separados por ponto. O payload é **público**; a assinatura só garante que ninguém o alterou.

Kamal — Ferramenta de deploy com Docker via SSH, feita pela equipe do Rails. Zero-downtime, sem agente na máquina.

Mass assignment — Vulnerabilidade em que o cliente manda um campo que não devia poder mandar (ex.: `role: admin`). Evitada com strong parameters.

Middleware — Camadas que a requisição atravessa antes de chegar ao controller. CORS, cookies e rate limit moram aí. `bin/rails middleware` lista a pilha.

Migration — Mudança no banco, versionada em código, que roda uma vez em cada máquina.

Minitest — O framework de teste que vem com o Rails.

mise — Gerenciador de versões de runtime. O `pyenv` do mundo Ruby.

MVC — *Model-View-Controller*. Model = dados e regras; View = a saída (aqui, JSON); Controller = recebe a requisição e decide.

N+1 — Carregar uma lista e depois consultar o banco item a item. 100 registros viram 101 consultas. Resolve-se com `includes`.

NSG — *Network Security Group*. O firewall da Azure, fora da máquina, com regras de entrada e saída por porta e origem. É o equivalente na nuvem do `ufw`, e vem antes dele: o pacote nem chega.

OIDC — *OpenID Connect*. Permite o GitHub Actions autenticar na nuvem com um token de curta duração, sem guardar segredo de longa duração. Aparece no apêndice de nuvem.

OOM killer — O mecanismo do kernel Linux que mata processos quando a RAM acaba. É por isso que a VM tem swap.

Origin CA — Certificado emitido pelo Cloudflare para o trecho Cloudflare → seu servidor. Navegador não confia nele diretamente, e não precisa: o usuário vê o certificado público do Cloudflare.

PAT — *Personal Access Token*. Token do GitHub que substitui a senha em ferramentas de linha de comando. O do curso tem só `write:packages` e `read:packages`, para o `ghcr.io`.

ORM — *Object-Relational Mapping*. Traduz objetos em linhas de tabela.

OTP — *One-Time Password*. O código de 6 dígitos, válido por 15 minutos e uma vez só.

Payload — O miolo do JWT, onde ficam `sub`, `exp`, `jti`. Público.

Porta — Número de 1 a 65535 que identifica um programa dentro da máquina. 22 SSH, 80 HTTP, 443 HTTPS, 5432 Postgres, 3000 Rails em dev.

Postgres — O banco relacional que usamos em desenvolvimento e em produção.

Proxied (Cloudflare) — Nuvem laranja no DNS: o tráfego passa pelo Cloudflare antes de chegar à sua máquina, e o IP real não aparece no DNS.

Proxy reverso — Fica na frente do servidor: recebe na 443, termina o TLS e repassa para a aplicação em HTTP interno. O `kamal-proxy` é um.

Puma — O servidor web do Rails.

rack-attack — Middleware que bloqueia abuso por IP. Está no `seem-backend`, não no app do curso.

Registry — Repositório de imagens Docker. O "GitHub das imagens".

REST — Estilo de organizar a API: cada coisa é um recurso com um endereço, e o verbo HTTP diz o que fazer com ele.

Rollback — O banco desfazendo uma transação que falhou no meio. Também: `kamal rollback`, que volta para a versão anterior da imagem.

root — O usuário do Linux que pode tudo, sem confirmação. Você trabalha como usuário comum e chama `sudo` quando precisa.

RuboCop — O linter de Ruby. O `black` / `ruff` do mundo Python.

Salt — Valor aleatório misturado à senha antes do hash, para que senhas iguais gerem digests diferentes. O bcrypt gera um por senha, automaticamente.

Scope — Consulta com nome, definida no model e encadeável:

```
user.login_events.recentes.limit(5).
```

Serializer — Decide o que de um objeto vai para o JSON — e, principalmente, o que não vai.

Service object — Classe que representa um caso de uso (`Users::Register`). Tira a regra de negócio do controller.

Solid Cache / Solid Queue — Cache e fila do Rails 8, guardados no próprio Postgres. A denylist do logout usa o cache.

Strong parameters — Filtro que declara quais campos a requisição pode mandar. No Rails 8, `params.expect`.

Struct — Classe de uma linha para agrupar valores sem comportamento. O `namedtuple` / `dataclass` do Ruby. Todo service do projeto devolve um.

sub — *Subject*. O claim do JWT que diz de quem é o token — no nosso caso, o `user.id`.

Swap — Área em disco usada como extensão da RAM. Lenta, mas evita que o OOM killer mate a aplicação.

.test — Domínio de topo reservado para testes por RFC. Ninguém consegue registrar, então nunca colide com um site de verdade.

TLS — O túnel criptografado que embrulha o HTTP e o transforma em HTTPS. Mesmo protocolo, só que fechado.

Token version — Contador no `User` que vai dentro do JWT. Incrementar invalida todos os tokens daquele usuário de uma vez.

Transação — Bloco em que ou tudo acontece, ou nada acontece. `User.transaction do ... end`.

TTL — *Time To Live*. Por quanto tempo algo vale. Nosso código OTP tem TTL de 15 minutos; as entradas da denylist têm o TTL do que restava do token.

Variável de ambiente — Par nome=valor que o sistema entrega ao processo. Lida com `ENV["NOME"]`. É como configuração e segredo entram sem passar pelo código.

Volume (Docker) — Área de disco que sobrevive ao container. É o que faz o banco não sumir a cada deploy. **Não é backup.**

sshd — O servidor de SSH: o programa que fica escutando na porta 22 e atende quem chega. Na Aula 4 você instala um na sua própria máquina, e é por ele que o Kamal entra.

ufw — *Uncomplicated Firewall*. A casca amigável do firewall do Ubuntu. `ufw allow 22/tcp`. Aparece no apêndice de nuvem, onde a máquina está exposta.

VM — *Virtual Machine*. Um computador inteiro simulado em software: kernel, disco, rede, usuários. Uma VPS é uma dessas, alugada.

VPS — *Virtual Private Server*. Uma VM alugada, num datacenter, com IP público. Você tem acesso root e opera tudo. É onde o `seem-backend` roda, e é o assunto do apêndice de nuvem.

WSL2 — *Windows Subsystem for Linux*. Um Linux de verdade dentro do Windows. Obrigatório para quem desenvolve Rails no Windows.

XSS — *Cross-Site Scripting*. O atacante roda JavaScript dentro da sua página, geralmente porque você exibiu texto de usuário sem escapar. É contra isso que `httponly` protege o token.

Zeitwerk — O autoloader do Rails. Descobre o arquivo pelo nome da classe:

`Api::V1::StatusController` → `app/controllers/api/v1/status_controller.rb`. É o que dispensa o `require`.

Índice — Estrutura no banco que acelera busca. Com `unique: true`, vira restrição: o banco recusa duplicata mesmo com duas requisições simultâneas.

Troubleshooting

Erros que acontecem de verdade, e a saída de cada um. Os da parte de infraestrutura vieram do histórico real de deploy do `seem-backend`.

Ambiente

`mise: command not found`

A linha de `activate` não foi para o arquivo certo. Confira o fim do `~/.bashrc` (ou `~/.zshrc`) e rode `exec bash`.

A instalação do Ruby falha compilando

Faltou dependência de build. Rode o bloco de `apt install` / `brew install` de `00-preparacao.md` e tente `mise install ruby@3.4.9` de novo.

`gem install rails` reclama de permissão

Você está usando o Ruby do sistema, não o do mise. Confira:

```
which ruby # tem que apontar para dentro de ~/.local/share/mise
```

`permission denied` ao rodar `docker`

Você não está no grupo `docker`:

```
sudo usermod -aG docker $USER
```

E **saia e entre de novo na sessão**. Reabrir o terminal não basta.

No Windows, `docker` não aparece dentro do Ubuntu

Docker Desktop → Settings → Resources → WSL Integration → habilite a distro `Ubuntu-24.04`.

Banco

`address already in use` ao subir o compose

Já existe um Postgres na sua máquina ocupando a 5432.

```
DB_PORT=5433 docker compose up -d
export DB_PORT=5433          # e exporte no shell onde roda o Rails
```

Para achar quem ocupa: `ss -ltnp | grep 5432`.

`PG::ConnectionBad: could not connect to server`

Nesta ordem:

```
docker compose ps          # o container está "healthy"?
docker compose logs db     # o que ele diz?
echo $DB_PORT              # bate com a porta publicada?
```

`PG::UndefinedTable` ou `PendingMigrationError`

Faltou migrar:

```
bin/rails db:migrate
```

Quero começar do zero

```
bin/rails db:reset          # apaga, recria, carrega o schema e roda os seeds
```

Testes

Um teste passa sozinho e falha junto com os outros

Vazamento de estado entre testes. Suspeitos, em ordem:

1. um `Rails.cache` compartilhado (o `test_helper` troca por `MemoryStore` a cada teste);
2. `ActionMailer::Base.deliveries` não limpo;
3. dependência de ordem: o Minitest embaralha de propósito, e isso é uma qualidade.

Rode com a mesma semente para reproduzir: `bin/rails test --seed 1234`.

O teste de logout passa mas a revogação não funciona

Em teste, o `Rails.cache` padrão é o `null_store`: não guarda nada, e `revoked?` sempre devolve `false`. O `test_helper.rb` troca por `MemoryStore` justamente por isso. Se você recriou o arquivo, recoloque o `setup`.

`Bullet::Notification::UnoptimizedQueryError`

Consulta N+1: você carregou uma coleção e acessou uma associação item a item. Adicione `includes`. (O `seem-backend` usa o Bullet; o app do curso não.)

Autenticação

Sempre 401, mesmo com o token certo

Cheque, nesta ordem:

1. o cabeçalho é exatamente `Authorization: Bearer <token>`, com o espaço;
2. o token não foi revogado (você fez logout com ele?);
3. o `token_version` do banco bate com o `ver` do token: trocar a senha incrementa a versão;
4. o `JWT_SECRET` é o mesmo que emitiu o token. Reiniciar o app em dev sem `JWT_SECRET` definido usa o `secret_key_base`, que é estável, mas em produção um segredo diferente invalida tudo.

Decodifique o token em jwt.io e olhe o `exp` e o `ver`.

O e-mail não chega em desenvolvimento

Ele não é enviado: o `letter_opener` abre no navegador. Se você está rodando `curl` e não vendo nada, o arquivo está em `tmp/letter_opener/`.

`ActionController::ParameterMissing (400)`

Faltou um campo obrigatório no corpo. O `params.expect` exige todas as chaves declaradas. Confira o JSON e o `Content-Type: application/json`.

422 no cadastro sem dizer o motivo direito

Olhe `error.details` na resposta: cada item tem `field` e `message`.

GitHub

O CI ficou vermelho logo no primeiro push

O `rails new` já cria um `.github/workflows/ci.yml`, e ele roda sozinho a cada push. Nas Aulas 1 e 2 ele deve passar. Se ficar vermelho, abra o log em **Actions** e veja qual dos três jobs quebrou:

- **lint**: é o RuboCop. Rode `bin/rubocop -a` para corrigir o que dá sozinho.
- **test**: rode `bin/rails test` na sua máquina; o erro é o mesmo.
- **scan_ruby**: Brakeman ou uma gem com CVE. `bin/brakeman --no-pager` mostra o motivo.

Na Aula 4 esse arquivo é substituído pelo nosso, que roda os testes contra um Postgres de verdade.

`gh: command not found`

O GitHub CLI não está instalado. Volte ao passo 7 de `00-preparacao.md`.

`gh repo create` reclama de autenticação

```
gh auth status      # tem que dizer "Logged in to github.com"
gh auth login       # se não estiver
```

O SSH da sua máquina

Na Aula 4 o servidor é a sua própria máquina, e o Kamal chega nela por SSH.

`Connection refused` na porta 22

O `sshd` não está rodando.

```
sudo service ssh start      # Ubuntu, Debian, WSL2
sudo systemctl start sshd   # Fedora
```

No macOS, ligue em **Ajustes do Sistema** → **Geral** → **Compartilhamento** → **Sessão remota**.

Confira que ele está escutando:

```
ss -tlnp | grep :22      # Linux e WSL2
sudo lsof -i :22          # macOS
```

Funcionava, reiniciei o Windows, e parou

É o WSL2: sem o systemd habilitado, ele não sobe o `sshd` no boot.


```
sudo service ssh start
```

Para não repetir isso toda vez, habilite o systemd em `/etc/wsl.conf`:

```
[boot]
systemd=true
```

E depois, no PowerShell: `wsl --shutdown`.

Permission denied (publickey)

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/capacita ~/.ssh/authorized_keys
ssh -i ~/.ssh/capacita -o IdentitiesOnly=yes "$USER"@127.0.0.1
```

O SSH **recusa** usar chave com permissão frouxa, e recusa `authorized_keys` que outros possam escrever. Se persistir, confira que a chave pública realmente entrou:

```
cat ~/.ssh/authorized_keys
```

Too many authentication failures

O SSH ofereceu todas as chaves do seu agente e o servidor cortou antes de chegar na certa.

```
ssh -i ~/.ssh/capacita -o IdentitiesOnly=yes "$USER"@127.0.0.1
```

Host key verification failed

A chave do host mudou (reinstalou o `sshd`, ou trocou de máquina).

```
ssh-keygen -R 127.0.0.1
```

docker: command not found só pela SSH

Uma sessão SSH não interativa carrega um `PATH` menor que o seu terminal. Confira:

```
ssh -i ~/.ssh/capacita "$USER"@127.0.0.1 'which -a docker'
```

Se não achar, o Docker foi instalado de um jeito que só existe no seu shell interativo (Docker Desktop com integração WSL, por exemplo). Instale o pacote do sistema, ou acrescente o caminho ao `~/.profile` do usuário.

Deploy

`kamal config` reclama de variável faltando

Você esqueceu o `source .env`. Ele diz exatamente qual variável.

```
source .env && bundle exec kamal config
```

O `push` da imagem falha por cota ou armazenamento

Na conta grátis do GitHub, **pacote privado** tem uma cota de armazenamento pequena, e uma imagem Rails ocupa quase toda ela.

A saída é deixar o pacote público, o que é grátis e ilimitado. Isso **não** expõe o seu código: o repositório continua privado, e o que fica público é a imagem já compilada.

github.com/SEU-USUARIO?tab=packages → **automic-auth-api** → **Package settings** → **Change visibility** → **Public**

Se preferir manter privado, apague as versões antigas do pacote na mesma tela: cada deploy publica uma, e elas se acumulam.

`denied` ou `unauthorized` ao empurrar a imagem para o ghcr.io

O `KAMAL_REGISTRY_PASSWORD` tem que ser um PAT (classic) com `write:packages` e `read:packages`. Um token de granularidade fina (*fine-grained*) não serve para o ghcr.io.

E o `GHCR_USER` é o seu usuário do GitHub, em minúsculas: o registry não aceita maiúscula no nome da imagem.

`exec format error` ao subir o container

Arquitetura errada: você buildou para `amd64` e a VM é `arm64` (ou o contrário).

```
ssh -i ~/.ssh/capacita ubuntu@SEU_IP 'dpkg --print-architecture'
```

Ajuste `SERVER_ARCH` no `.env`, `source .env` de novo, e refaça o deploy.

Kamal: `Docker is not installed`

O primeiro deploy precisa ser `setup`, não `deploy`:

```
source .env && bundle exec kamal setup
```

A aplicação sobe mas não acha o banco

`kamal deploy` **não** sobe accessories.

```
kamal accessory boot db
```

Depois de mudar env de accessory, `boot` não basta: é `kamal accessory reboot db`.

O `curl` recusa o certificado, com o erro 60

Isso é o esperado, e é a Prática 6 da apostila. A frase exata muda com a versão do `curl`; o que não muda é o número 60 e a expressão *self-signed*. O túnel TLS subiu; o que faltou foi confiança.

```
curl --cacert tls/capacita-cert.pem https://seunome.test/api/v1/status
```

`curl` diz **Could not resolve host: seunome.test**

Falta a linha no `/etc/hosts`:

```
echo "SEU_IP seunome.test" | sudo tee -a /etc/hosts
getent hosts seunome.test
```

No Windows, o navegador consulta `C:\Windows\System32\drivers\etc\hosts`, não o do WSL2.

`curl` conecta mas devolve 404 do proxy

O `APP_HOST` do `.env` tem que ser **exatamente** o nome que você está chamando. O `kamal-proxy` roteia pelo cabeçalho `Host`: se o certificado é para `seunome.test` e o `APP_HOST` está `outro.test`, ele não entrega a ninguém.

`curl` dá **Connection refused** na 443

```
ssh -i ~/.ssh/capacita ubuntu@SEU_IP 'docker ps && sudo ufw status'
```

O `kamal-proxy` está na lista? A 443 está liberada no `ufw`?

SMTP dá timeout na VM

Da VM local, a saída costuma funcionar: se não funcionar, é o firewall da sua rede (algumas redes universitárias bloqueiam a 587).

Numa VPS é mais comum: vários provedores de nuvem bloqueiam a saída na porta 25, e alguns na 587. Teste a alternativa do seu provedor de e-mail (o Resend aceita 2587) e ajuste `SMTP_PORT` no `env.clear`.

O deploy passa mas o site responde 500

```
source .env && kamal app logs -f
```

Suspeitos comuns: migration pendente (`kamal app exec "bin/rails db:migrate"`),
`RAILS_MASTER_KEY` errada, ou um `ENV.fetch` sem default para uma variável que ninguém cadastrou.

Preciso voltar atrás agora

```
source .env && kamal rollback
```

Volta para a versão anterior da imagem, que ainda está na VM.

Apêndice: nuvem (Azure e Cloudflare)

Estes só acontecem no percurso de `apendice-azure.md`.

`NotAvailableForSubscription` ao criar a VM

O tamanho escolhido não está habilitado para a sua assinatura naquela região.

Assinaturas → **Uso + cotas** → **filtre por Compute** e escolha outra região ou outro tamanho. Pedir aumento de cota **não** resolve quando a oferta não está habilitada.

SSH dá timeout na VM da Azure

Quase sempre é o NSG:

1. o seu IP mudou? `curl -4 ifconfig.me` e compare com a regra;
2. a regra libera a porta 22, protocolo TCP, direção Entrada?
3. a prioridade da regra de permissão é **menor** que a de alguma negação?

Perdi o acesso depois de mexer no sshd, e não tenho sessão aberta

Sobrou o **Serial Console** do portal da Azure (menu da VM → Suporte + solução de problemas → Console Serial), que não passa pelo SSH.

OIDC: `AADSTS700213`

O *subject* da credencial federada não bate com o que o GitHub emite. Ele precisa ser, caractere por caractere:

```
repo:SEU-USUARIO/SEU-REP0:ref:refs/heads/main
```

Confira maiúsculas, o nome do repositório e a branch.

`az: command not found` ou falha transitória do Azure CLI

O `azure/login` às vezes falha por instabilidade. Rode o workflow de novo antes de investigar.

A regra temporária ficou no NSG

O passo de remoção tem `if: always()`, mas se o job for cancelado à força a regra pode sobrar. Apague na mão:

```
az network nsg rule delete \  
  --resource-group SEU_RG --nsg-name SEU_NSOG \  
  --name allow-github-actions-deploy-ssh
```

E confira depois de todo deploy que falhou de forma estranha.

Cloudflare 521 / 522

O Cloudflare não conseguiu falar com o seu servidor.

- a porta 443 está aberta no NSG?
- `docker ps` na VM mostra o `kamal-proxy` rodando?
- o registro A aponta para o IP certo?

Cloudflare 525 / 526

Falha no handshake TLS entre Cloudflare e a sua VM.

- **525**: o proxy não apresentou certificado. Os secrets `KAMAL_PROXY_SSL_*` estão cadastrados?
- **526**: o certificado é inválido para o modo Full (strict). Ele cobre o hostname exato? Um wildcard `*.capacita.exemplo.tech` cobre `seunome.capacita.exemplo.tech`, mas **não** cobre `x.y.capacita.exemplo.tech`.

Let's Encrypt falha atrás do Cloudflare

Esperado. Com o DNS proxied, o desafio HTTP-01 não chega como o Let's Encrypt espera. Use o certificado Origin CA: é a razão de ele existir.